



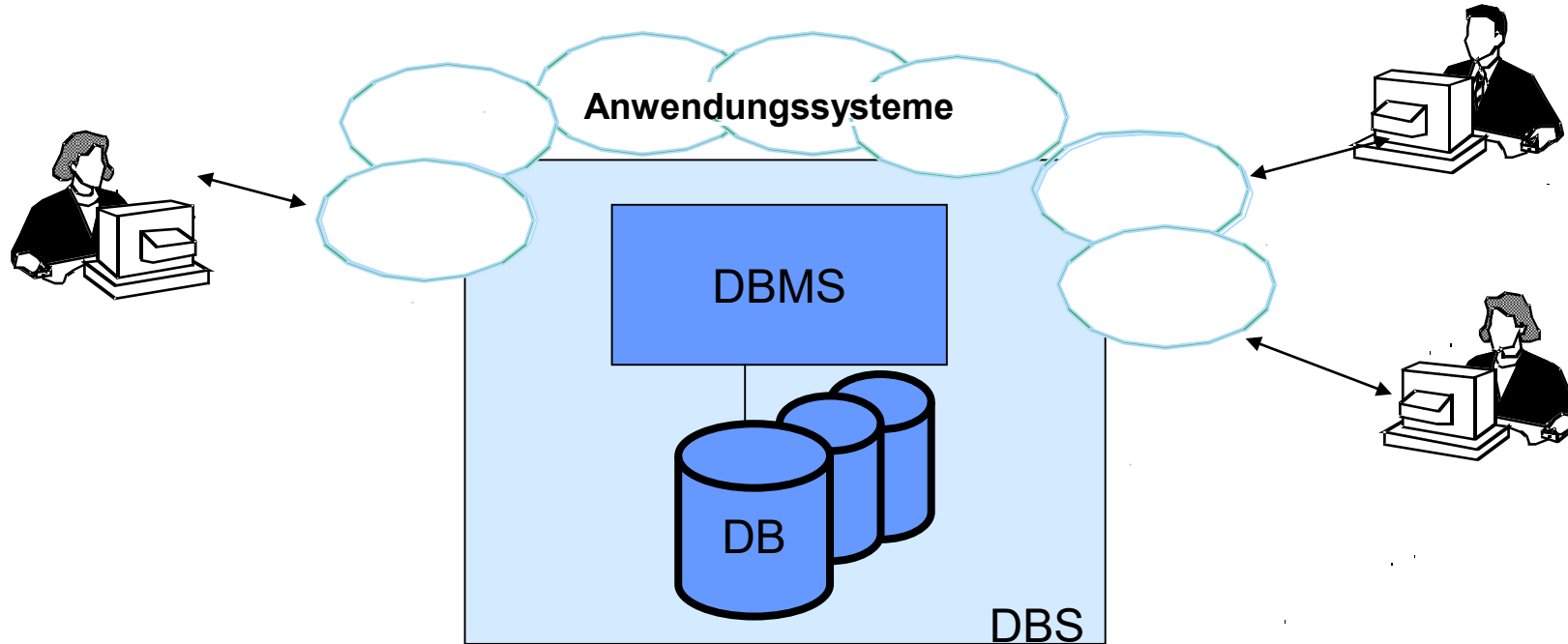
1. EINFÜHRUNG / GRUNDLAGEN VON DBS

- DBS vs. Dateisysteme
- Eigenschaften von DBS
- Datenmodelle
- Transaktionskonzept (ACID)
- Aufbau von DBS
 - Schemaarchitektur
 - Schichtenmodell
- Einsatzformen von DBS: OLTP vs. OLAP
- Historische Entwicklung
 - Datenmodelle
 - Architekturen



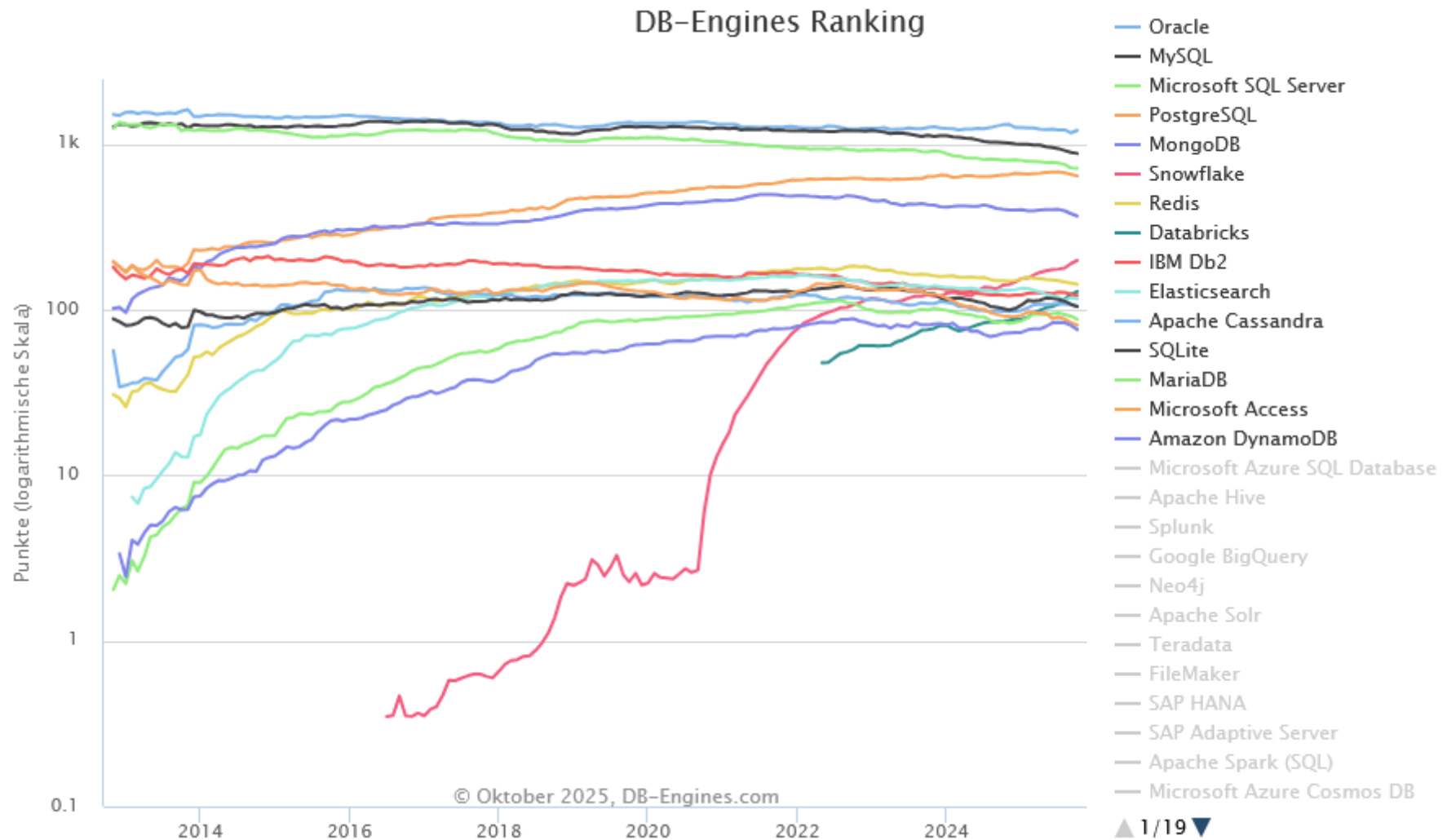
- System zur dauerhaften Speicherung und Verwaltung großer Datenmengen
- Realisierung datenintensiver Anwendungen
- typische Anwendungen:
 - Personaldatenbank
 - Kundenverwaltung
 - Bankanwendungen mit Konten und Zahlungsverkehr
 - Buchungssysteme/Warenverwaltung in Online-Shops
 - soziale Netzwerke (Nutzer, Aktivitäten ...)
 - Infektionsgeschehen
 -
- Daten sind wertvoll

DBS ALS KERN VON INFORMATIONSSYSTEMEN (IS)



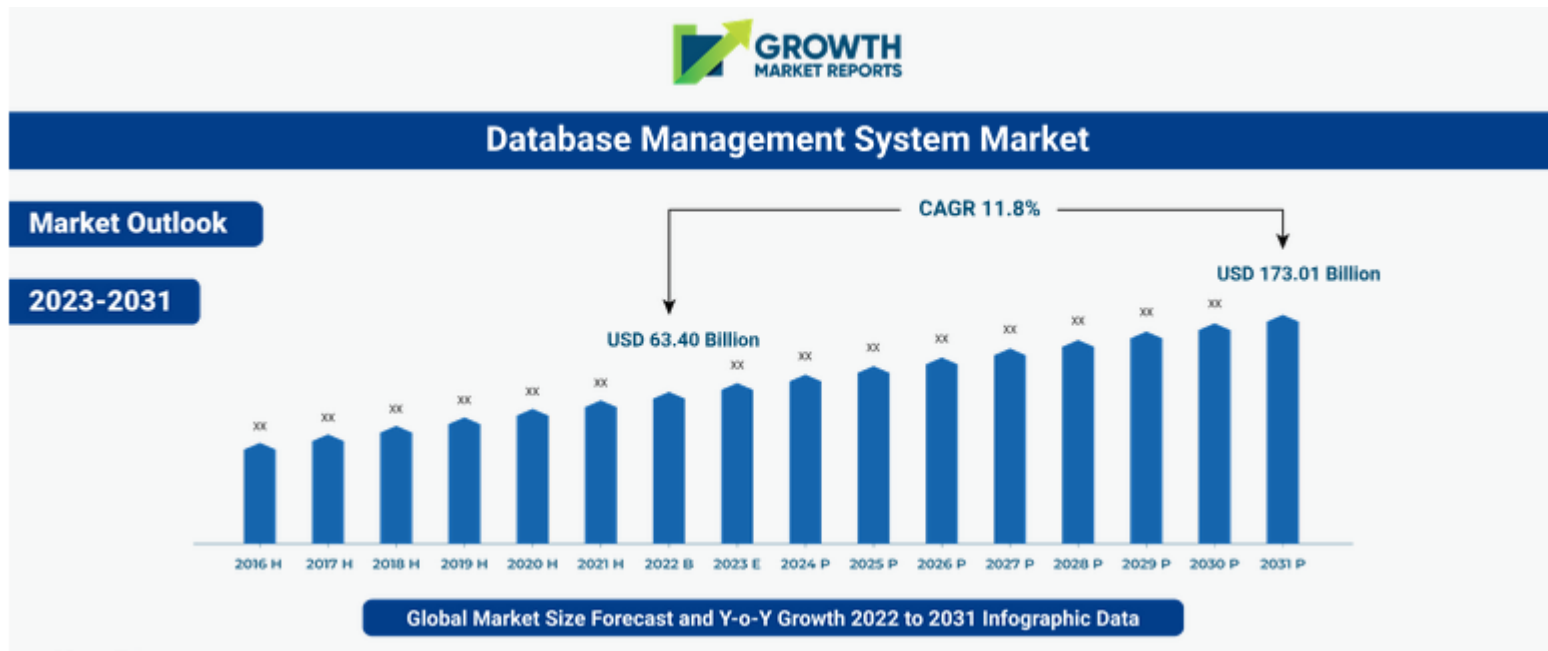
- IS = DBS + Anwendungssysteme + Benutzerschnittstellen
- DBS = DB + Datenbankverwaltungssystem (DBVS, DBMS)
 - DB: Menge der gespeicherten Daten
 - Datenbankverwaltungssystem: generisches Software-System zur Definition, Verwaltung, Verarbeitung und Auswertung der DB-Daten. Einsatz für unterschiedlichste Anwendungen

BELIEBTESTE DATENBANKSYSTEME 2025

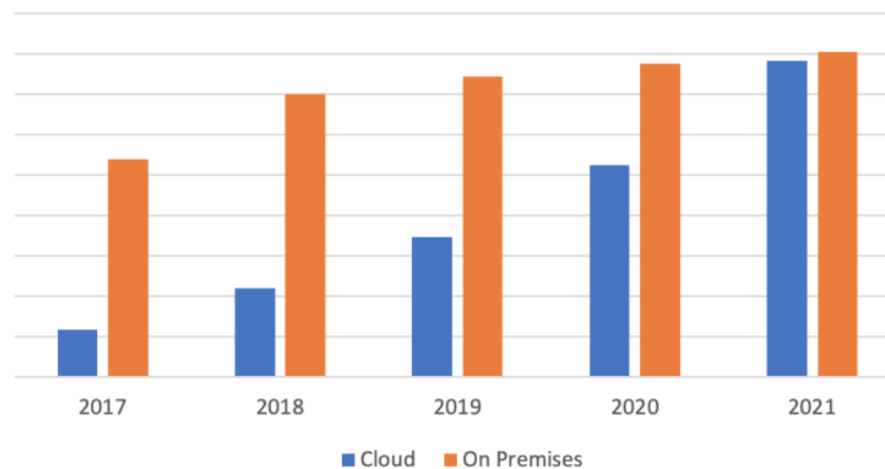


Quelle: https://db-engines.com/de/ranking_trend

DBMS-MARKT



Cloud and On Premises DBMS Revenue



source: Gartner.com, 2022

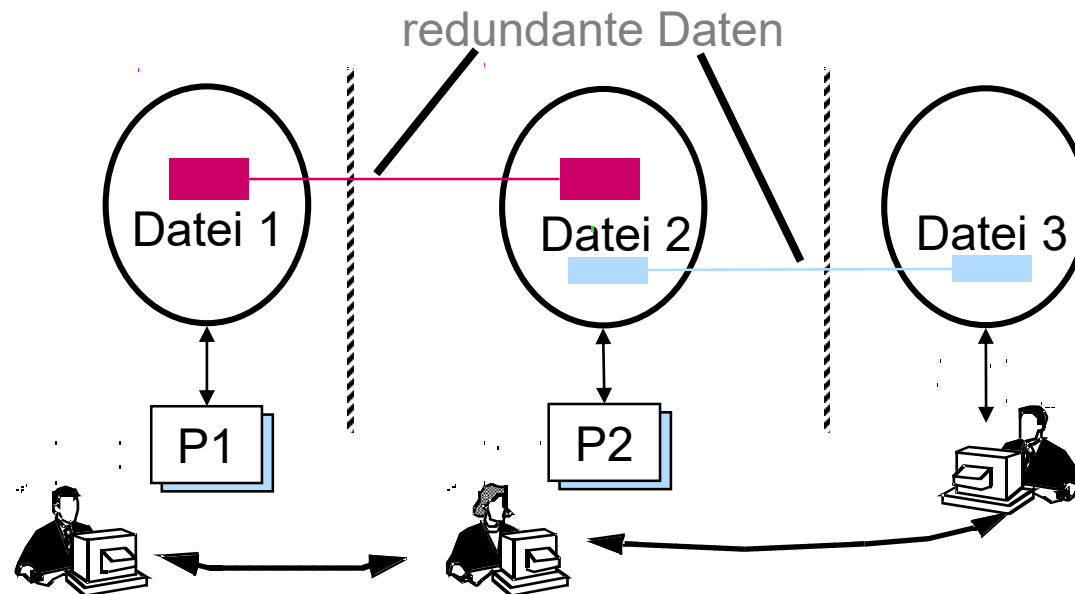
BEISPIELE FÜR INFORMATIONSSYSTEME



- Hochschulinformationssystem (Universitäts-DB)
 - Verwaltung von Studenten, Fakultäten, Professoren, Mitarbeitern
 - Studenten belegen Vorlesungen von Professoren und legen bei ihnen Prüfungen ab
 - Anwendungsvorgänge: Im/Exmatrikulation, Rückmeldung, Prüfungsverwaltung, Stundenplanerstellung, Raumplanung, etc.
- Datenbank eines Produktionsbetriebes
 - Verwaltung von Abteilungen und deren Beschäftigten
 - Produktdaten: Zusammensetzung von Endprodukten aus Baugruppen und Einzelteilen. Lieferbeziehungen für Teile
 - Anwendungsvorgänge : Personalverwaltung (Einstellung / Entlassung, Lohn- und Gehaltsabrechnung), Bestellung und Lieferung von Einzelteilen, Verkauf von Fertigprodukten, Lagerhaltung, Bedarfsplanung, Stücklistenauflösung

MOTIVATION FÜR EINSATZ EINES DBS

- typische Probleme bei Informationsverarbeitung ohne DBVS (z.B. Nutzung von Dateisystemen)
- Redundanz und Inkonsistenz
- beschränkte Zugriffsmöglichkeiten
- hohe Entwicklungskosten für Anwendungsprogramme



DBS-MOTIVATION / PROBLEME DATEISYSTEME (2)



- enge Bindung von Dateistrukturen an Programmstrukturen (geringe „Datenunabhängigkeit“)
 - Änderungen im Informationsbedarf sowie bei Leistungsanforderungen erfordern Anpassungen der Datenstrukturen, die auf Anwendungen durchschlagen
 - verschiedene Anwendungen brauchen verschiedene Sichten auf die selben Daten
- Probleme beim Mehrbenutzerbetrieb
- Verlust von Daten
- Integritätsverletzung
- Sicherheitsprobleme
 - Annahmen: Alles bleibt stabil ! Alles geht gut !

AUFGABEN/EIGENSCHAFTEN VON DBS



- generell: effiziente und flexible Verwaltung großer Mengen persistenter Daten (z. B. T Bytes)

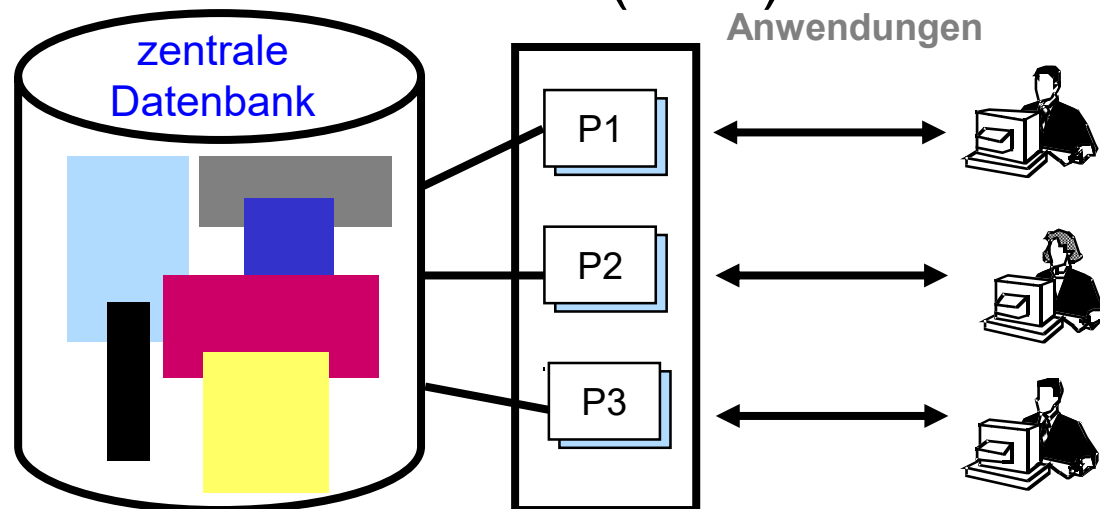
1. Zentrale Kontrolle über die operationalen Daten
2. Hoher Grad an Datenunabhängigkeit
3. Hohe Leistung und Skalierbarkeit
4. Mächtige Datenmodelle und Anfragesprachen / leichte Handhabbarkeit
5. Transaktionskonzept (ACID)
6. Automatisierte Zugriffskontrolle / Datenschutz
 - Zugriffsrechte für einzelne DB-Objekte
7. Ständige Verfügbarkeit / Betriebsbereitschaft
 - 24-Stundenbetrieb, keine Offline-Zeiten für DB-Reorganisation u. ä.



1. ZENTRALE KONTROLLE DER DATEN

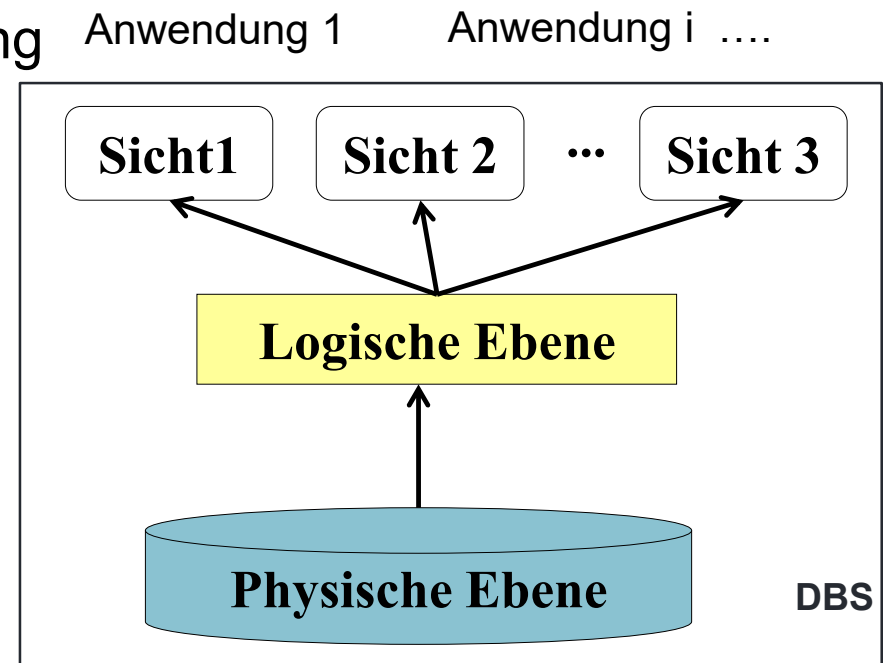
- alle (operationalen) Daten können gemeinsam benutzt werden
 - keine verstreuten privaten Dateien
 - ermöglicht inhaltliche Querauswertungen
- Eliminierung der Redundanz
 - Vermeidung von Inkonsistenzen
 - keine unterschiedlichen Änderungsstände
- einfache Erweiterung/Anpassung der DB
- Verwaltung durch Datenbankadministrator (DBA)

zentrale DB
statt verteilter Dateien



2. HOHE DATENUNABHÄNGIGKEIT

- Datenunabhängigkeit = Maß für die Isolation zwischen Anwendungsprogrammen und Daten
- gefordert: Isolation der Anwendungsprogramme von den Daten
sonst: extremer Wartungsaufwand für die Anwendungsprogramme
- physische Datenunabhängigkeit
 - Unabhängigkeit gegenüber Art der physischen Datenspeicherung (Geräteeigenschaften, Indexstrukturen ...)
- logische Datenunabhängigkeit
 - Unabhängigkeit gegenüber logischer Strukturierung der Daten (z.B. durch Sichten)
 - i. a. nur teilweise erreichbar



Abstraktionsebenen eines DBS

3. HOHE LEISTUNG UND SKALIERBARKEIT

- hoher Durchsatz / kurze Antwortzeiten für DB-Operationen auf großen Datenmengen
 - „trotz“ loser Bindung der Programme an die Daten (Datenunabhängigkeit)
- Leistung ist DBS-Problem, nicht Anwendungsproblem
 - Optimierung von DB-Anfragen durch das DBS (Query-Optimierung)
 - automatische Nutzung von Mehrprozessorsystemen, großen Hauptspeichern, parallelen Plattensystemen (Disk Arrays) etc.
 - Tuning (z.B. Festlegung von Indexstrukturen) automatisch durch DBS oder manuell durch DBA
- hohe Skalierbarkeit
 - Nutzung zusätzlicher/schnellerer Hardware-Ressourcen
 - Anpassung an steigende Leistungsanforderungen (wachsende Datenmengen und Anzahl der Benutzer)

4. MÄCHTIGE DATENMODELLE

- Datenmodell/DBS-Schnittstelle
 - Definition des Datenbankaufbaus durch **DB-Schema**
 - Definition von Integritätsbedingungen und Zugriffskontrollbedingungen (Datenschutz)
 - DDL (Data Definition Language): Operationen zur Definition von Datenstrukturen
 - DML (Data Manipulation Language): Operationen zum Aufsuchen und Verändern von Daten



- Beschreibung der logischen Aspekte der Daten, neutral gegenüber Anwendungen
 - Anwendung erhält logische, auf ihren Bedarf ausgerichtete Sicht auf die Daten
- formatierte Datenstrukturen, feste Satzstruktur
 - Beschreibung der Objekte durch Satztyp S_i , Attribute A_j und Attributwerte AW_k
 - jeder Attributwert AW_k wird durch Beschreibungsinformation (Metadaten) A_j und S_i in seiner Bedeutung festgelegt

Universität

Name	Gründungsjahr	Studierende	Fakultäten

RELATIONENMODELL

Beispiel: Universitäts-DB (*Schema*)

FAK

<u>FNR</u>	FNAME	DEKAN
------------	-------	-------

PROF

<u>PNR</u>	PNAME	FNR	FACHGEB
------------	-------	-----	---------

STUDENT

<u>MATNR</u>	SNAME	FNR	W-ORT
--------------	-------	-----	-------

PRÜFUNG

<u>PNR</u>	<u>MATNR</u>	FACH	DATUM	NOTE
------------	--------------	------	-------	------

RELATIONENMODELL (2)



Beispiel: Universitäts-DB (*Daten / Instanzen*)

FAK

FNR	FNAME	DEKAN
MI	Mathematik/ Informatik	2223

STUDENT

MATNR	SNAME	FNR	W-ORT
654 711	ABEL	MI	Leipzig
196 481	MAIER	MI	Delitzsch
225 332	MÜLLER	MI	Leipzig

PROF

PNR	PNAME	FNR	FACHGEB
1234	RAHM	MI	DBS
2223	MEYER	MI	AN
6780	BREWKA	MI	KI

PRÜFUNG

PNR	MATNR	FACH	DATUM	NOTE
6780	654 711	FA	19.9.	2
1234	196 481	DBS	15.10.	1
1234	654 711	DBS	17.4.	2
6780	196 481	KI	25.3.	3

RELATIONENMODELL (3)



Beispielanfragen mit SQL

Finde alle Studenten der Fakultät MI mit Wohnort Leipzig:

```
SELECT *  
FROM   STUDENT  
WHERE  FNR = 'MI' AND W-ORT = 'Leipzig'
```

Finde alle Studenten der Fakultät MI, die im Fach DBS eine Note 2 oder besser erhielten:

```
SELECT S.*  
FROM   STUDENT S, PRUEFUNG P  
WHERE  S.FNR = 'MI' AND P.FACH = 'DBS'  
AND P.NOTE <= 2 AND S.MATNR = P.MATNR
```



- Anfragesprache (query language) abhängig vom Datenmodell
 - Hierarchisches / Netzwerk-Datenmodell: navigierende / satzorientierte Operationen
 - Relationenmodell: deskriptive / mengenorientierte Operationen
- wünschenswert
 - deskriptive Problemformulierung, leichte Erlernbarkeit
 - hohe Auswahlmächtigkeit, z.B. einfache Verknüpfung mehrerer Satztypen („typübergreifende“ Operationen)
 - Standardisierung (SQL)
 - interaktiver DB-Zugriff sowie DB-Zugriff von Programmen aus
 - Unterstützung verschiedener Nutzerklassen

DBS-NUTZER/PERSONEN

- Endbenutzer von DB-Anwendungen
 - kommuniziert nur mit Anwendung
 - benötigt keine Datenbankkenntnisse
- Anwendungsprogrammierer
 - intensive Nutzung der Datenbank mit DML
- DB-Analyst
 - interaktive Nutzung der Datenbank, z.B. mit SQL
- DB-Modellierer
 - entwirft DB-Schema
- DB-Administrator
 - richtet DB ein, vergibt Zugriffsrechte
 - führt DB-Sicherungen durch
 - optimiert die Leistung durch Tuning-Maßnahmen ...
- (DBMS-Implementierer)



5. TRANSAKTIONSKONZEPT

- Eine Transaktion ist eine Folge von DB-Operationen (DML-Befehlen), für die das DBS die vier sogenannten **ACID**-Eigenschaften gewährleistet:

- A**tomicity: 'Alles oder Nichts'-Eigenschaft (Fehlerisolierung)
→ Transaktion wird vollständig ausgeführt oder überhaupt nicht
- C**onsistency: eine erfolgreiche Transaktion erhält die DB-Konsistenz (Gewährleistung der definierten Integritätsbedingungen), so dass sie die DB von einem logisch konsistenten Zustand in einen (möglicherweise geänderten) logisch konsistenten Zustand überführt.
- I**solation: alle Aktionen innerhalb einer Transaktion müssen vor parallel ablaufenden Transaktionen verborgen werden („logischer Einbenutzerbetrieb“)
- D**urability: Überleben von Änderungen erfolgreich beendeter Transaktionen trotz beliebiger (erwarteter) Fehler garantieren (*Persistenz*).



TRANSAKTIONSKONZEPT (2)

- Programmierschnittstelle für Transaktionen
 - begin of transaction (BOT)
 - commit transaction („commit work“ in SQL)
 - rollback transaction („rollback work“ in SQL)
- mögliche Ausgänge einer Transaktion

BOT
DML1
DML2
...
DMLn
COMMIT WORK

normales Ende

BOT
DML1
DML2
...
DMLn
ROLLBACK WORK

abnormales Ende

BOT
DML1
DML2

erzwungenes ROLLBACK

abnormales Ende

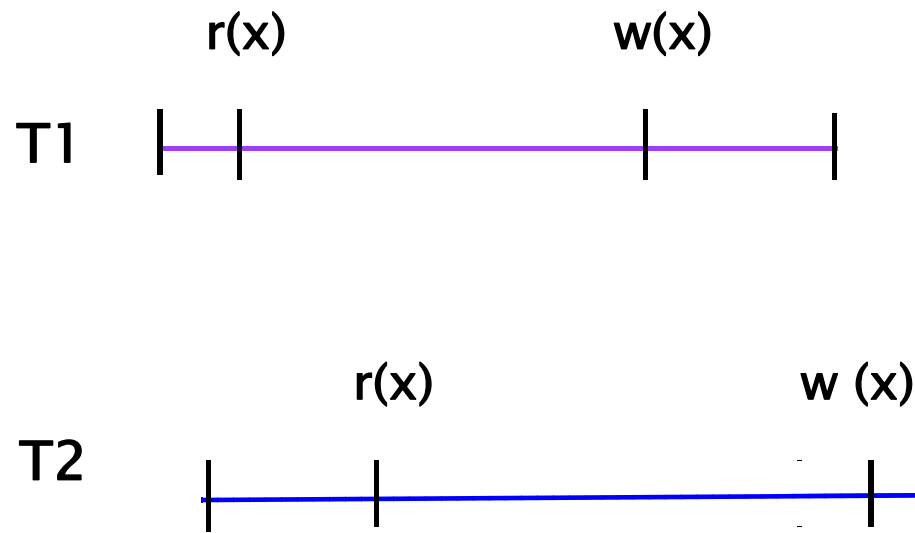
**Systemausfall,
Programm-
fehler usw.**



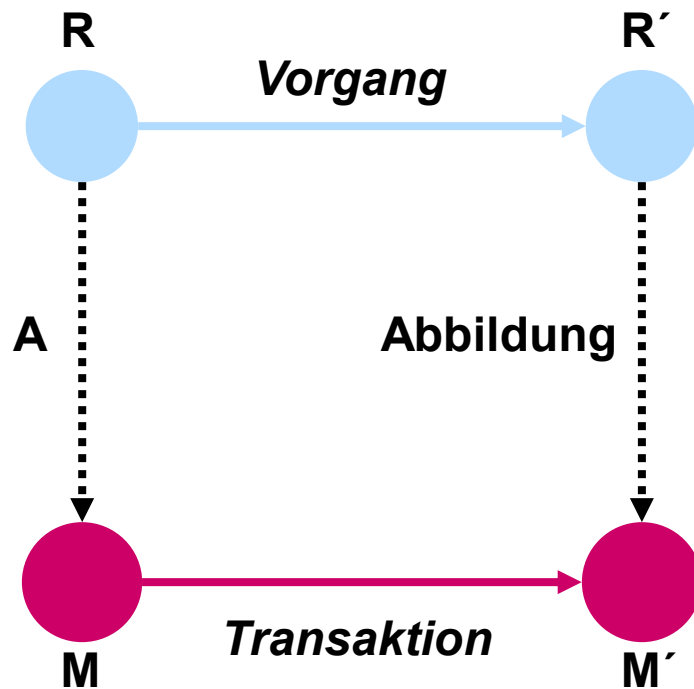
- ACID impliziert verschiedene Arten an Datenintegrität
- Consistency: Erhaltung der *logischen Datenintegrität*
- Erhaltung der *physischen Datenintegrität*
 - Führen von Änderungsprotokollen für den Fehlerfall (*Logging*)
 - Bereitstellen von Wiederherstellungsalgorithmen im Fehlerfall (*Recovery*)
- kontrollierter Mehrbenutzerbetrieb (*Ablaufintegrität*)
 - *logischer Einbenutzerbetrieb* für jeden von n parallelen Benutzern (Leser + Schreiber)
 - Synchronisation i. a. durch Sperren (*Locking*)
 - Ziel: möglichst geringe gegenseitige Behinderung

SYNCHRONISATIONSBEISPIEL

- Konto x mit Kontostand 100 Euro
- zwei gleichzeitige Buchungstransaktionen
 - T1: Einzahlung 100 Euro
 - T2: Abhebung 50 Euro



MODELL EINER MINIWELT: GROBE ZUSAMMENHÄNGE



R: Realitätsausschnitt (Miniwelt)

M: Modell der Miniwelt
(beschrieben durch DB-Schema)

A: Abbildung aller wichtigen Objekte und
Beziehungen (Entities und Relationships)
=> Abstraktionsvorgang

- Transaktion:
 - garantiert ununterbrechbaren Übergang von M nach M'
 - implementiert durch Folge von DB-Operationen
- Integritätsbedingungen:
 - Zusicherungen über A und M
 - Ziel: möglichst gute Übereinstimmung von R und M

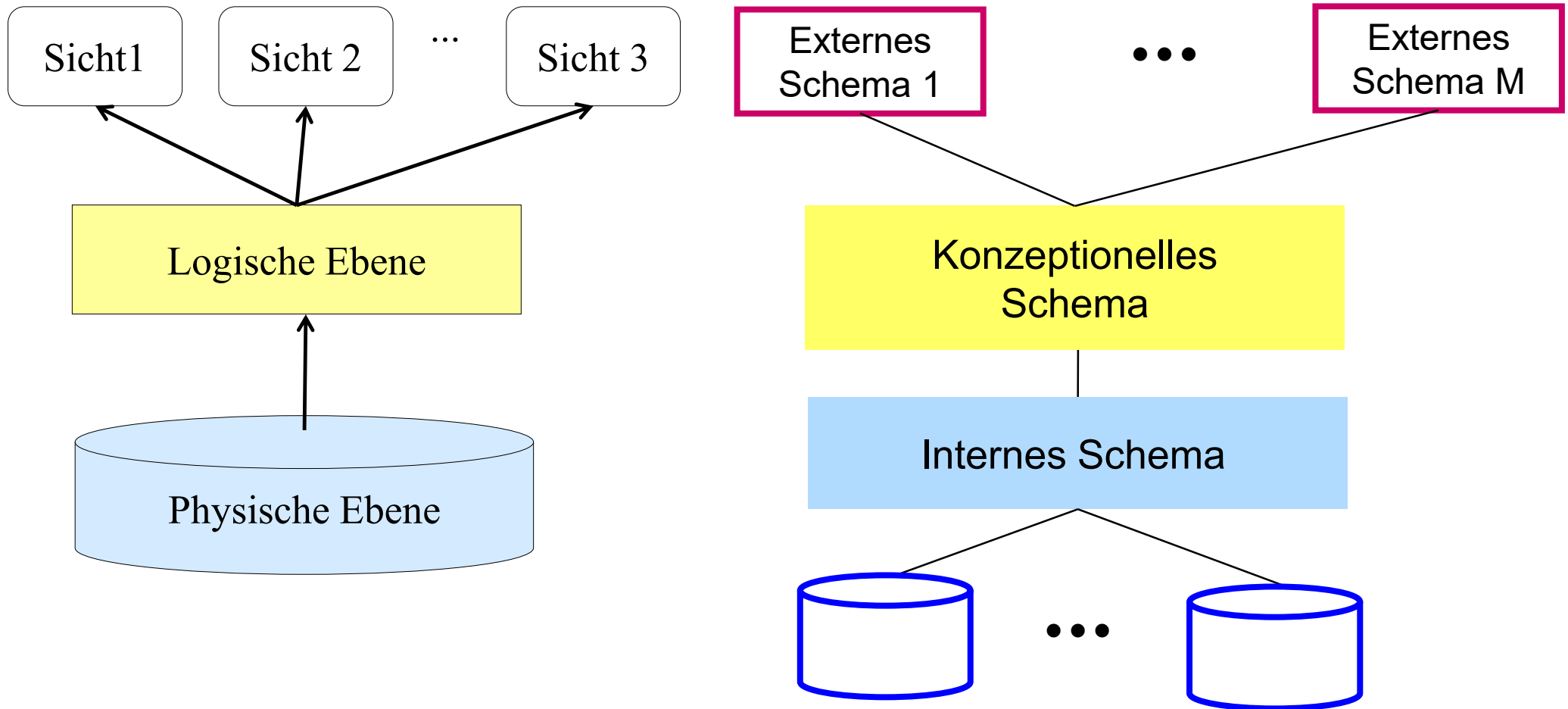


1. EINFÜHRUNG / GRUNDLAGEN VON DBS

- DBS vs. Dateisysteme
- Eigenschaften von DBS
- Datenmodelle
- Transaktionskonzept (ACID)
- Aufbau von DBS
 - Schemaarchitektur
 - Schichtenmodell
- Einsatzformen von DBS: OLTP vs. OLAP
- Historische Entwicklung
 - Datenmodelle
 - Architekturen

SCHEMAARCHITEKTUR

– 3-Ebenen-Architektur nach ANSI / SPARC



SCHEMAARCHITEKTUR (2)



- **konzeptionelles Schema:**

- logische Gesamtsicht auf die Struktur der Datenbank
- abstrahiert von internem Schema -> *physische Datenunabhängigkeit*

- **internes Schema**

- legt physische Struktur der DB fest (physische Satzformate, Indexstrukturen etc.)

- **externe Schemata / Sichtenbildung**

- definieren spezielle Benutzersichten auf DB-Struktur (für Anwendungsprogramm bzw. Analysten)
- abstrahieren von konzeptionellem Schema: ermöglicht partiell *logische Datenunabhängigkeit*
- Zugriffsschutz: nur ausgewählte Relationen und Attribute bleiben zugänglich
- reduzierte Komplexität: Anwendung sieht nur die erforderlichen Daten

BEISPIEL-DATENBESCHREIBUNG (VEREINFACHT)



Externes Schema 1

MITARBEITER

PNR	CHAR	(6)
ABT	CHAR	(30) ...

Externes Schema 2

ABT-INFO

ANR	CHAR(4)
ANZ-MA	INT ...

Konzeptionelles Schema:

PERSONAL

(PERSONAL_NUMMER	CHAR (6)
ABT_NUMMER	CHAR (4)
...	)

ABT

(ANR ...
ABTNAME ...

Internes Schema:

STORED_PERS LENGTH=18

PREFIX TYPE=BYTE(6), OFFSET=0

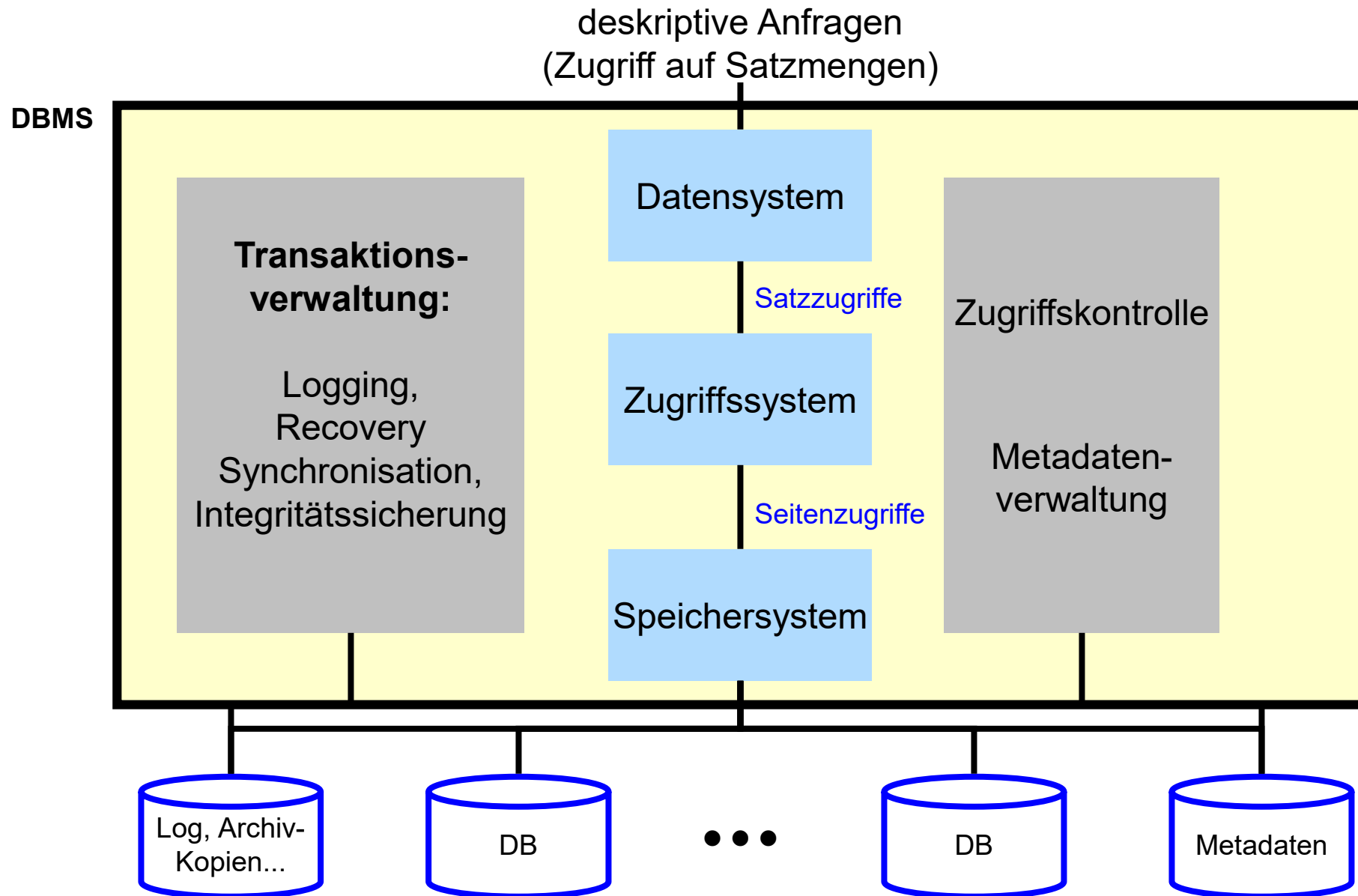
PNUM TYPE=BYTE(6), OFFSET=6, INDEX=PNR

ABT# TYPE=BYTE(4), OFFSET=12

PAY TYPE=FULLWORD, OFFSET=16

...

GROBAUFBAU EINES DBS





OLTP VS. OLAP (1)

- OLTP (Online Transaction Processing)
 - dominierende Einsatzform von DBS, u.a. für E-Business
 - Ausführung vorgeplanter Anwendungsprogramme mit DB-Zugriff
- Beispiele für OLTP-Transaktionen:
 - Produktbestellung, Platzreservierung (Flug, Hotel, etc.)
 - Kontostandsabfrage; Überweisung
 - Kreditkartenbezahlung
 - Abwickeln eines Telefonanrufes, ...
- Transaktionsmerkmale
 - kurze Bearbeitungszeit notwendig
 - wenige Daten pro Transaktion
 - häufige Änderungen
 - Einhaltung von ACID essenziell

OLTP VS. OLAP (2)

– OLAP (Online Analytical Processing)

- umfassende Auswertung/Analyse großer Datenbestände
- Vorbereitung von Geschäftsentscheidungen (Decision Support)
- Anwendungen: Vertriebskontrolle, Preisoptimierung, Kundenbindung
- ...

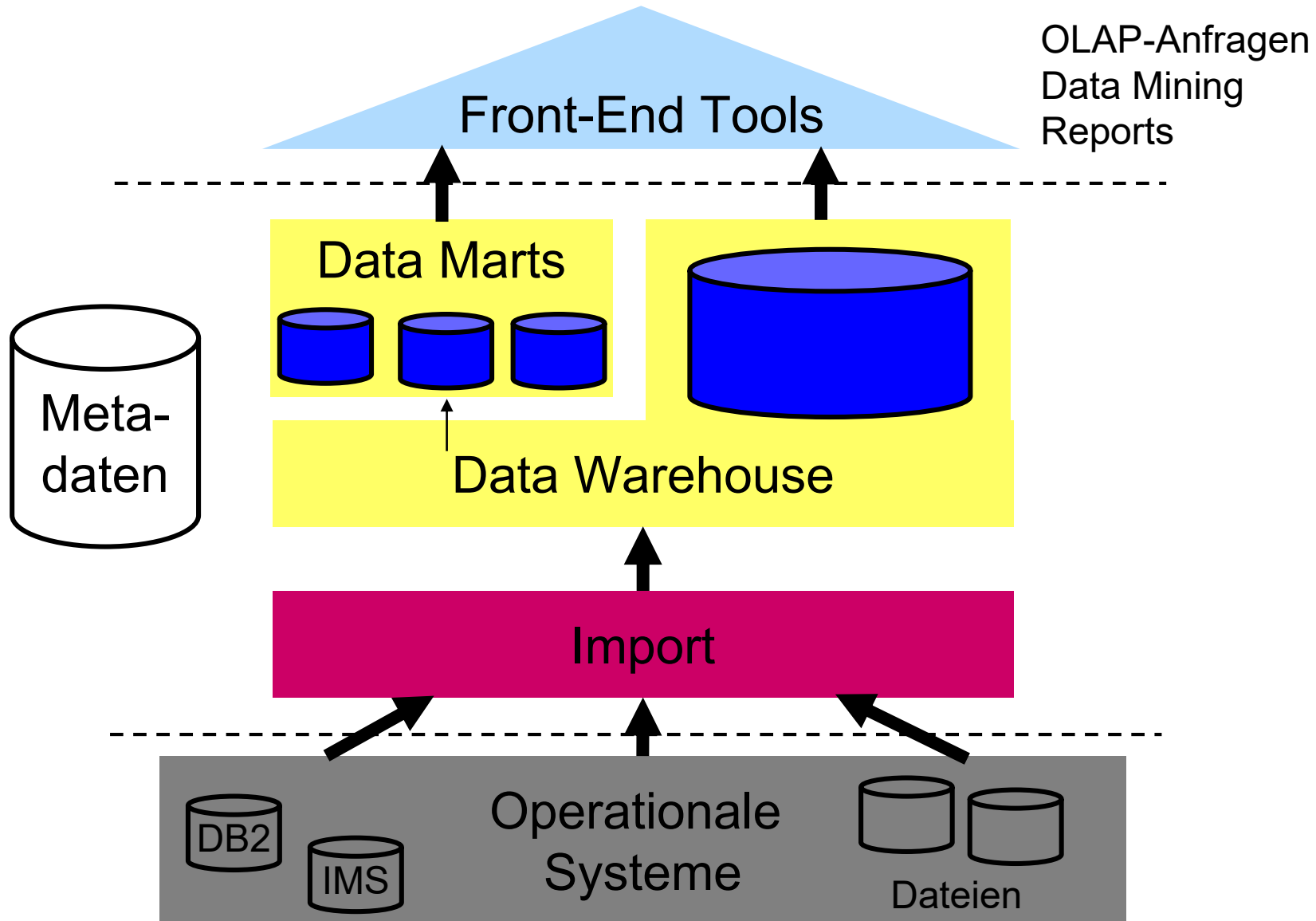
– häufiger Einsatz von **Data Warehouses**

- Konsolidierung und Integration der Datenbestände eines Unternehmens für Analysen
- Bsp.: Umsatzentwicklung nach Zeit, Produktklasse, Region, etc.

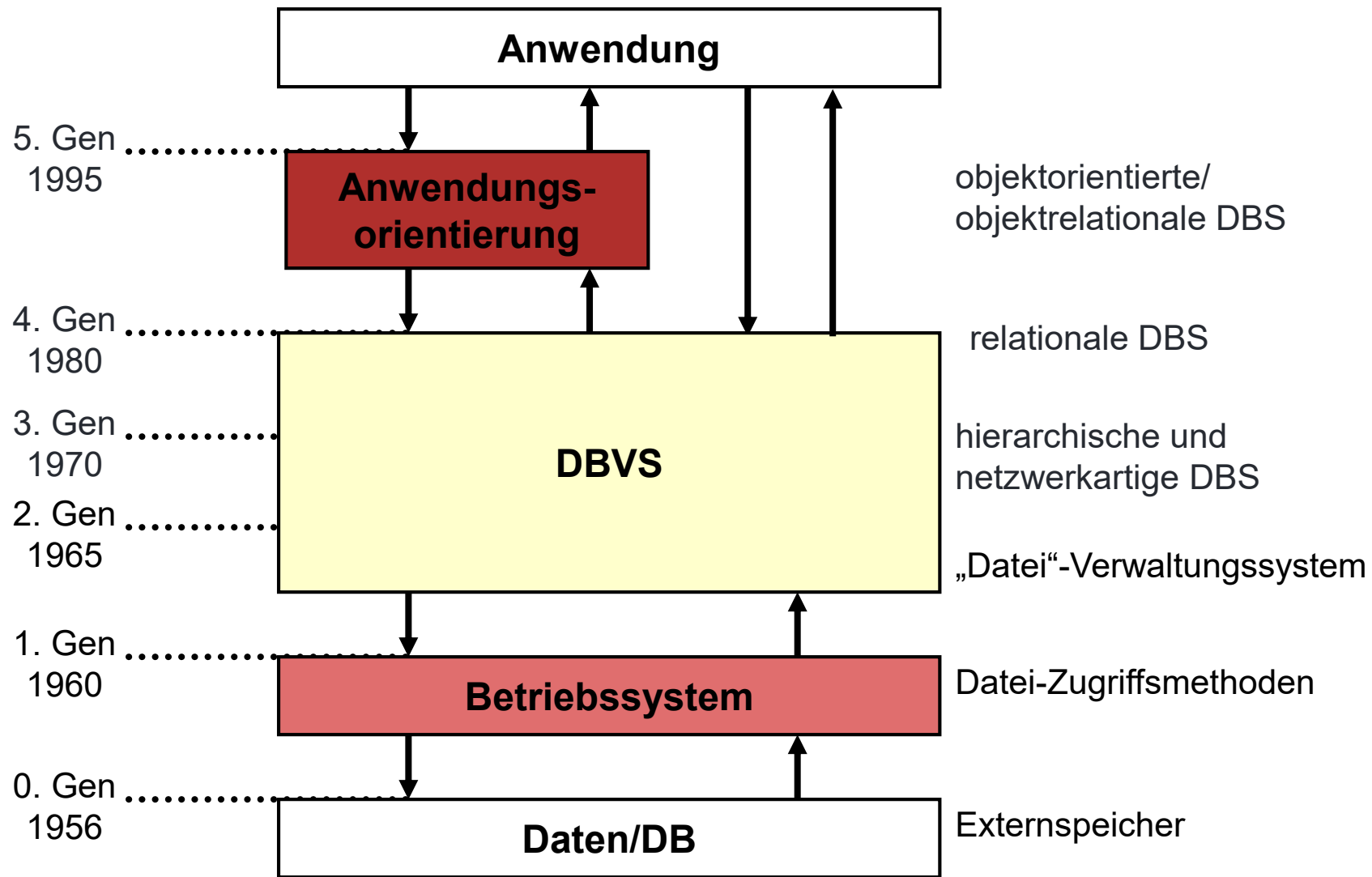
– **Data Mining / Machine Learning**

- eigenständiges Aufspüren von inhärenten Mustern in großen Datenbeständen (nicht nur Beantwortung gestellter Fragen)
- Vorhersagen, z.B. auf Basis von Beispieldaten

DATA-WAREHOUSE-UMFELD



HISTORISCHE ENTWICKLUNG VON DBS





HISTORISCHE ENTWICKLUNG (2)

– semistrukturierte Daten / XML-Dokumente

- optionales Schema
- Mischung von strukturierten Daten und Text
- Anfragesprache XQuery für XML

```
<Vorlesung >  
  <Thema>DBS1</Thema>  
  <Dozent>  
    <Name>Prof. Rahm</Name>  
    <Einrichtung>Uni Leipzig</Einrichtung>  
  </Dozent>  
</Vorlesung>
```

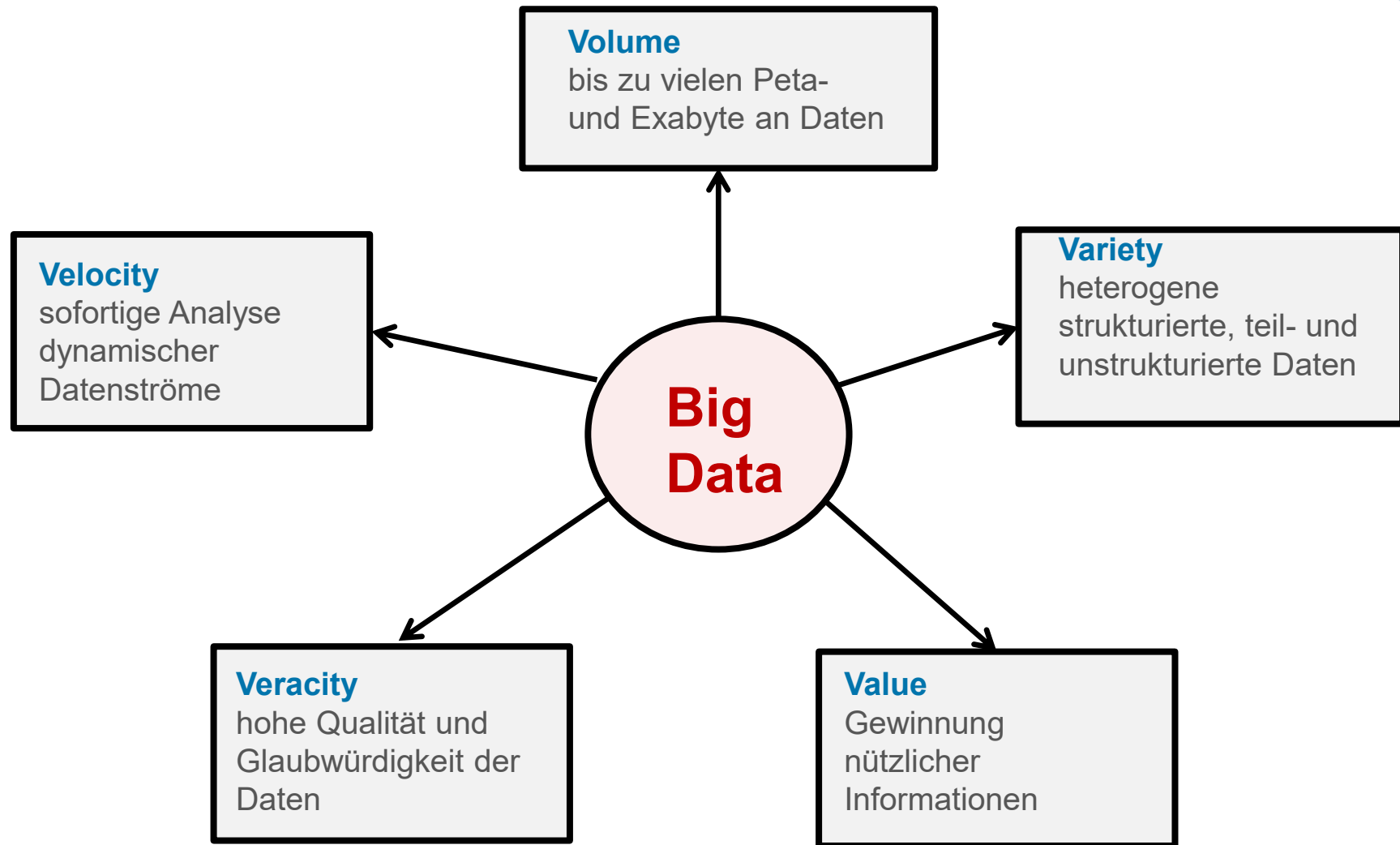
– NoSQL-Datenbanken / Datenbanken für „Big Data“ (seit ca 2009)

- verteilte Key Value Stores
- Dokumenten-Stores, z.B. MongoDB
- Graph-Datenbanken, z.B. Neo4j

– NewSQL-Systeme

- effizientere Implementierungen relationaler DBS
- In-Memory-Datenbanken, Column Store statt RecordStore, ...

BIG DATA CHALLENGES



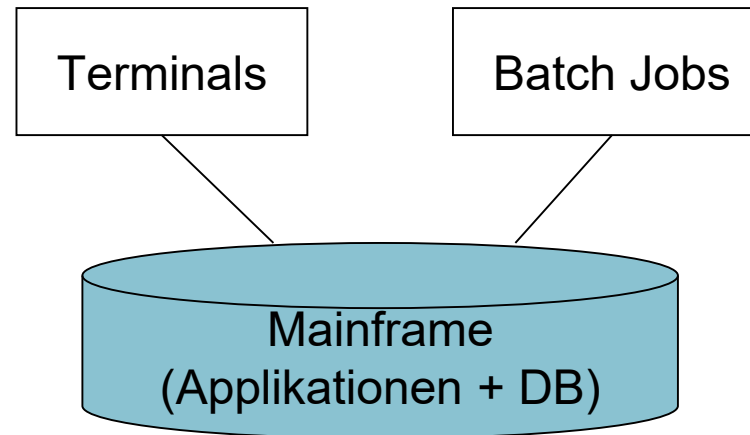
DATA CENTER



Quelle: Google

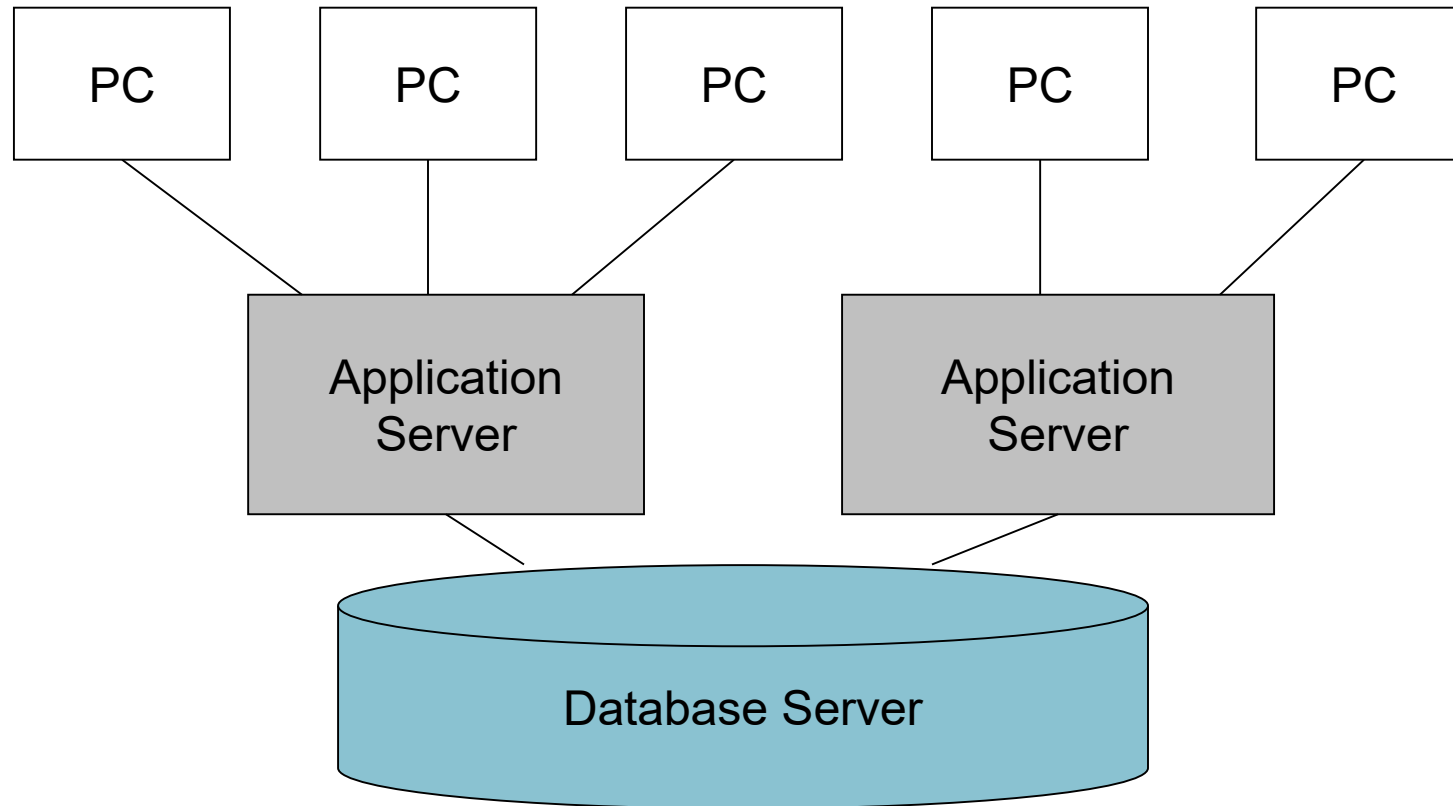
ENTWICKLUNG VON DBS-ARCHITEKTUREN

- DBS auf Mainframe (1960er/70er Jahre)



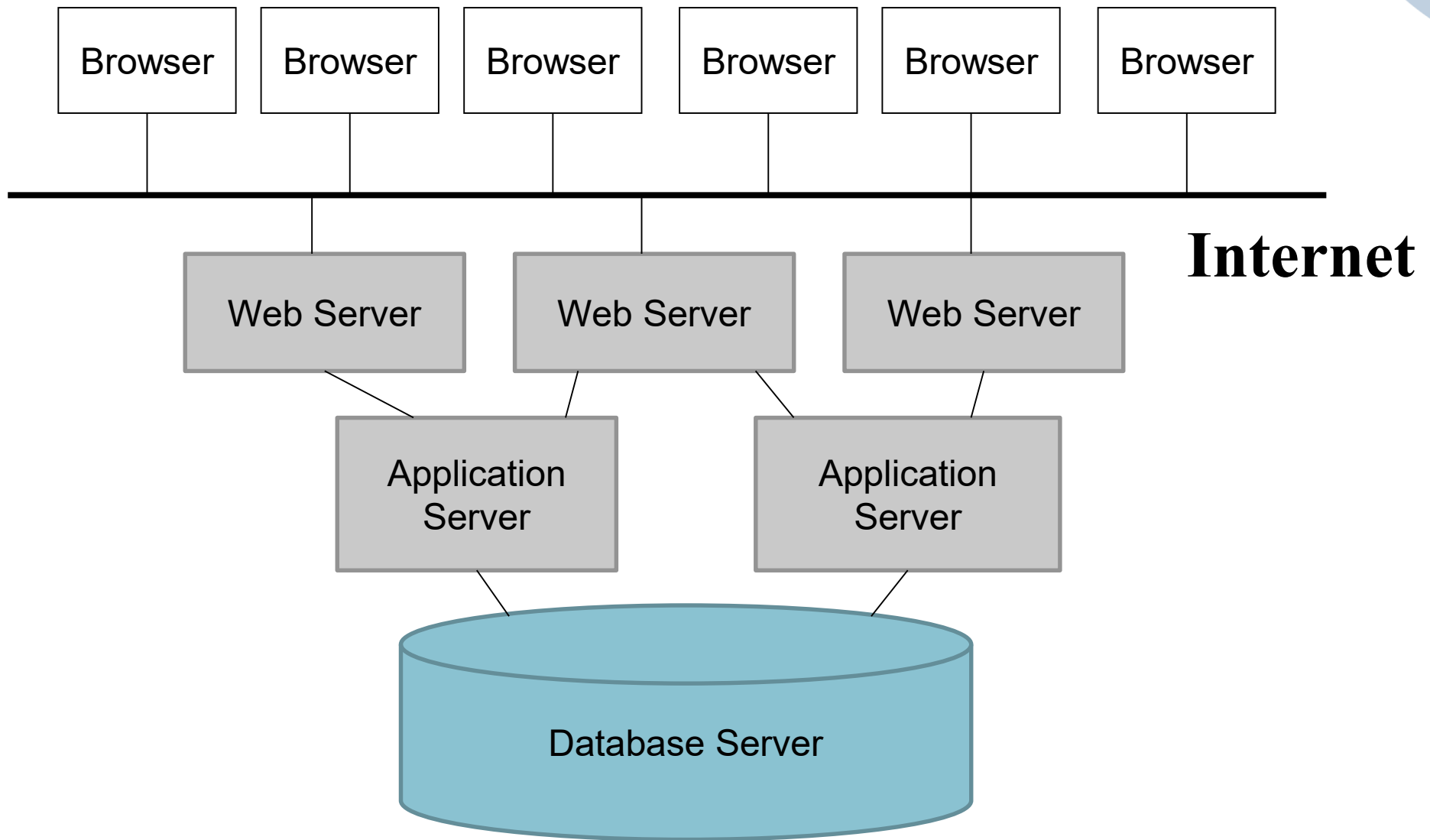
- Client/Server-Systeme
 - 2-stufig (80er Jahre)
 - 3-stufig (90er Jahre)
- Mehrrechner-DBS / Parallele DBS (seit 90er Jahre)
 - mehrere DB-Server
 - Cluster oder ortsverteilt
- Web-Einbindung
- Cloud-Einsatz

THREE-TIER CLIENT/SERVER

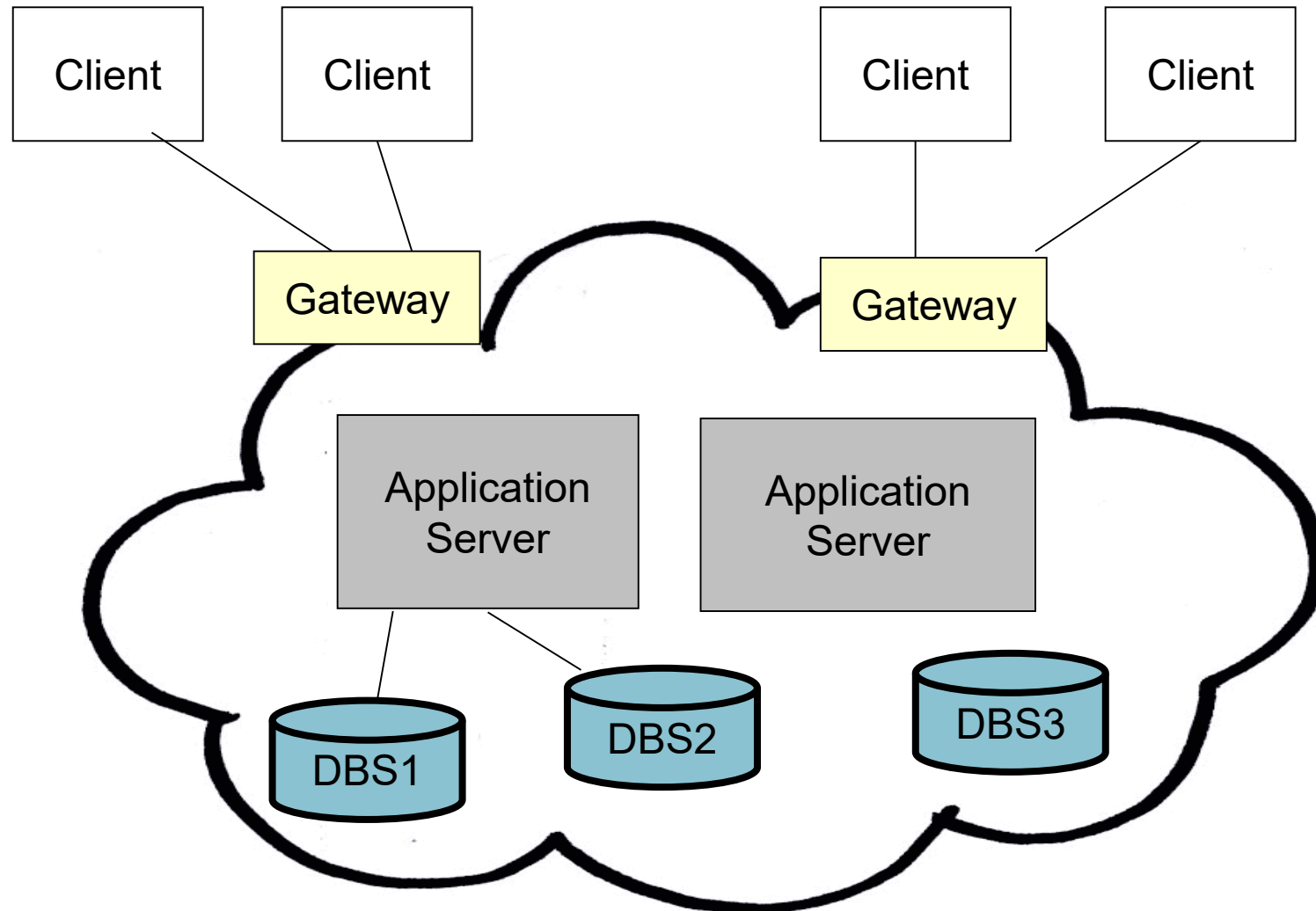


- mehrere Applikationsserver zur Skalierbarkeit auf viele Nutzer
- DB-Zugriffe erfordern Kommunikation zwischen Application und DB Server
- DB-Server kann paralleles DBS in einem Cluster sein

DATENBANKEN IM WEB



CLOUD-DBS



ZUSAMMENFASSUNG (1)



- Datenverwaltung durch Dateisysteme unzureichend
- DBS-Charakteristika
 - effiziente Verwaltung persistenter und strukturierter Daten
 - Datenstrukturierung und Operationen gemäß Datenmodell/DB-Sprache
 - Transaktionskonzept (ACID): Atomarität, Konsistenzerhaltung, kontrollierter Mehrbenutzerbetrieb, Persistenz erfolgreicher Änderungen
 - zentrale (integrierte) Datenbank mit hohem Grad an Datenunabhängigkeit
- dominierendes Datenmodell: Relationenmodell
 - mengenorientierte DB-Schnittstelle
 - standardisierte Anfragesprache SQL

ZUSAMMENFASSUNG (2)



- Schema-Architektur mit 3 Ebenen:
externes, konzeptionelles, internes Schema
- Schichtenmodell eines DBVS
 - interne Schichten für Seiten, Sätze und Satzmenngen
 - Querschnittsaufgaben: Transaktionsverwaltung und Metadaten
- Haupt-Einsatzformen von DBS in Unternehmen:
 - OLTP-Transaktionsverarbeitung / E-Business
 - Entscheidungsunterstützung: OLAP, Data Mining, maschinelles Lernen
- Architekturvarianten
 - DB-Zugriff über Applikations-Server und Web-Interfaces
 - zentrale DB-Server bzw. Parallele DBS
 - Server im Unternehmen oder extern, z.B. in Cloud

2. INFORMATIONSMODELLIERUNG MIT ENTITY-RELATIONSHIP-MODELL UND UML



- Einführung Modellierung / Abstraktionskonzepte
- Entity-Relationship-Modell
 - Entity-Mengen, Attribute und Wertebereiche
 - Primärschlüssel
 - Relationship-Mengen
 - Klassifikation der Beziehungstypen (1:1, n:1, 1:n, n:m)
 - schwache Entity-Mengen
- Modellierung mit UML (Klassendiagramme)
 - Kardinalitätsrestriktionen (Multiplizitäten)
 - Generalisierung / Spezialisierung
 - Aggregation



LERNZIELE KAPITEL 2

- Kenntnis der Vorgehensweise beim DB-Entwurf
- Grundkonzepte des ER-Modells sowie von UML-Klassendiagrammen
- Kenntnis der Abstraktionskonzepte, insbesondere von Generalisierung und Aggregation
- Fähigkeit zur praktischen Anwendung der Konzepte
 - Erstellung von ER-Modellen und -Diagrammen bzw. UML-Modellen für gegebene Anwendungsszenarien
 - Festlegung der Primärschlüssel, Beziehungstypen, Kardinalitäten, Existenzabhängigkeiten etc.
 - Interpretation gegebener ER- bzw. UML-Modelle



– Ziele:

- modellhafte Abbildung eines anwendungsorientierten Ausschnitts der realen Welt (Miniwelt)
- Entwurf der logischen DB-Struktur (DB-Entwurf)

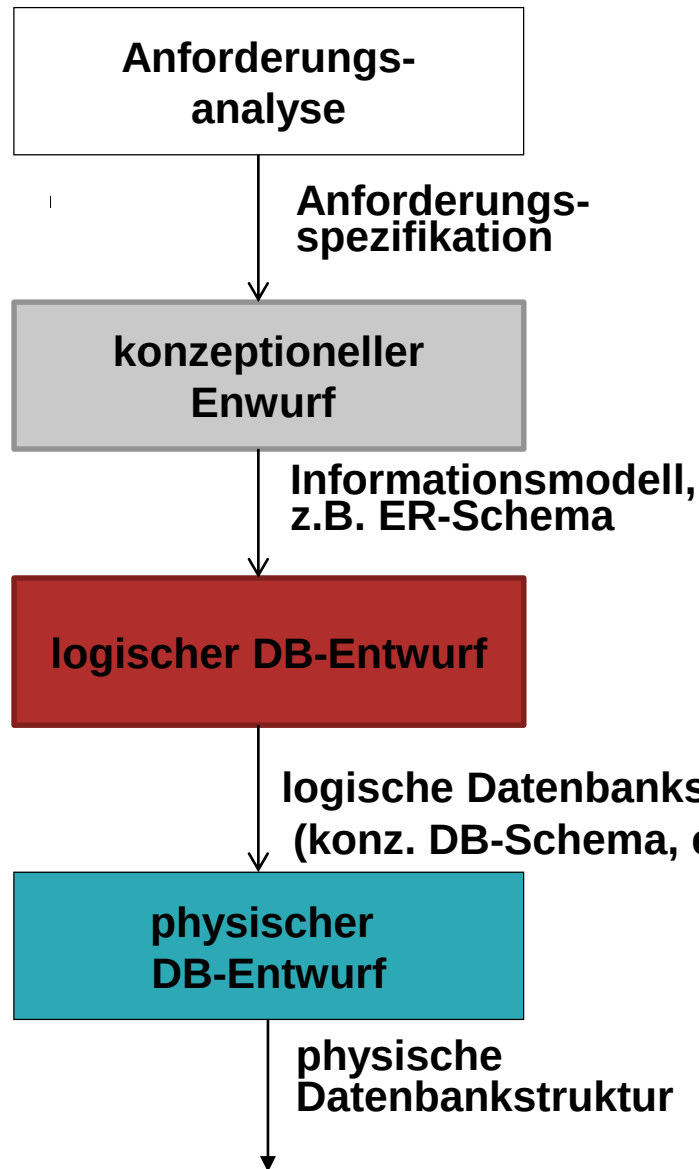
– Nebenbedingungen:

- Vollständigkeit
- Korrektheit
- Minimalität
- Lesbarkeit, Modifizierbarkeit

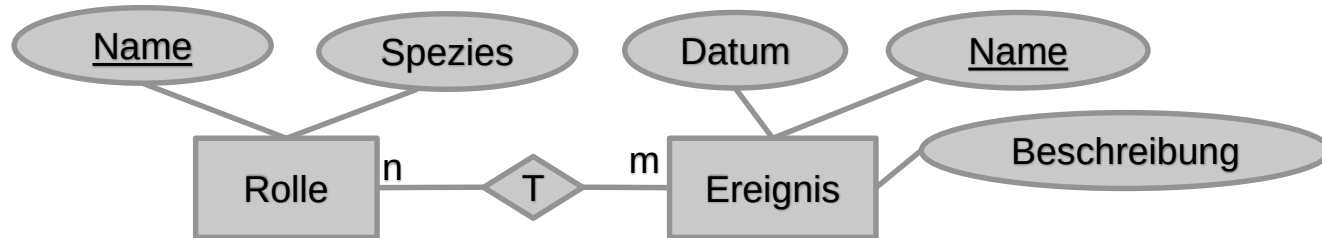
– schrittweise Ableitung (verschiedene Sichten)

- 1) Information in unserer Vorstellung
- 2) Informationsstruktur: Organisationsform der Information
- 3) Logische Datenstruktur (Was-Aspekt)
- 4) Physische Datenstruktur (Was- und Wie-Aspekt)

PHASEN DES DB-ENTWURFS



Die Zielsetzung dieses Projekts ist die Erstellung einer Datenbank zur Speicherung von Informationen über die Rollen im Buch "Der Herr der Ringe" von J.R.R. Tolkien und die Ereignisse an denen sie beteiligt waren...



Rolle		Ereignis		
<u>Name</u>	Spezies	<u>Name</u>	Datum	Beschr.

Teilnahme	
<u>R_Name</u> (FK)	<u>E_Name</u> (FK)

```
CREATE TABLE Rolle(
  name varchar(20) PRIMARY KEY,
  spezies varchar(100)
);
```

```
CREATE TABLE Ereignis(
  name varchar(20) PRIMARY KEY,
  beschreibung varchar(200),
  datum date
);
```

```
CREATE TABLE teilnahme(
  r_name varchar(20) PRIMARY KEY,
  e_name varchar(200),
  FOREIGN KEY (r_name) references Rolle,
  FOREIGN KEY (e_name) references Ereignis,
  PRIMARY KEY (r_name, e_name)
);
```



**Ausschnitt der
realen Miniwelt**

manuelle/intellektuelle Modellierung

LLM unterstützt

konzeptionelles Schema
(ER-Schema bzw. UML-Klassendiagramm)

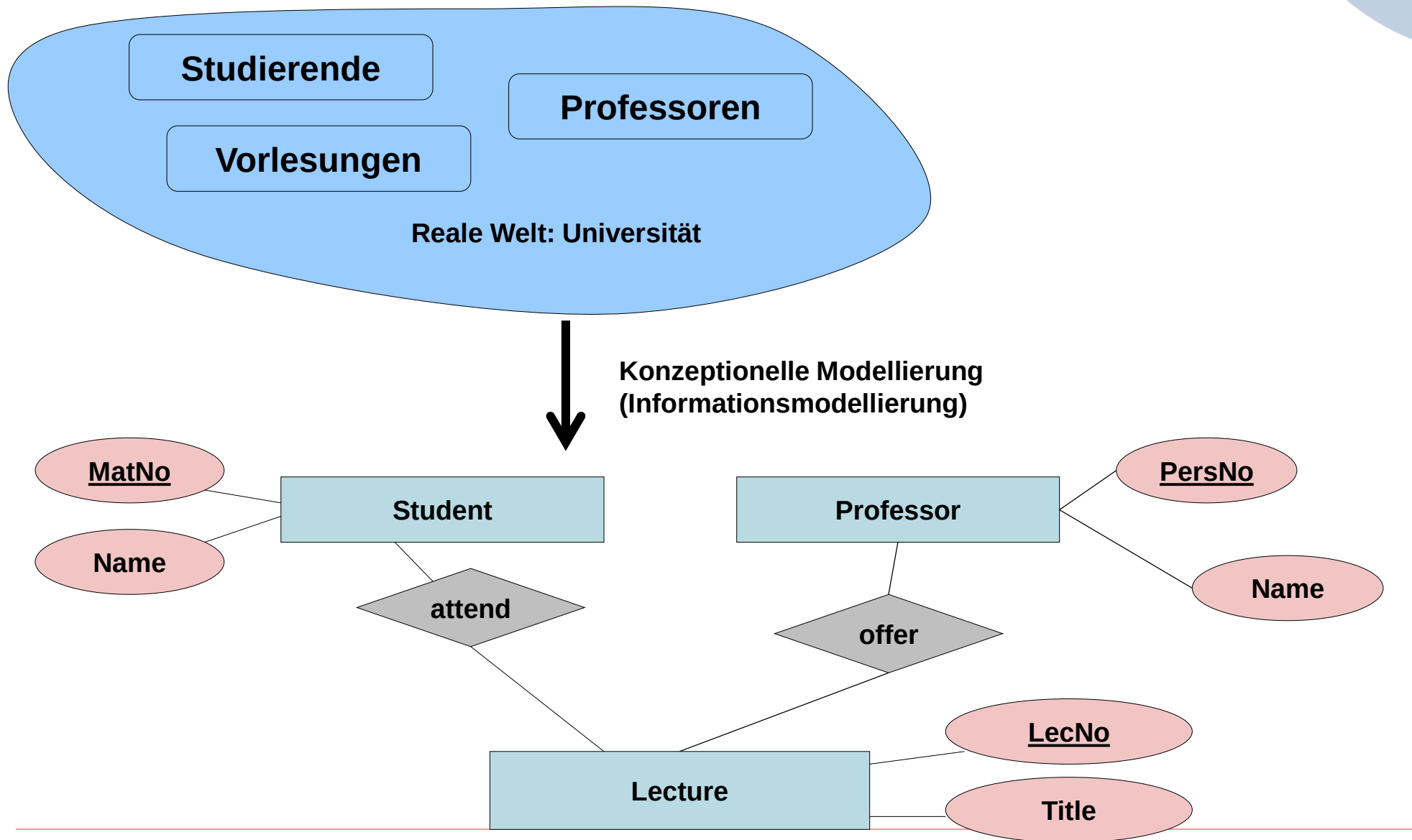
teilautomatische
Transformation

**relationales
Schema**

**XML
Schema (DBS II)**

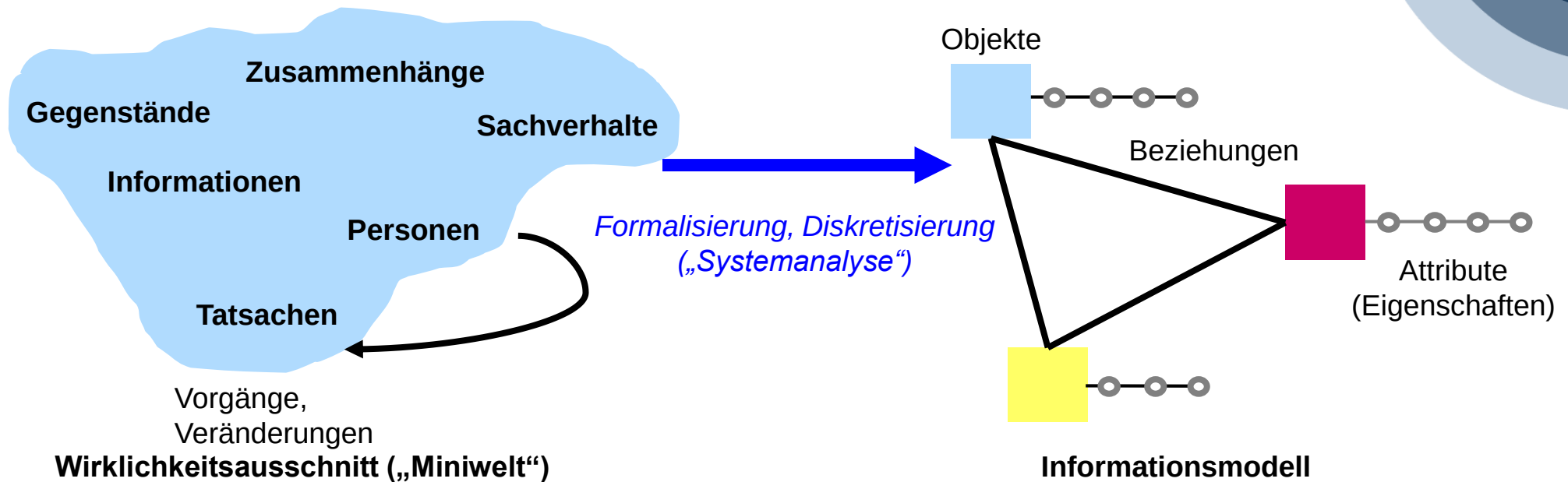
**objektorientiertes
Schema**

MODELLIERUNGSBEISPIEL





INFORMATIONSMODELLIERUNG



- Darstellungselemente + Regeln:
 - Objekte (Entities) und Beziehungen (Relationships)
 - Klassen von Objekten / Beziehungen
 - Eigenschaften (Attribute)
- Informationen über Objekte und Beziehungen nur wenn:
 - relevant
 - unterscheidbar und identifizierbar, selektiv beschreibbar



ABSTRAKTIONSKONZEPTE

- Informations- und Datenmodelle basieren auf drei grundlegenden Abstraktionskonzepten
- **Klassifikation**: fasst Objekte (Entities, Instanzen) mit gemeinsamen Eigenschaften zu einem neuen (Mengen-) Objekt (Entity-Menge, Klasse, Objekttyp) zusammen.
 - Instanzen/Objekten einer Klasse unterliegen gleicher Struktur (Attribute), gleichen Integritätsbedingungen, gleichen Operationen
 - mathematisch: Mengenbildung
- **Aggregation**: Zusammenfassung potentiell unterschiedlicher Teilobjekte (Komponenten) zu neuem Objekt
 - mathematisch: Bildung von kartesischen Produkten
- **Generalisierung / Spezialisierung**: Teilmengenbeziehungen zwischen Elementen verschiedener Klassen
 - mathematisch: Bildung von Potenzmengen (bzw. Teilmengen)
 - wesentlich: Vererbung von Eigenschaften an Teilmengen



ENTITY-RELATIONSHIP-MODELL

- entwickelt von P. P. Chen
 - ACM Transactions on Database Systems 1976
- Konzepte:
 - Entity-Mengen
 - Beziehungsmengen (Relationship-Mengen)
 - Attribute
 - Wertebereiche
 - Primärschlüssel
- unterstützt Abstraktionskonzepte Klassifikation und Aggregation
- zahlreiche Erweiterungsvorschläge
- graphische Darstellung durch Diagramme
- weite Verbreitung über DB-Entwurf hinaus
 - Systemanalyse
 - Unternehmensmodellierung



ENTITY-MENGEN

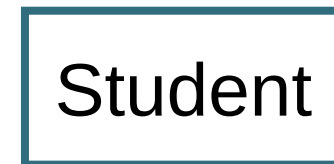
- *Entity* (Entität, Gegenstand): repräsentiert abstraktes oder physisches Objekt der realen Welt
- gleichartige Entities (d. h. Entities mit gemeinsamen Eigenschaften) werden zu *Entity-Mengen* (Gegenstandstypen, Objekttypen) zusammengefasst (Klassifikation)
 - => Entities sind Elemente einer (homogenen) Menge: $e \in E$
 - z. B. Personen, Projekte ...
 - Bücher, Autoren ...
 - Kunden, Vertreter, Wein, Behälter
- DB enthält endlich viele Entity-Mengen: $E_1, E_2, \dots, E_n$
 - nicht notwendigerweise disjunkt, z.B. Student und Person
- Symbol für Entity-Menge E:





ATTRIBUTE UND WERTEBEREICHE

- Attribute und Attributwerte:
 - Eigenschaften von Entity-Mengen werden durch Attribute bestimmt
 - Eigenschaften einzelner Entities sind durch Attributwerte festgelegt
 - **Nullwert**: spezieller Attributwert, dessen Wert unbekannt oder nicht möglich ist (z. B. akadTitel)
- jedem Attribut ist ein Wertebereich (Domain) zugeordnet, der festlegt, welche Attributwerte zulässig sind (Integritätsbedingung !)
 - $E (A1: D1, A2: D2, \dots A_n: D_n)$
 - Beispiel: Student (MatNo: int, Name: char(50))
- Attributsymbol in ER-Diagrammen:





ATTRIBUTARTEN

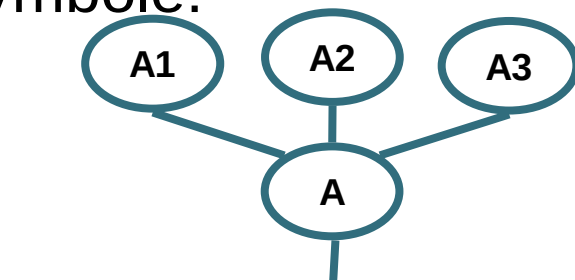
– einfache vs. zusammengesetzte Attribute

- Beispiele: NAME [FirstName: char (30), LastName: char (30)]
ADDRESS [Street: char (30), City: char (30), ZIP: char (5)]
- Domain für zusammengesetztes Attribut A [A1, A2, ... Ak]:
 $W(A1) \times W(A2) \times \dots \times W(Ak)$

– einwertige vs. mehrwertige Attribute

- Beispiele: Color: {char (20)}
- Child: {[Name: char (30), Age: int]}

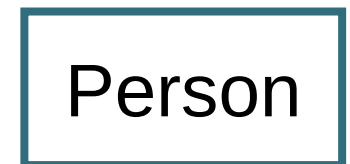
– Symbole:



zusammengesetztes Attribut



mehrwertiges Attribut



SCHLÜSSELKANDIDAT



– Schlüsselkandidat oder kurz Schlüssel (key)

- einwertiges Attribut oder Attributkombination, die jedes Entity einer Entity-Menge eindeutig identifiziert
- keine Nullwerte!
- mehrere Schlüsselkandidaten pro Entity-Menge möglich

– Definition Schlüsselkandidat

$A = \{A_1, A_2, \dots, A_m\}$ sei Menge der Attribute zu Entity-Menge E

$K \subseteq A$ heißt Schlüsselkandidat von $E \Leftrightarrow$

1. $\forall e_i, e_j \in E$ mit $e_i \neq e_j \rightarrow K(e_i) \neq K(e_j)$;
2. **K minimal**, d.h. keine echte Teilmenge von K bietet Eigenschaft

1.

Ist {MatNo, Name} Schlüsselkandidat für Student?

PRIMÄRSCHLÜSSEL

- Primärschlüssel = Schlüsselkandidat
 - ggf. unter mehreren Kandidaten einen auszuwählen
 - ggf. künstlich zu erzeugen (lfd. Nr.)

Beispiel:

Prof				
Room	SecrPhone	FirstName	LastName	PNo
11	34567	Karl	Meier	294
12	34567	Anita	Schulz	343
23	45678	Karl	Schulz	569

- Primärschlüsselattribute werden durch Unterstreichungen gekennzeichnet

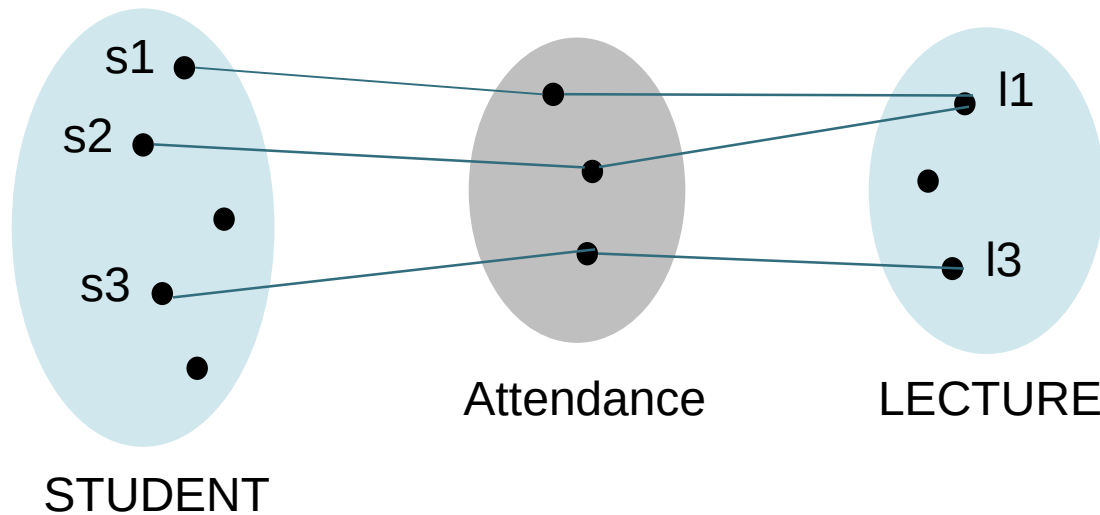
Attr.Name

Prof



RELATIONSHIPS

- Relationship-Menge: Zusammenfassung gleichartiger Beziehungen (Relationships) zwischen Entities, die jeweils gleichen Entity-Mengen angehören
- Beispiel: Beziehungen *Vorlesungsteilnahme* zwischen *Student* und *Vorlesung*



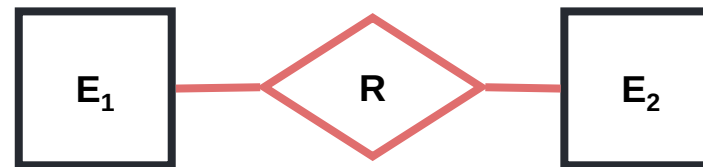


RELATIONSHIPS (2)

- Relationship-Menge R
entspricht mathematischer Relation zwischen n Entity-Mengen E_i

- $R \subseteq E_1 \times E_2 \times \dots \times E_n$,
- d. h. $R = \{r = [e_1, e_2, \dots, e_n] \mid e_1 \in E_1, \dots, e_n \in E_n\}$
- gewöhnlich: $n=2$ oder $n=3$

- Symbol:



- Beispiel:



- $\text{Attendance} \subseteq \text{Student} \times \text{Lecture}$
- konkret $= \{[s1, l1], [s2, l1], [s3, l3]\}$

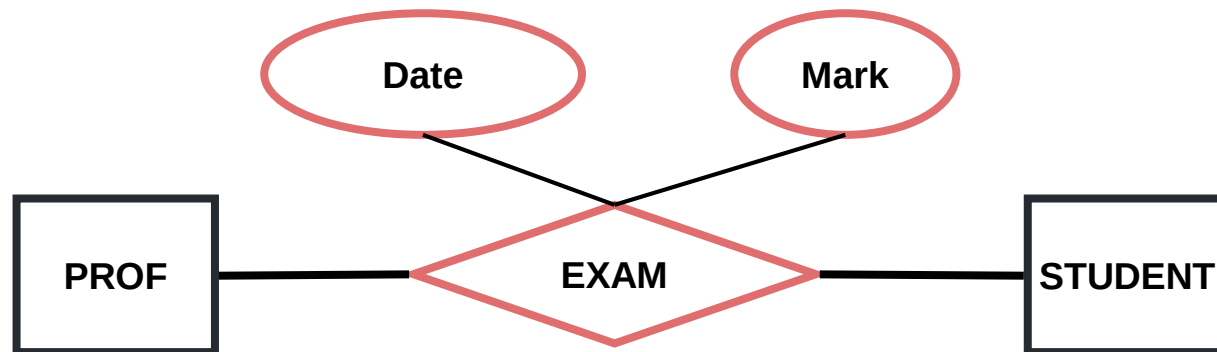
RELATIONSHIPS (3)

- Relationship-Mengen können auch Attribute besitzen

$$R \subseteq E_1 \times E_2 \times \dots \times E_n \times W(A_1) \times \dots \times W(A_m)$$

$$\text{d. h. } R = \{r = [e_1, e_2, \dots, e_n, a_1, a_2, \dots, a_m] \mid e_i \in E_i, a_j \in W(A_j)\}$$

- Beispiel



PROF (PNo, PName, Topic)

STUDENT (MatNo, SName, StartDate)

EXAM (PROF, STUDENT, Date, Mark)

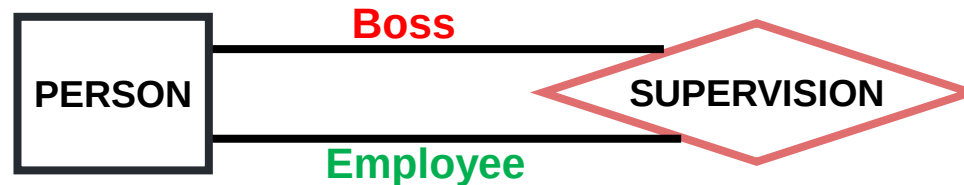
- Entities e_i können durch ihre Primärschlüssel repräsentiert werden z.B. Prüfung ($p1, s3, 11.7.2022, 2.3$)

RELATIONSHIPS (4)

- keine Disjunktheit der beteiligten Entity-Mengen gefordert (rekursive Beziehungen)

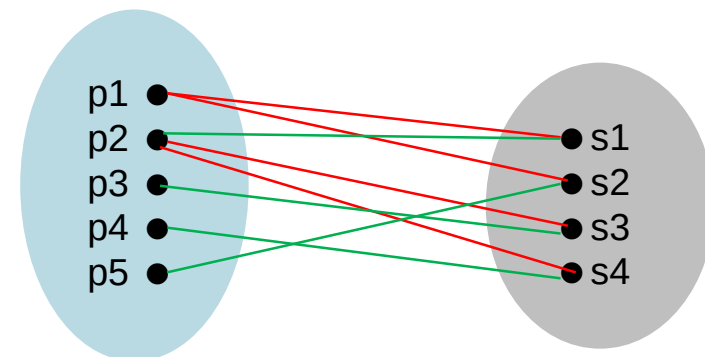
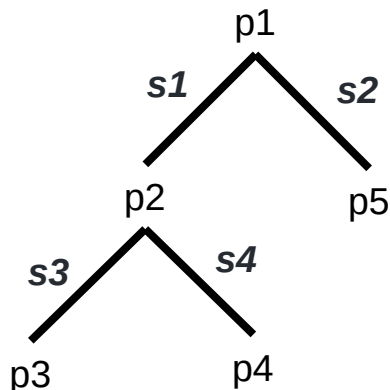
$\text{MARRIED} \subseteq \text{PERSON} \times \text{PERSON}$

$\text{SUPERVISION} \subseteq \text{PERSON} \times \text{PERSON}$



- Einführung von Rollennamen möglich (Reihenfolge !)

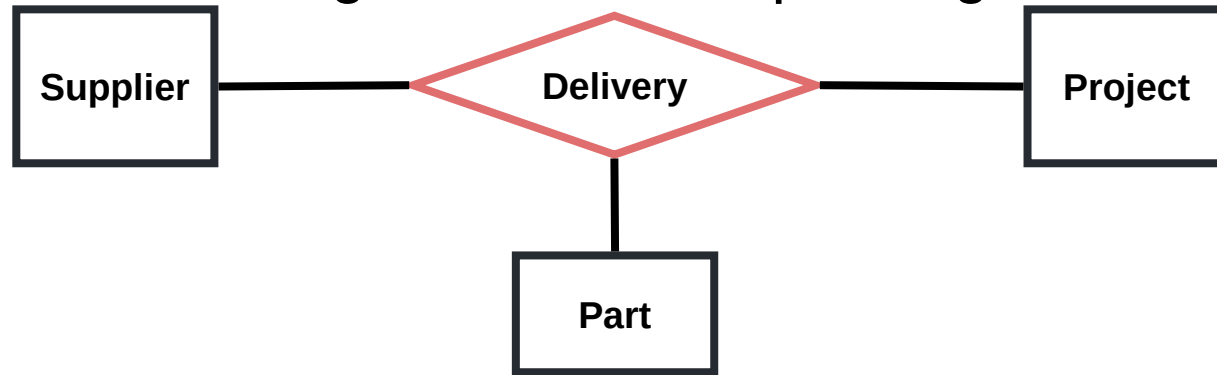
$\text{SUPERVISION} (\text{Boss: PERSON}, \text{Employee: PERSON})$



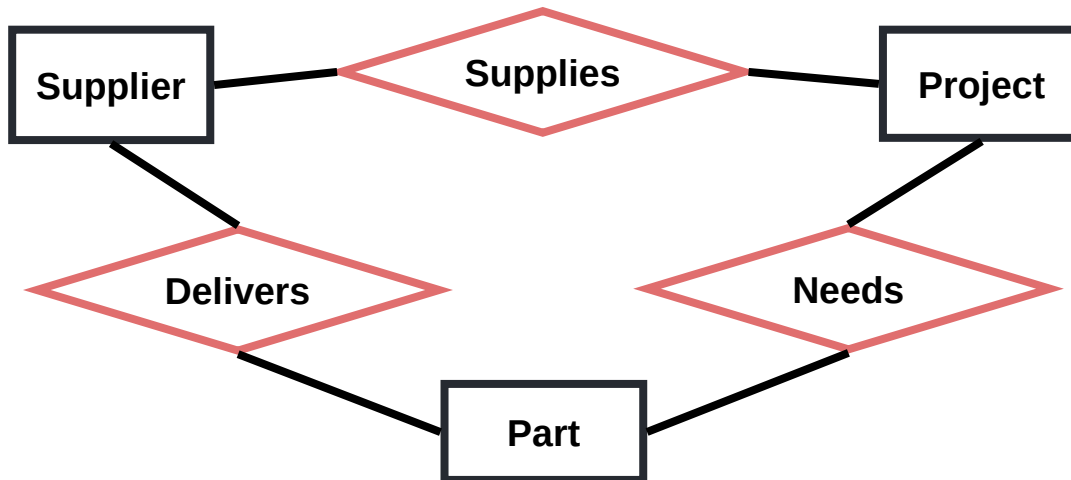


RELATIONSHIPS (5)

- Beispiel einer 3-stelligen Relationship-Menge



- nicht gleichwertig mit drei 2-stelligen (binären) Relationship-Mengen!



Beispiel:

S1 delivers part T1,
S1 supplies project P1,
P1 needs T1

impliziert nicht notwendigerweise

Delivery (S1, P1, T1)



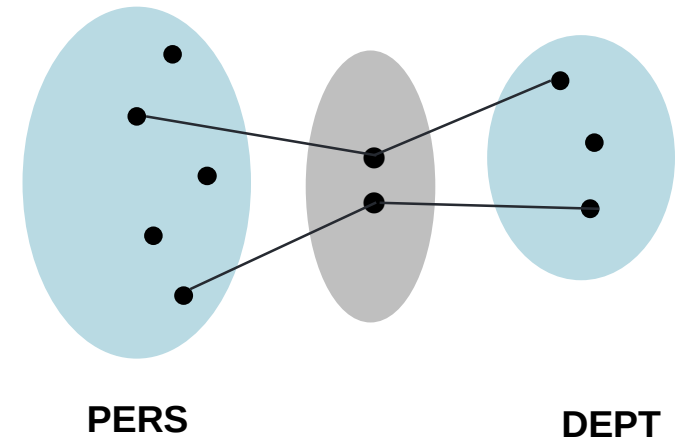
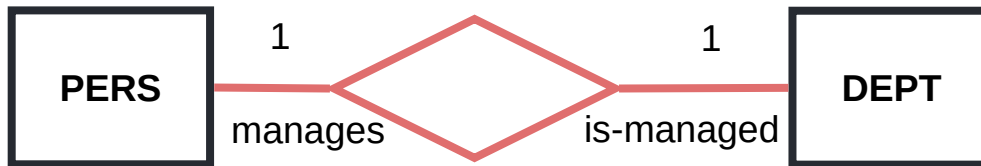
KARDINALITÄT VON BEZIEHUNGEN

- Festlegung wichtiger struktureller Integritätsbedingungen
- unterschiedliche Abbildungs- bzw. Beziehungstypen für binäre Beziehung zwischen Entity-Mengen E_i und E_j
 - **1:1** eineindeutige Funktion (bijektive Abbildung)
 - **n:1** mathematische Funktion (funktionale Abbildung)
 - **1:n** invers funktionale Abbildung
 - **n:m** mathematische Relation (komplexe Abbildung)
- Abbildungstypen implizieren nicht, dass für jedes $e \in E_i$ auch tatsächlich ein $e' \in E_j$ existiert!
 - n:1- sowie 1:1-Beziehungen repräsentieren somit i.a. nur partielle Funktionen
- Präzisierung der Kardinalitätsrestriktionen durch (Min-Max-) Multiplizitäten in UML

1:1-BEZIEHUNGEN

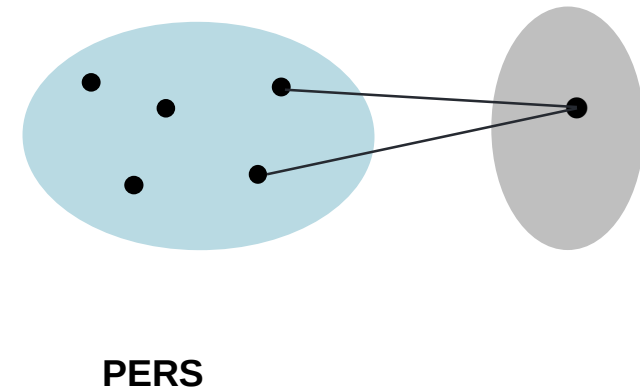
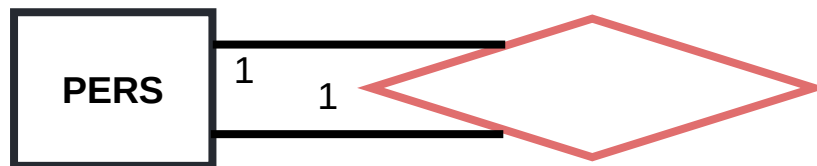
– 1:1-Beziehung zwischen unterschiedlichen Entity-Mengen

1:1 MANAGES/IS-MANAGED: PERS ↔ DEPT



– rekursive 1:1-Beziehung

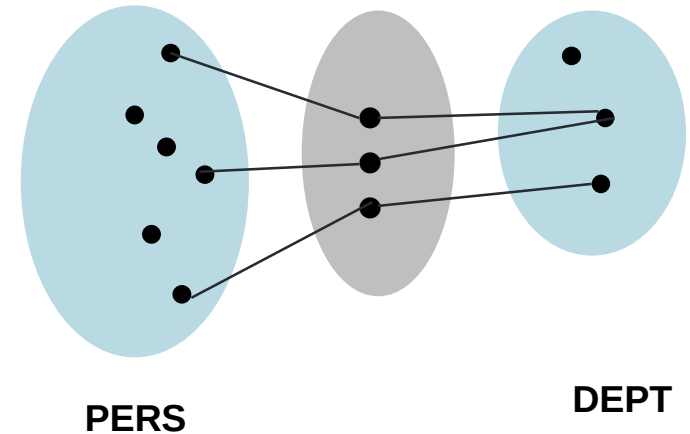
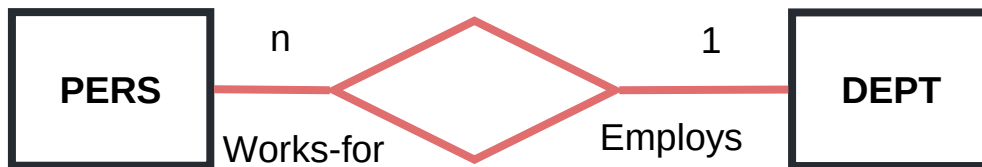
1:1 MARRIED: PERS ↔ PERS



N:1-BEZIEHUNG

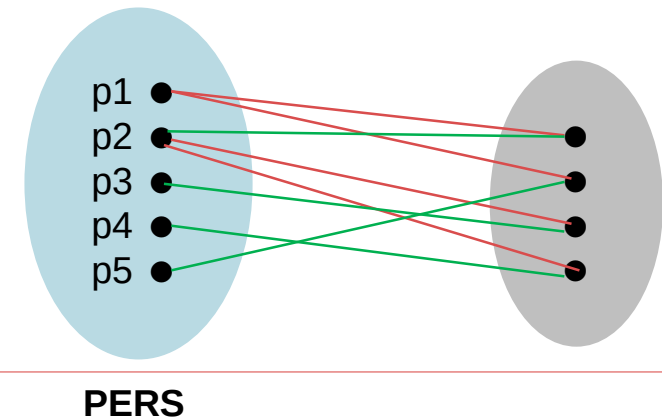
- n:1-Beziehung zwischen unterschiedlichen Entity-Mengen

n:1 WORKS-FOR/EMPLOYS: PERS → DEPT



- rekursive n:1-Beziehung

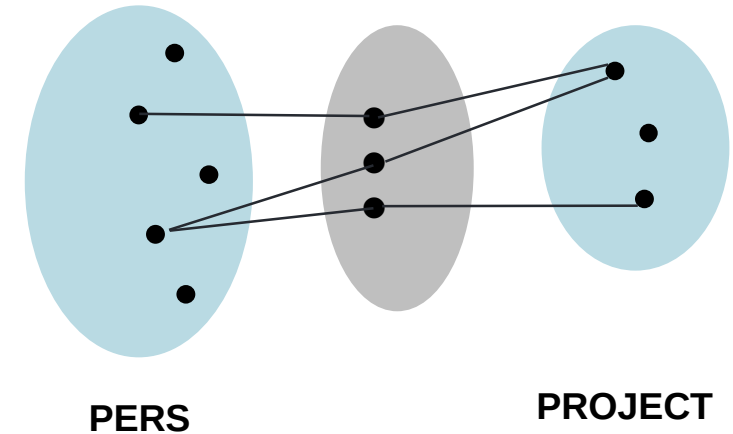
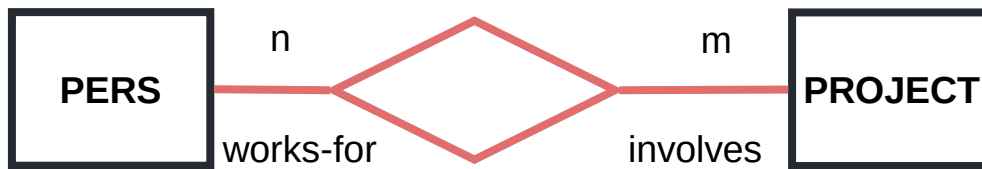
n:1 EMPLOYEE/BOSS: PERS → PERS



N:M-BEZIEHUNG

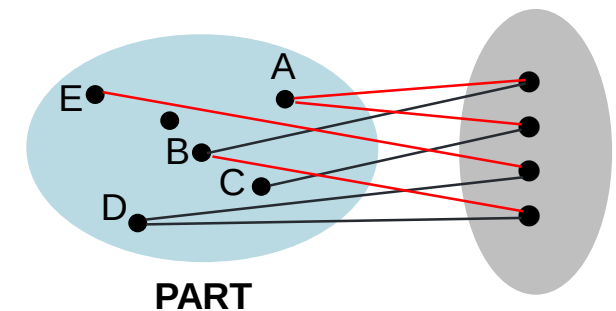
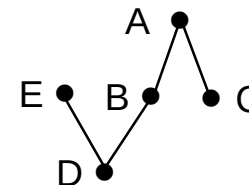
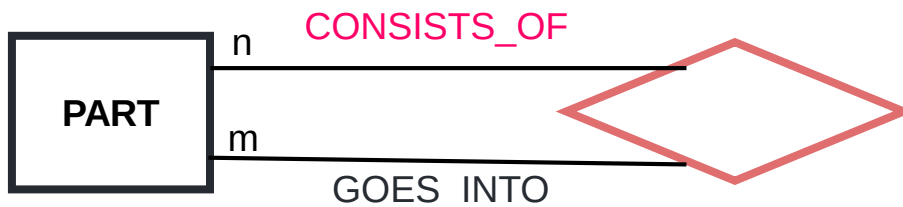
- n:m-Beziehung zwischen unterschiedlichen Entity-Mengen

n:m ... WORKS-FOR/INVOLVES: PERS — PROJECT

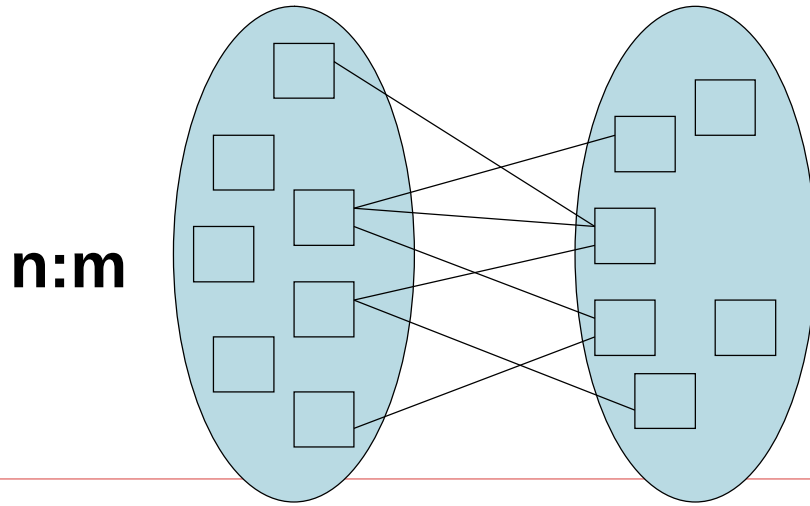
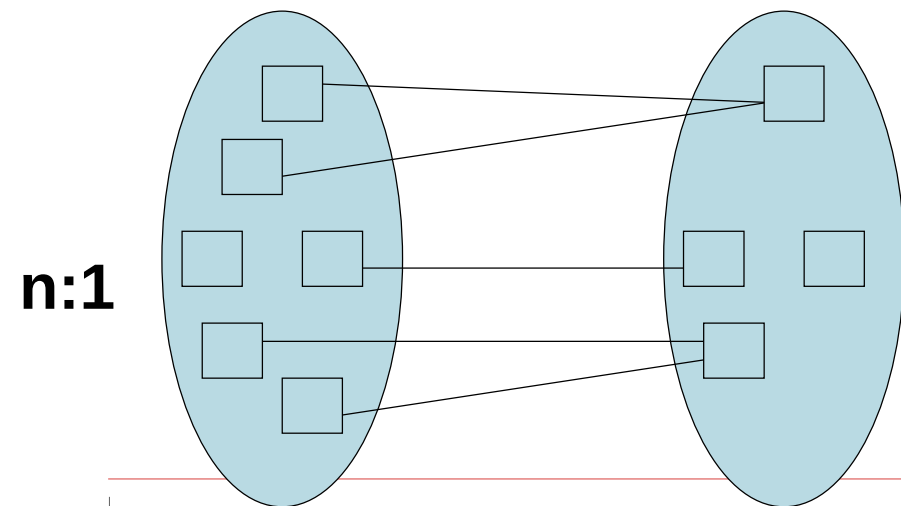
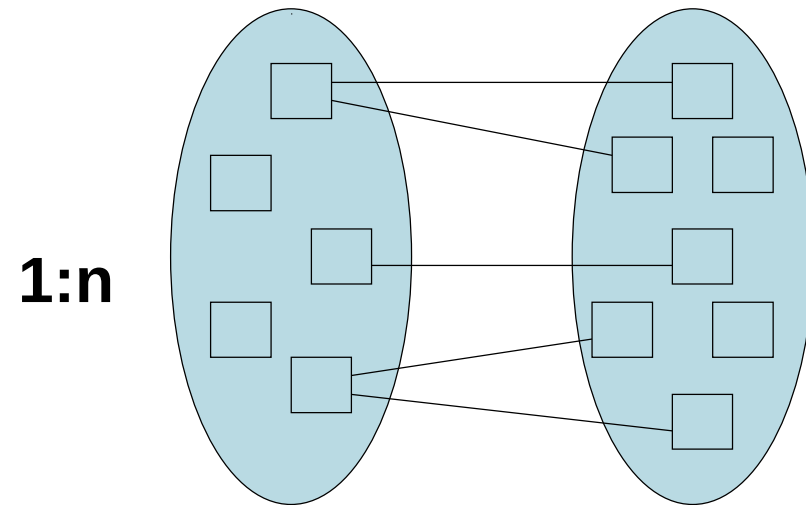
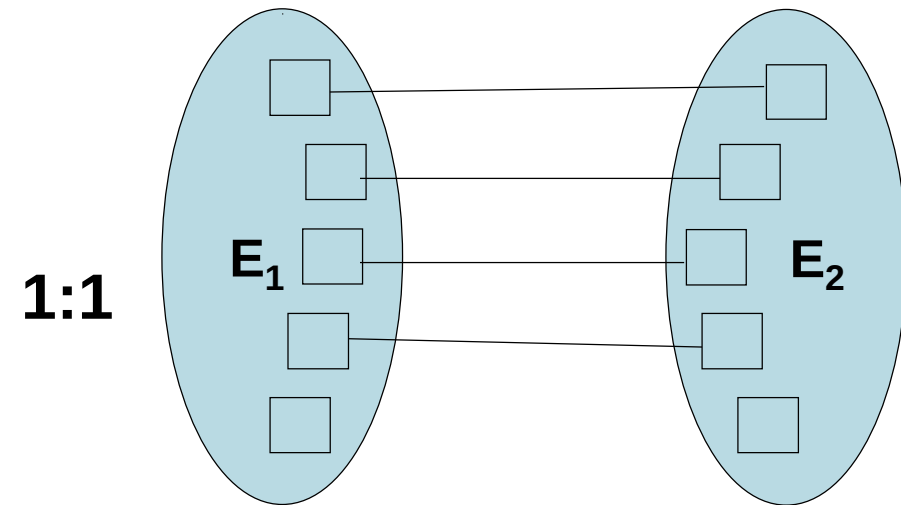
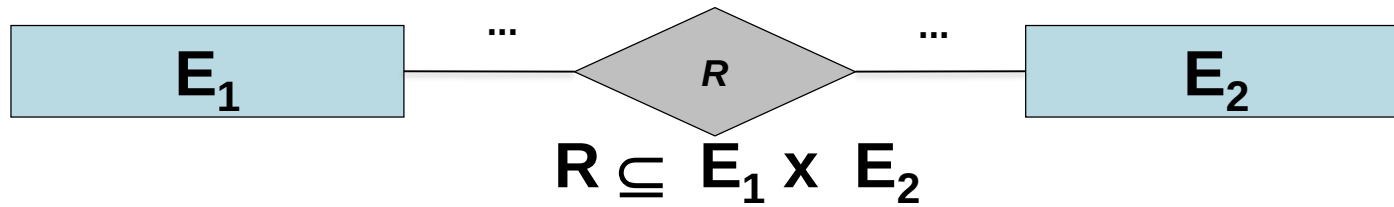


- rekursive n:m-Beziehung

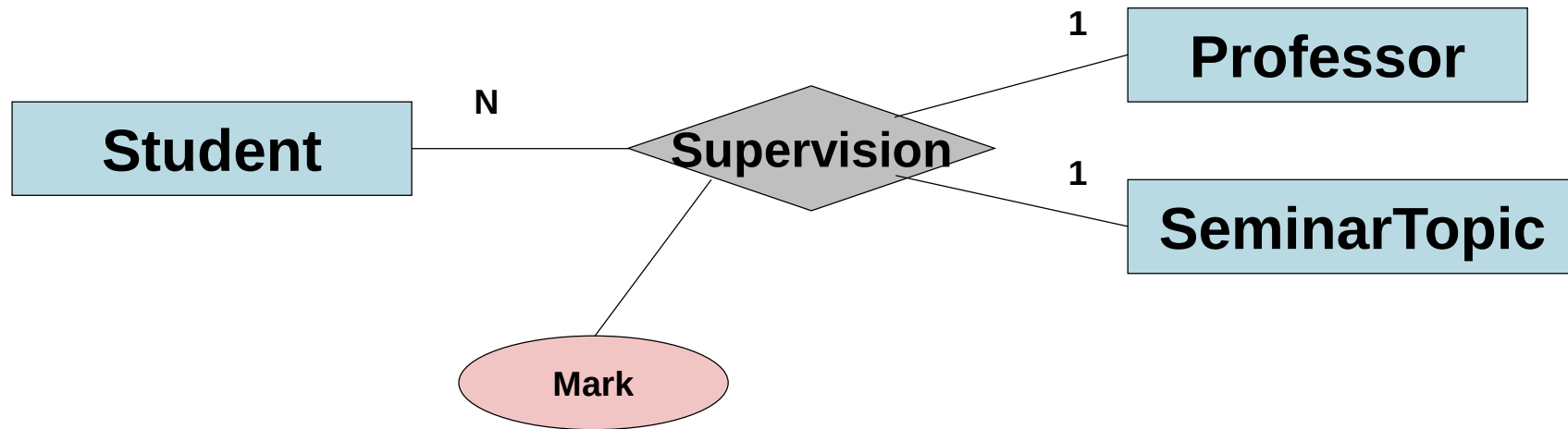
n:m CONSISTS_OF/ GOES INTO: PART — PART



ZUSAMMENFASSENDE ÜBERBLICK



KARDINALITÄTEN BEI MEHRSTELLIGEN BEZIEHUNGEN



Supervision: Professor x Student $\rightarrow$ SeminarTopic

Supervision: SeminarTopic x Student $\rightarrow$ Professor

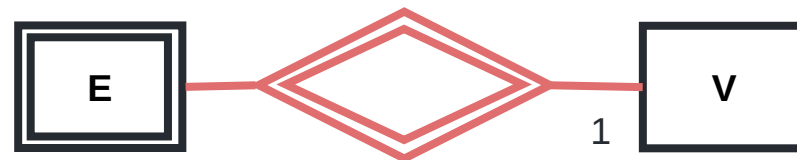
Kardinalitätsrestriktion 1 für Entity-Menge E bezieht sich auf fixe Kombination von Entities der anderen Entitätsmengen



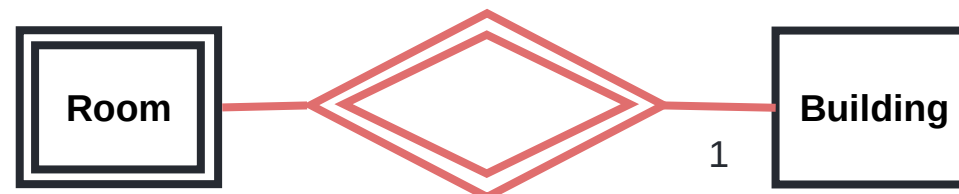
SCHWACHE ENTITY-MENGEN (WEAK ENTITIES)

- Entity-Menge mit Existenzabhängigkeit zu anderer Entity-Menge
 - kein eigener Schlüsselkandidat, sondern Identifikation über Beziehung zur übergeordneten Entity-Menge
 - Bsp.: Entity-Menge Room (RNo, Area) abhängig von Building
- Konsequenzen
 - jedes schwache Entity muss in Relationship-Menge mit übergeordneter Entity-Menge vertreten sein (obligatorische Beziehungsteilnahme)
 - Primärschlüssel teilweise von übergeordneter Entity-Menge abgeleitet
 - meist n:1 bzw. 1:1-Beziehung zwischen schwacher und übergeordneter Entity-Menge

– ER-Symbole:



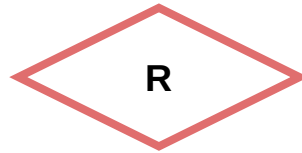
– Beispiel:



ÜBERBLICK ÜBER ER-DIAGRAMMSYMBOLS



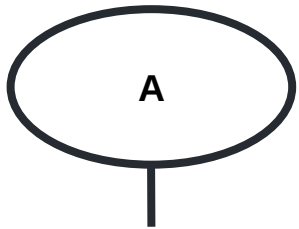
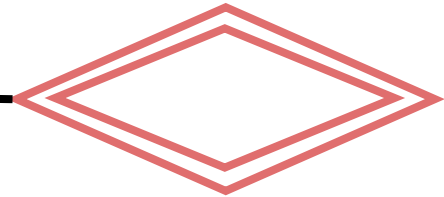
Entity-Menge



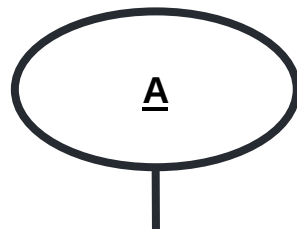
Relationship-Menge



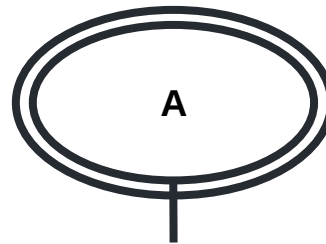
schwache Entity-Menge



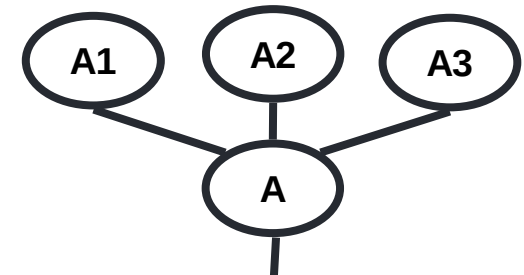
Attribut



Schlüsselattribut

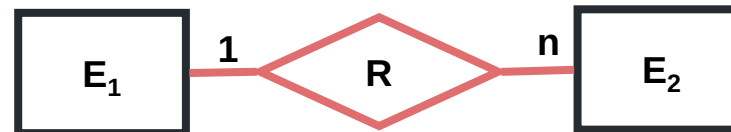


mehrwertiges Attribut



zusammengesetztes Attribut

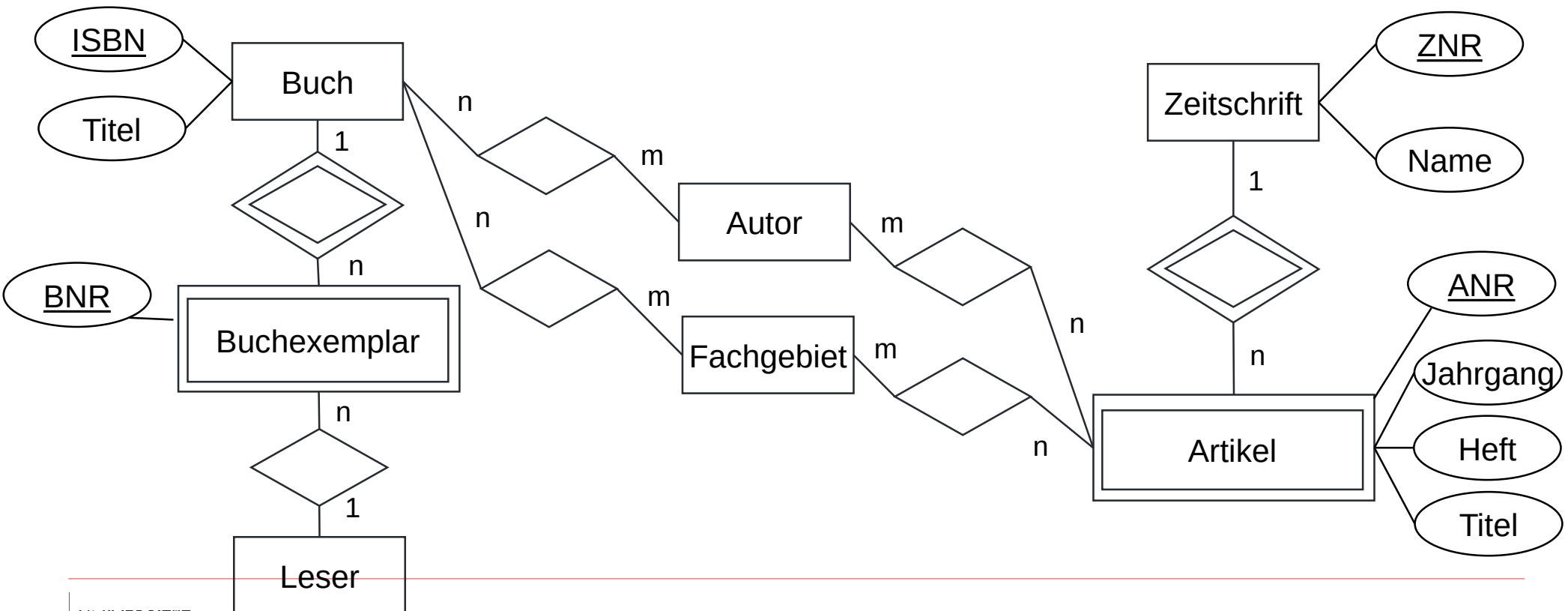
Beziehungstypen
(Kardinalitätsangaben)





ERM: ANWENDUNGSBEISPIEL

Eine Bibliothek besteht aus Büchern und Zeitschriften. Jedes Buch kann mehrere Autoren haben und ist eindeutig durch seine ISBN gekennzeichnet. Die Bibliothek besitzt teilweise mehrere Exemplare eines Buches. Zeitschriften dagegen sind jeweils nur einmal vorhanden. Die Artikel einer Zeitschrift erscheinen in einzelnen Heften eines Jahrgangs. Die Artikel sind ebenso wie Bücher einem oder mehreren Fachgebieten (z. B. Datenbanksysteme, Künstliche Intelligenz) zugeordnet. Ausgeliehen werden können nur Bücher (keine Zeitschriften).





UNIFIED MODELING LANGUAGE (UML)

- standardisierte graphische Notation/Sprache zur objektorientierten Modellierung
- Kombination unterschiedlicher Modelle bzw. Notationen, u.a.
 - Booch
 - Rumbaugh (OMT)
 - Jacobson (Use Cases)
- Standardisierung seit 1997 durch Herstellervereinigung OMG (Object Management Group)
- mehrere UML-Teile für unterschiedliche Phasen der Systementwicklung
 - Anwendungsfälle (use cases)
 - Aktivitätsdiagramme
 - Klassendiagramme ...
- Infos: www.uml.org



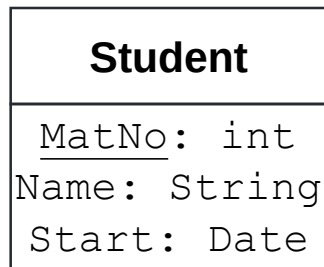


DARSTELLUNG VON KLASSEN UND OBJEKTEN

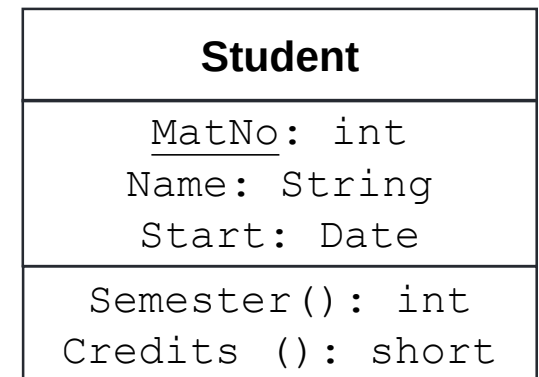
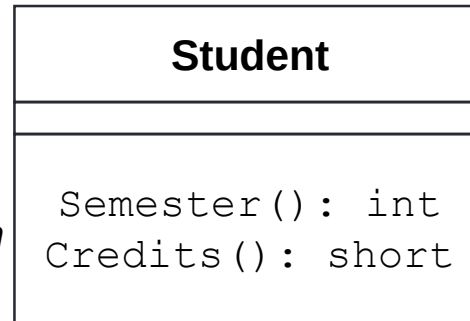
- Klassensymbol: Angabe von Klassenname, Attribute (optional), Methoden (optional)
 - i. a. werden nur relevante Details gezeigt



Attribute



Methoden





ASSOZIATIONEN

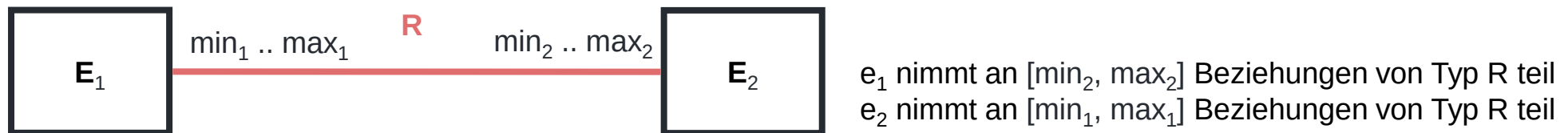
- Repräsentation von Beziehungen (relationships)
- optional: Festlegung eines Assoziationsnamens, seiner Leserichtung (▶ bzw. ◀), von Rollennamen und von Kardinalitätsrestriktionen





KARDINALITÄTSRESTRIKTIONEN IN UML

- Verfeinerung der Semantik eines Beziehungstyps durch Kardinalitätsrestriktionen
 - bisher nur grobe strukturelle Festlegungen (z. B.: 1:1 bedeutet “höchstens eins zu höchstens eins”)
 - Festlegung der minimalen Kardinalität
- Definition: für binäre Assoziation $R \subseteq E1 \times E2$
 - **Multiplizität** $\text{min}_1 \dots \text{max}_1$ ($\text{min}_2 \dots \text{max}_2$) bedeutet, dass zu jedem $E2$ ($E1$)- Element wenigstens min_1 (min_2) und höchstens max_1 (max_2) Instanzen von $E1$ ($E2$) in Beziehung stehen müssen (mit $0 \leq \text{min}_i \leq \text{max}_i$, $\text{max}_i \geq 1$)
 - Bezugnahme zur „gegenüberliegenden“ Klasse
- erlaubt Unterscheidung, ob Beziehungsteilnahme optional (Mindestkardinalität 0) oder obligatorisch (Mindestkardinalität ≥ 1) ist





KARDINALITÄTSRESTRIKTIONEN IN UML (2)

zulässige Multiplizitätsfestlegungen

x..y mindestens x, maximal y Objekte nehmen an der Beziehung teil

0..* optionale Teilnahme an der Beziehung

1..* obligatorische Teilnahme an Beziehung

0..1

1 genau 1

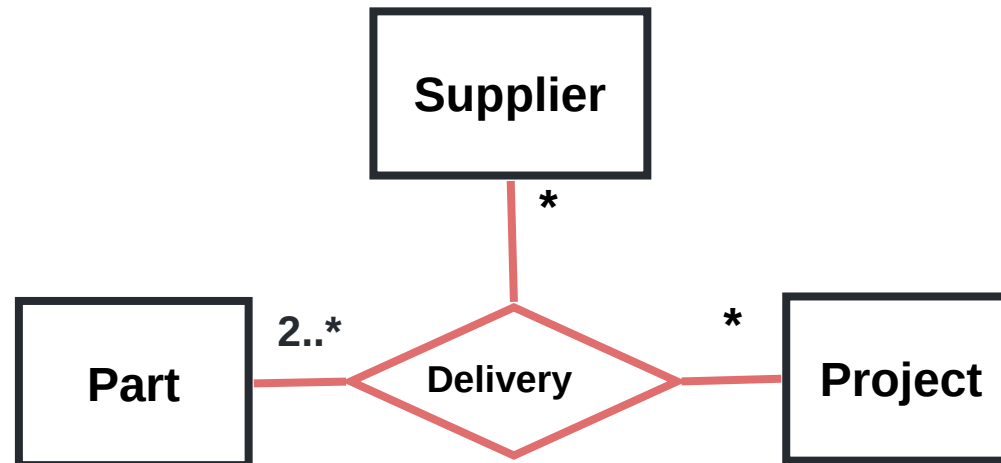
* „viele“



Beispiele

R	E ₁	E ₂	klassischer Beziehungstyp	min1..max1	min2..max2
Dept. Lead	DEPT	PERS	1:1	0..1	1..1
Party member	PARTY	PERS	1:n	0..1	7..*
Married	PERS	PERS	1:1	0..1	0..1
Attendance	LECTURE	STUDENT	n:m	0..*	3..800
Occupancy	PERS	ROOM	n:1	0..5	0..1

N-STELLIGE ASSOZIATIONEN



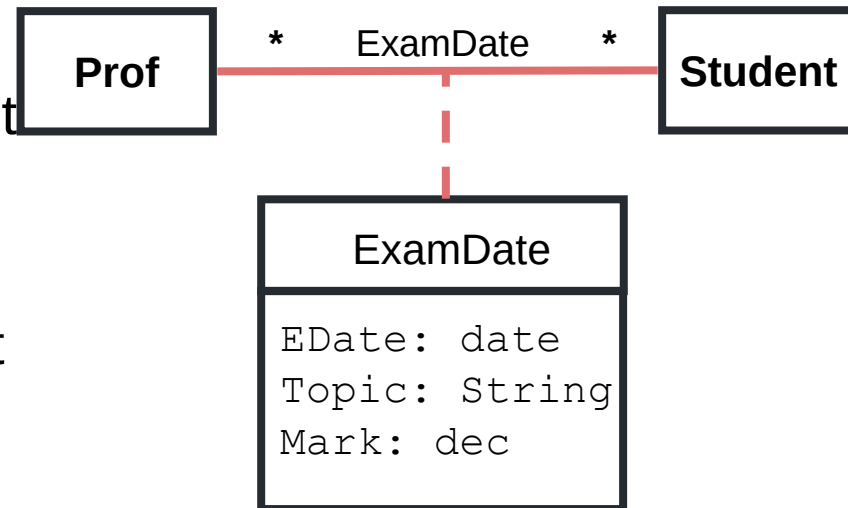
- Multiplizitätsangabe einer Klasse
 - regelt bei n-stelligen Beziehungen die Anzahl möglicher Instanzen (Objekte) zu einer fixen Kombination von je einem Objekt der übrigen n-1 Assoziationsenden
- Beispiel 3-stellige Assoziation Lieferung
 - pro Projekt und Lieferant müssen wenigstens 2 unterschiedliche Teile geliefert werden



WEITERE ASSOZIATIONEN

– Assoziations-Klassen

- notwendig für Beziehungen mit eigenen Attributen
- gestrichelte Linie
- Name der A.-Klasse entspricht dem der Assoziation



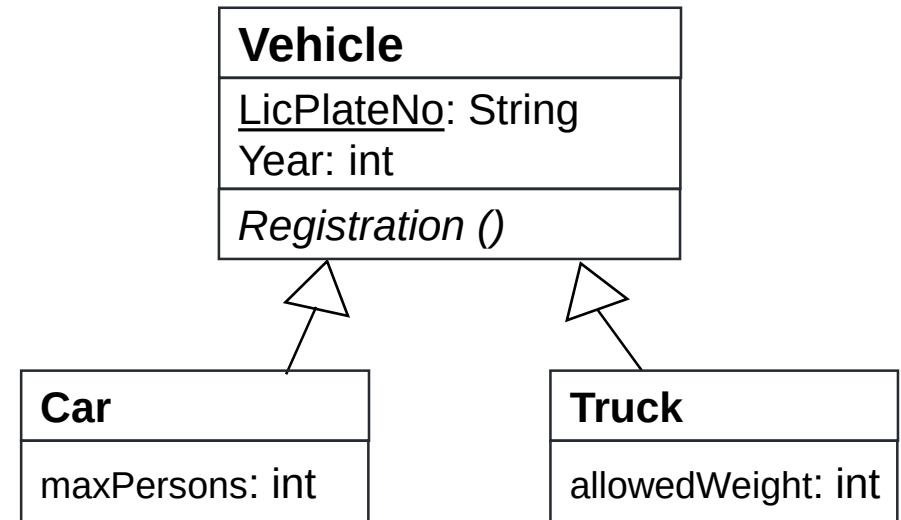
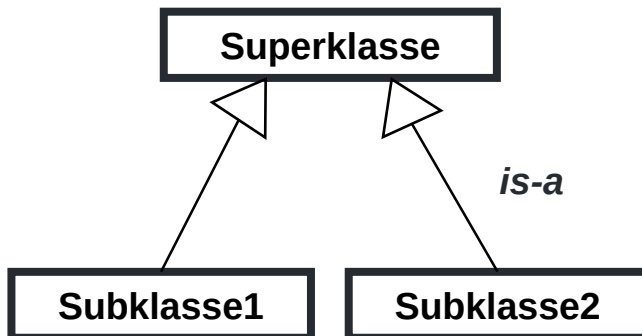
– gerichtete Assoziation

- Einschränkung der Navigierbarkeit: keine direkte Navigationsmöglichkeit in umgekehrter Richtung (einfachere Implementierung)
- auf konzeptioneller Ebene nicht notwendigerweise festzulegen



IS-A-BEZIEHUNGEN

- Is-A-Beziehung zwischen Klassen (Entity-Mengen)
 - K_1 is-a K_2 : jedes Objekt aus Subklasse K_1 ist auch ein Objekt der Superklasse K_2 , jedoch mit zusätzlichen strukturellen Eigenschaften
 - *Substitutionsprinzip*: alle Objekte (Instanzen) einer Subklasse sind auch Instanzen der Superklasse
- Vererbung von Eigenschaften (Attribute, Integritätsbedingungen, Methoden ...) der Superklasse an alle Subklassen
 - Wiederverwendbarkeit, Erweiterbarkeit
 - keine Wiederholung von Beschreibungsinformation, Fehlervermeidung





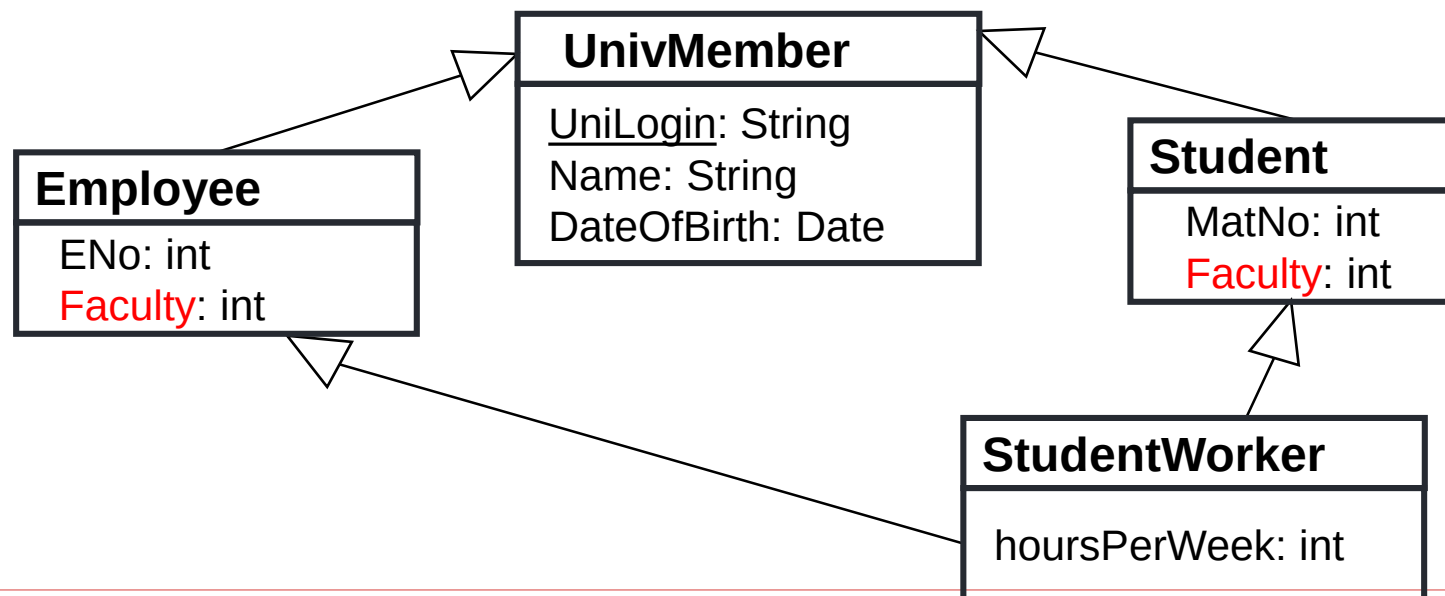
GENERALISIERUNG/SPEZIALISIERUNG

- Is-A-Beziehungen realisieren Abstraktionskonzepte der Generalisierung und Spezialisierung
- Generalisierung: Bottom-Up-Vorgehensweise
 - Bildung allgemeinerer Superklassen aus zugrundeliegenden Subklassen
 - Übernahme gemeinsamer Eigenschaften und Unterdrückung spezifischer Unterschiede
 - rekursive Anwendbarkeit => Generalisierungshierarchie
- Spezialisierung: Top-Down-Vorgehensweise
 - zuerst werden die allgemeineren Objekte (Superklassen), dann die spezielleren (Subklassen) beschrieben



GENERALISIERUNG/SPEZIALISIERUNG (2)

- oft keine reine Hierarchien, sondern Netzwerke (n:m)
 - eine Klasse kann Subklasse mehrerer Superklassen sein
 - ein Objekt kann gleichzeitig Instanz verschiedener Klassen sein
 - Zyklen nicht erlaubt/sinnvoll (A is-a B, B is-a A)
- führt zum Problem der Mehrfach-Vererbung
 - Namenskonflikte möglich
 - benutzergesteuerte Auflösung, z. B. durch Umbenennung





SPEZIALISIERUNG: DEFINITIONEN

- **Klasse**: Menge von Objekten / Entities (Entity-Mengen)
- **Subklasse**: Klasse S , deren Objekte eine Teilmenge einer Superklasse G sind (is-a-Beziehung), d. h. $S \subseteq G$
d. h. jedes Element (Ausprägung/Instanz) von S ist auch Element von G
- **Spezialisierung**: $Z(G) = \{S_1, S_2, \dots, S_n\}$

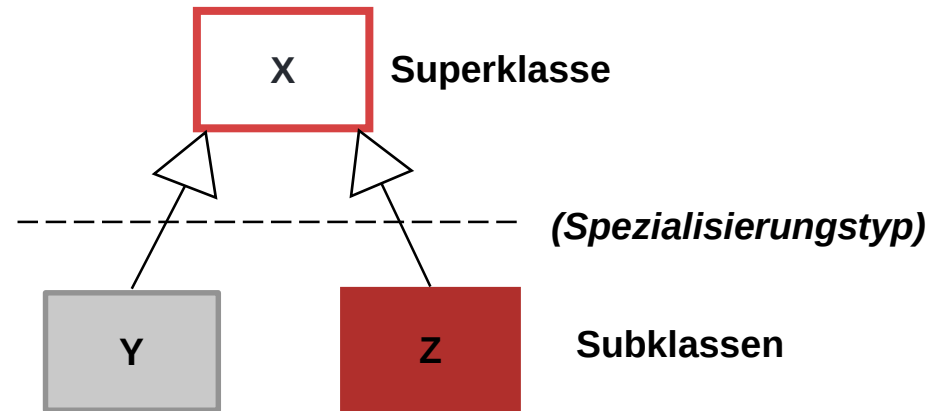
Menge von Subklassen S_i mit derselben Superklasse G

- zusätzliche Integritätsbedingungen: Vollständigkeit (Überdeckung) und Disjunktheit von Spezialisierungen

Z heisst **vollständig (complete)**, falls gilt: $G = \bigcup S_i$ ($i = 1..n$)
andernfalls **partiell (incomplete)**.

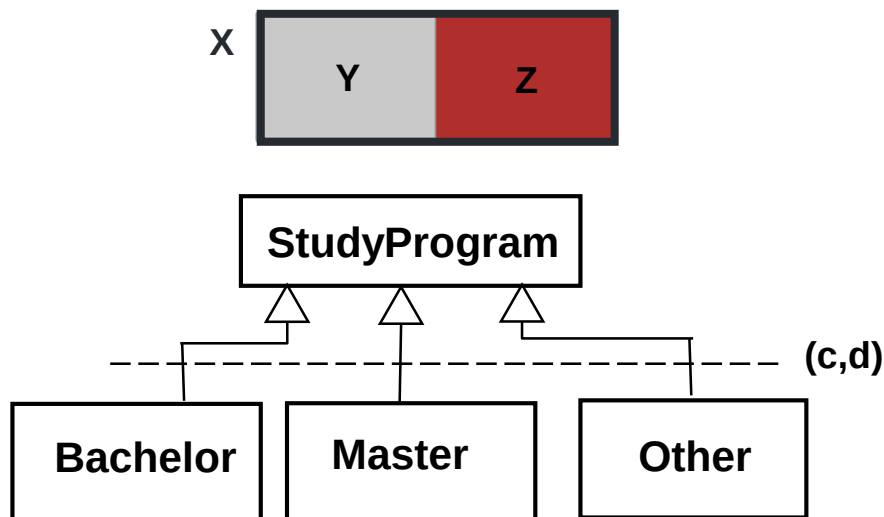
Z ist **disjunkt (disjoint)**, falls $S_i \cap S_j = \{ \}$ für $i \neq j$
andernfalls **überlappend (overlapping)**.

ARTEN VON SPEZIALISIERUNGEN

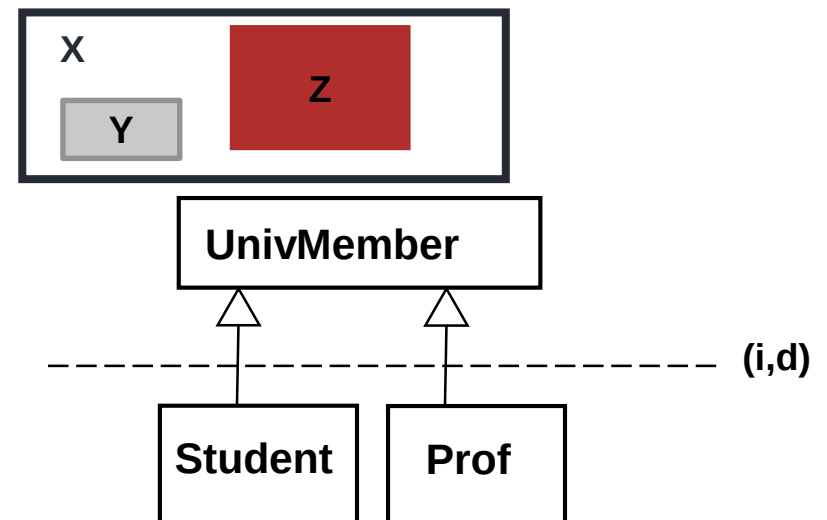


– disjunkte Spezialisierungen (Partitionierung)

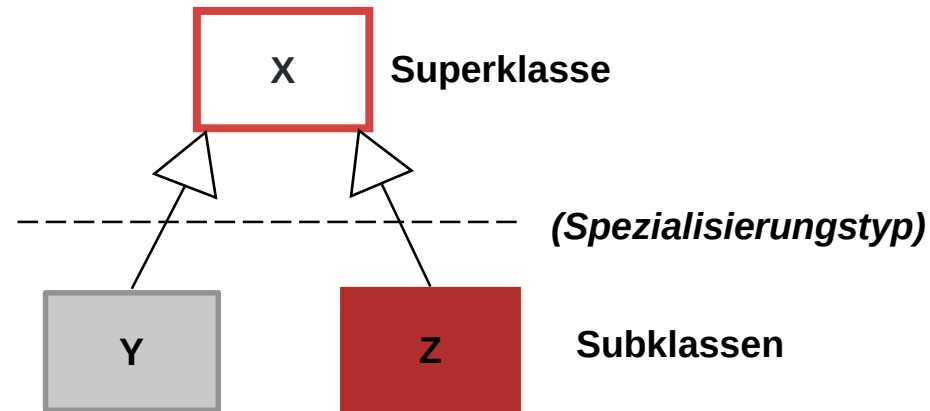
vollständig, disjunkt (*complete, disjoint*)



partiell, disjunkt (*incomplete, disjoint*)

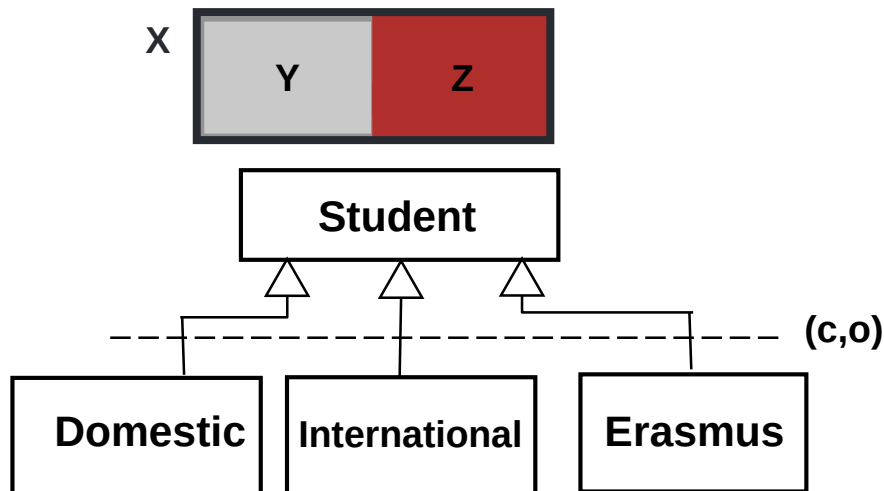


ARTEN VON SPEZIALISIERUNGEN (2)

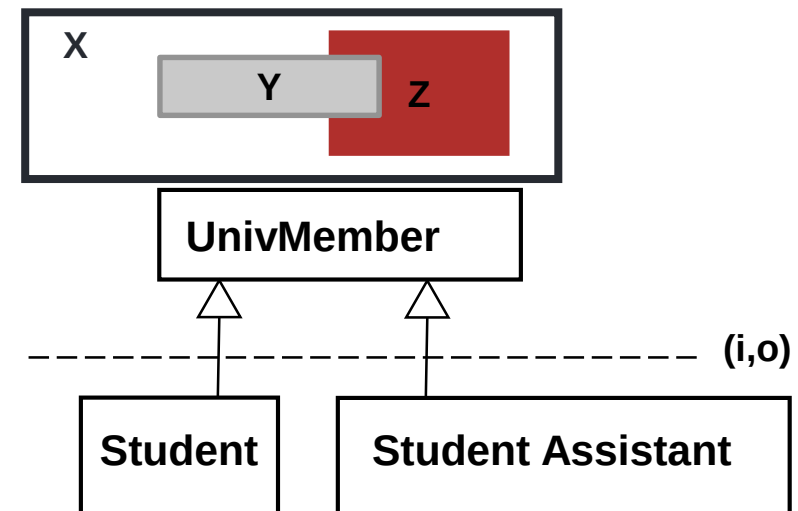


– überlappende Spezialisierungen

vollständig, überlappend (*complete, overlapping*)



partiell, überlappend (*incomplete, overlapping*)



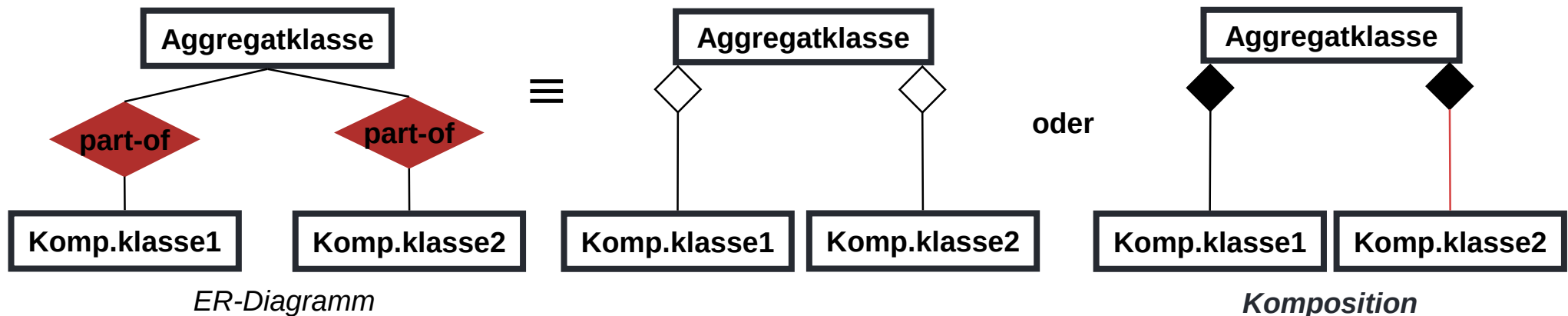


AGGREGATION

- Objekte werden als Zusammensetzung von anderen Objekten angesehen
 - eine Kombination von einfachen (atomaren, d.h. nicht weiter zerlegbaren) Objekten (Element, Teil) wird betrachtet als zusammengesetztes oder komplexes Objekt (Aggregatobjekt)
 - rekursive Anwendung des Aggregatsprinzips: Aggregationsobjekte mit komplexen Komponenten
- einfache Formen der Aggregation:
 - zusammengesetzte Attribute
 - Klasse (Entity-Menge) als Aggregation verschiedener Attribute
- Erweiterung auf Part-of-Beziehung zwischen Klassen

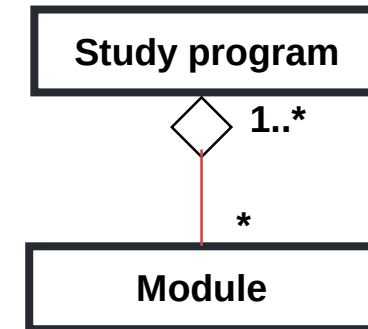
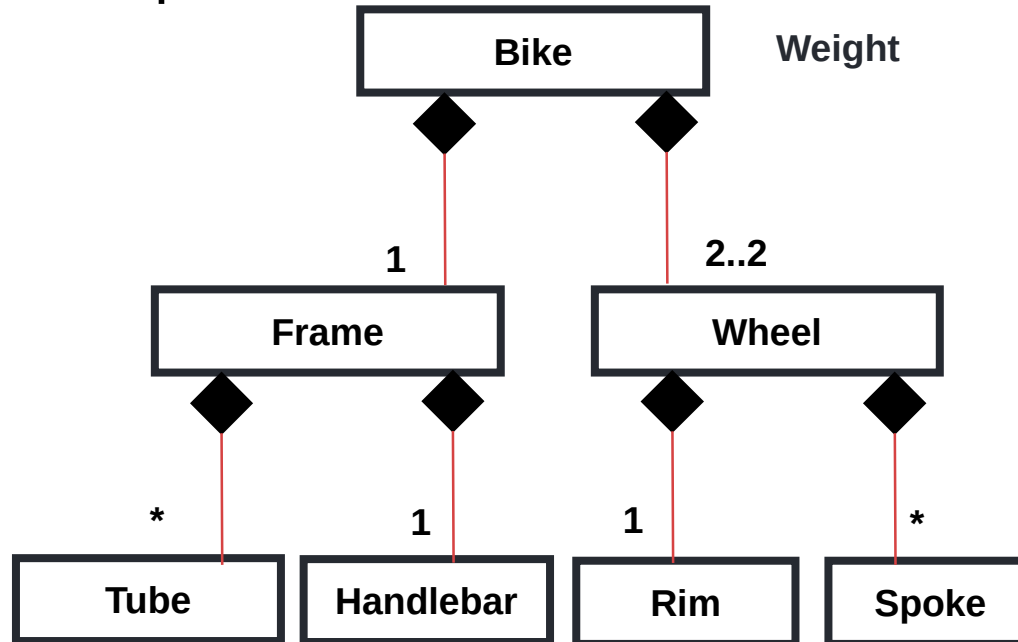
AGGREGATION (2)

- Part-of-Beziehung (Teil-von-Beziehung) zwischen Komponenten- und Aggregatobjekten
 - Elemente einer Subkomponente sind auch Elemente aller Superkomponenten dieser Subkomponente
 - Referenzsemantik ermöglicht, dass ein Objekt gleichzeitig Element verschiedener Komponenten bzw. Subkomponente von mehreren Superkomponenten ist → Netzwerke, (n:m) !
 - Wertesemantik (**Komposition**): Teil-Objekt gehört genau zu einem Aggregat-Objekt; Existenzabhängigkeit (wie für weak entities)!



AGGREGATION (3)

– Beispiele

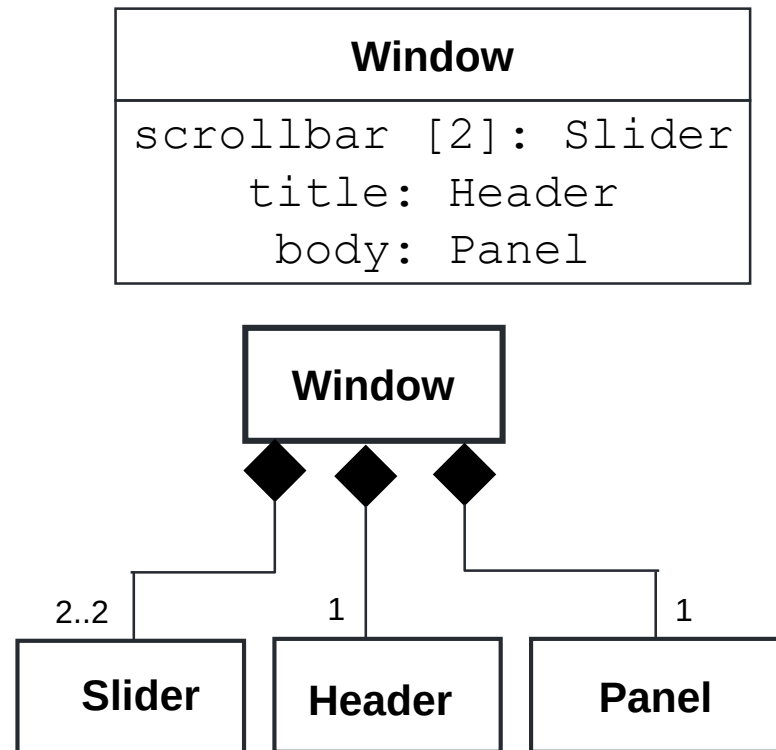


- Unterstützung komplex-strukturierter Objekte
- heterogene Komponenten möglich
- keine Vererbung !



AGGREGATION: KOMPOSITION

- Komponentenkasse einer Aggregatklasse A entspricht einem Attribut von A
 - aufgrund der Wertesemantik der Komposition
- äquivalente Repräsentationen:







ZUSAMMENFASSUNG

- DB-Entwurf umfasst
 - Informationsanalyse
 - konzeptueller Entwurf (→ Informationsmodell)
 - logischer Entwurf (→ logisches DB-Schema)
 - physischer Entwurf (→ physisches DB-Schema)
- Informationsmodellierung mit dem ER-Modell
 - Entity-Mengen und Relationship-Mengen
 - Attribute, Wertebereiche, Primärschlüssel
 - Beziehungstypen (1:1, n:1, n:m)
 - Diagrammdarstellung
- UML-Klassendiagramme: Unterschiede zu ER-Modell
 - standardisiert
 - Spezifikation von Verhalten (Methoden), nicht nur strukturelle Aspekte
 - genauere Kardinalitätsrestriktionen (Multiplizitäten)
 - Unterstützung der Abstraktionskonzepte der Generalisierung / Spezialisierung, Aggregation / Komposition
- keine festen Regeln zur eigentlichen Informationsmodellierung (i.a. viele Modellierungsmöglichkeiten einer bestimmten Miniwelt)



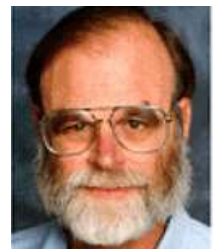
3. GRUNDLAGEN DES RELATIONALEN DATENMODELLS

- Grundkonzepte
- Relationale Invarianten
 - Primärschlüsselbedingung, Fremdschlüsselbedingung (referentielle Integrität)
 - Wartung der referentiellen Integrität
- Abbildung ERM / UML → RM
- Nachbildung von Generalisierung und Aggregation im RM
-
- Kapitel 4: Relationenalgebra
- Kapitel 5: Standard-Anfragesprache SQL
- Kapitel 6: Logischer DB-Entwurf (Normalformenlehre)
- Kapitel 7/8: Datendefinition und -kontrolle
- DB-Anwendungsprogrammierung: ->DBS2



RELATIONENMODELL - ENTWICKLUNG

- 1969/1970: Vorschlag von Edgar F. Codd (IBM)
 - A Relational Model of Data for Large Shared Data Banks. Commun. ACM 13(6): 377-387 (1970)
 - Konzept, Relationenalgebra
- ab ca.1975: erste Prototypen relationaler DBS
 - System R (IBM Research, San Jose), u.a. Query-Sprache SEQUEL / SQL (D. Chamberlin), ACID-Techniken (Jim Gray et al.), Query-Optimierung etc.
 - Ingres (Berkeley Univ.) unter Leitung von Mike Stonebraker, Query-Sprache QUEL
- seit ca. 1980: kommerzielle relationale DBS
 - zunächst Oracle (Larry Ellison): 1979 „Version 2“
 - IBM DB2 ...





RELATIONENMODELL - ÜBERSICHT

- Datenstruktur: Relation (Tabelle)

- einzige Datenstruktur
(neben atomaren Werten)
- alle Informationen ausschließlich
durch Werte dargestellt
- Integritätsbedingungen auf/zwischen Relationen: relationale
Invarianten

- Operatoren auf (mehreren) Relationen

- Vereinigung, Differenz
- kartesisches Produkt
- Projektion
- Selektion
- zusätzlich: Änderungsoperationen (Einfügen, Löschen, Ändern)



RELATIONENMODELL - GRUNDKONZEPTE



- **Relationsschema:** $R (A_1, A_2, \dots, A_i, \dots, A_n)$ Name der Relation und Attributnamen mit jeweiligen Wertebereichen $W(A_i)$
- **Relation:** $r(R) \subseteq W(A_1) \times W(A_2) \times \dots \times W(A_n)$
 - Relation = Untermenge des kartesischen Produktes der Attributwertebereiche
 - nur einfache Attribute (atomare Werte), also keine Unterstützung für mehrwertige/zusammengesetzte Attribute!
- Darstellungsmöglichkeit für $r(R)$: n-spaltige Tabelle
 - **Grad** der Relation: n
 - **Kardinalität:** Anzahl der Sätze (Tupel)

NORMALISIERTE RELATIONEN IN TABELLENDARSTELLUNG



- Grundregeln:
 - jede Zeile (Tupel) ist eindeutig und beschreibt ein Objekt (Entity) der Miniwelt
 - Reihenfolge der Zeilen ist ohne Bedeutung
 - Reihenfolge der Spalten ist ohne Bedeutung, da sie eindeutigen (Attribut-) Namen tragen
 - jeder Datenwert innerhalb einer Relation ist ein atomares Datenelement
 - alle für Benutzer relevanten Informationen sind ausschließlich durch Datenwerte ausgedrückt
- Darstellung von Beziehungen durch Fremdschlüssel (foreign key)
 - Attribut oder Attributkombination, das in Bezug auf den Primärschlüssel einer anderen (oder derselben) Relation definiert ist (gleicher Wertebereich)

FACULTY

<u>FNO</u>	FNAME	...
WI	Wirtschaftswiss.	...
MI	Math./Informatik	...

STUDENT

<u>MATNO</u>	SNAME	<u>FNO</u>	CITY
123 766	Coy	MI	Halle
654 711	Abel	WI	Leipzig
196 481	Maier	MI	Delitzsch
226 302	Schulz	MI	Leipzig



RELATIONALE INVARIANTEN

- inhärente Integritätsbedingungen des Relationenmodells (Modellbedingungen)

1. Primärschlüsselbedingung (Entity-Integrität)

- Eindeutigkeit des Primärschlüssels / Minimalität
- keine Nullwerte!

2. Fremdschlüsselbedingung (referentielle Integrität):

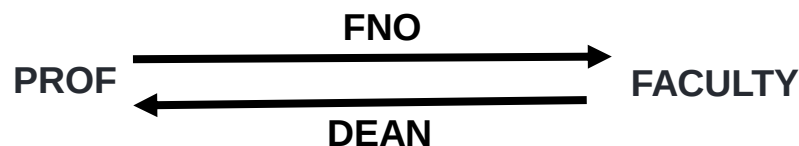
- zugehöriger Primärschlüssel muss existieren
- d.h. zu jedem Wert (ungleich Null) eines Fremdschlüsselattributs einer Relation R muss ein gleicher Wert des Primärschlüssels in irgendeinem Tupel von Relation S vorhanden sein
- graphische Notation:





RELATIONALE INVARIANTEN (2)

- Fremdschlüssel und zugehöriger Primärschlüssel tragen wichtige interrelationale (bzw. intrarelationale) Informationen
 - gleicher Wertebereich
 - gestatten Verknüpfung von Relationen
- Fremdschlüssel
 - können Nullwerte aufweisen, wenn sie nicht Teil eines Primärschlüssels sind.
 - Fremdschlüssel ist „zusammengesetzt“, wenn zugehöriger Primärschlüssel „zusammengesetzt“ ist
- eine Relation kann mehrere Fremdschlüssel besitzen, die die gleiche oder verschiedene Relationen referenzieren
- Zyklen sind möglich (geschlossener referentieller Pfad)
- eine Relation kann zugleich referenzierende und referenzierte Relation sein (selbstreferenzierende Tabelle)



PERS (PNO, Name, ..., MGR)



RELATIONALE INVARIANTEN (3)

- DDL-Spezifikation in SQL bei CREATE TABLE
 - PRIMARY KEY- und FOREIGN KEY-Klauseln
 - Angaben für Attributdefinition:
 - Attributname sowie Datentyp
 - Default-Werte
 - Eindeutigkeit (UNIQUE bzw. PRIMARY KEY)
 - FOREIGN KEY-Klausel
 - Verbot von Nullwerten (NOT NULL)

```
CREATE TABLE STUDENT (  
    MATNO INT,  
    SNAME VARCHAR (50) NOT NULL,  
    FNO INT,  
    PRIMARY KEY (MATNO) ,  
    FOREIGN KEY (FNO) REFERENCES FACULTY)  
CREATE TABLE FACULTY (  
    FNO INT PRIMARY KEY,  
    FNAME VARCHAR(50) NOT NULL,  
    DEAN INT,  
    FOREIGN KEY (DEAN) REFERENCES PROF ...)
```



WARTUNG DER REFERENTIELLEN INTEGRITÄT

- Gefährdung bei INSERT, UPDATE, DELETE



- **Fall 0:** INSERT auf S, DELETE auf R
 - keine Auswirkungen für referentielle Integrität
- **Fall 1:** INSERT in der referenzierenden (abhängigen) Relation R bzw. UPDATE auf Fremdschlüssel in R
 - Ablehnung falls kein zugehöriger Primärschlüssel-Wert in referenzierter Relation S besteht
- **Fall 2:** DELETE auf referenzierter Relation S bzw. UPDATE von Primärschlüssel von S
 - unterschiedliche Folgeaktionen auf referenzierender Relation R möglich, um referentielle Integrität zu wahren



WARTUNG DER REFERENTIELLEN INTEGRITÄT (2)

- SQL-Standard erlaubt Spezifikation der referentiellen Aktionen für jeden Fremdschlüssel
- sind Nullwerte verboten?
 - NOT NULL
- Löschregel für Zielrelation (referenzierte Relation S):
 - ON DELETE (NO ACTION | CASCADE | SET NULL | SET DEFAULT)
- Änderungsregel für Ziel-Primärschlüssel (Primärschlüssel oder Schlüsselkandidat):
 - ON UPDATE (NO ACTION | CASCADE | SET NULL | SET DEFAULT)





WARTUNG DER REFERENTIELLEN INTEGRITÄT (3)

- Lösch- und Änderungsregeln
 - NO ACTION (Voreinstellung): Operation wird nur zugelassen, wenn keine zugehörigen Sätze (Fremdschlüsselwerte) vorhanden sind. Es sind folglich keine referentiellen Aktionen auszuführen
 - CASCADE: Operation „kaskadiert“ zu allen zugehörigen Sätzen
 - SET NULL: Fremdschlüssel wird in zugehörigen Sätzen zu “Null” gesetzt (nicht zulässig, wenn Attribut nicht NULL sein darf)
 - SET DEFAULT: Fremdschlüssel wird auf einen benutzerdefinierten Default-Wert gesetzt

```
CREATE TABLE S (  
    A2 INT PRIMARY KEY)  
CREATE TABLE R (  
    A1 INT PRIMARY KEY,  
    A2 INT,  
    FOREIGN KEY (A2) REFERENCES S ON (DELETE | UPDATE)  
        (NO ACTION | CASCADE | SET NULL | SET DEFAULT) )
```

BEISPIEL WARTUNG REFERENTIELLE INTEGRITÄT



STUDENT

<u>MATNO</u>	SNAME	CITY
1266	Coy	Halle
6511	Abel	Leipzig
1981	Maier	Delitzsch
2202	Schulz	Leipzig

```
DELETE FROM STUDENT  
WHERE MATNO=1981
```

EXAM

<u>EID</u>	<u>MATNO</u>	TOPIC	DATE	MARK
1780	6511	FA	19.9.	2
1781	1981	DBS	15.10.	1
1782	6511	DBS	17.4.	2
1783	1981	KI	25.3.	3

```
CREATE TABLE EXAM (...  
    MATNO INT, NOT NULL, REFERENCES  
    STUDENT ON DELETE <Action>)
```

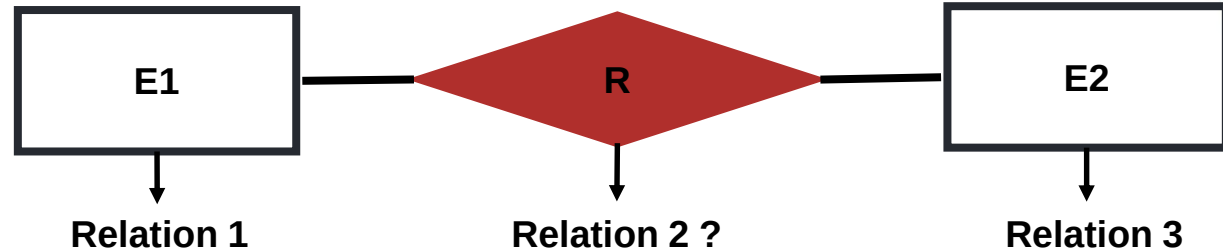
– Auswirkungen von **<Action>**

– NO ACTION:

– CASCADE:

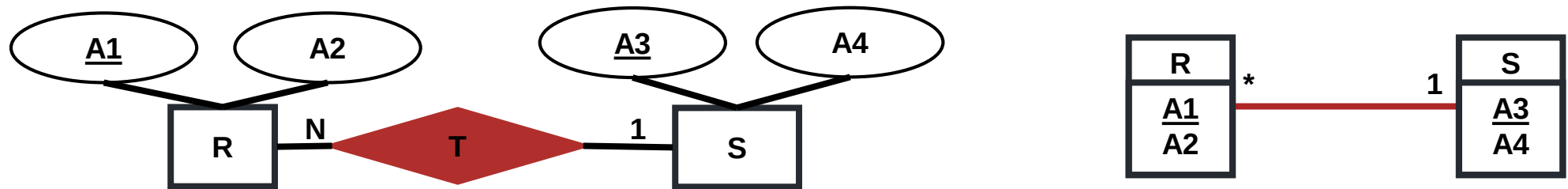
– SET NULL:

ABBILDUNG ERM / UML → RM



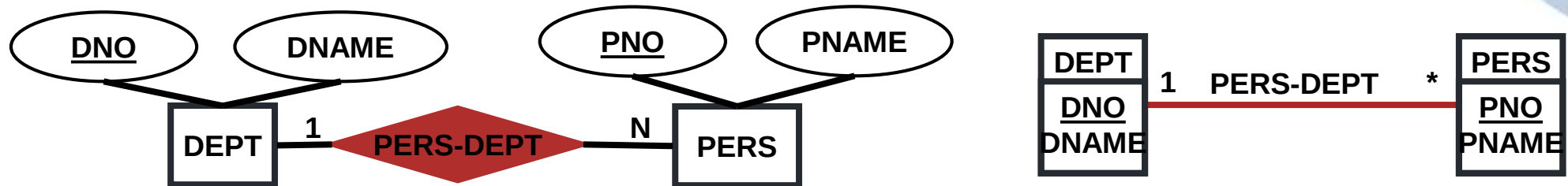
- Kriterien
 - Informationserhaltung
 - Minimierung der Redundanz
 - Minimierung des Verknüpfungsaufwandes
 - Natürlichkeit der Abbildung
 - keine Vermischung von Objekten
 - Verständlichkeit
- Regeln:
 - jede Entity-Menge muss als eigenständige Relation (Tabelle) mit einem eindeutigen Primärschlüssel definiert werden.
 - Relationship-Mengen können als eigene Relationen definiert werden, wobei die Primärschlüssel der zugehörigen Entity-Mengen als Fremdschlüssel zu verwenden sind.

2 ENTITY-MENGEN MIT N:1 - VERKNÜPFUNG



- **Regel:** n:1-Beziehungen lassen sich ohne eigene Relation darstellen.
 - in Relation R, der pro Tupel maximal 1 Tupel der anderen Relation S zugeordnet ist, wird Primärschlüssel von S als Fremdschlüssel verwendet
 - Fremdschlüssel in R `NOT NULL`, wenn genau 1 Tupel aus S ein Tupel aus R zugeordnet ist
 - Ausnahme: Relationship-Menge ist durch Attribute spezifiziert → 3 Relationen möglich
- $R(\underline{A1}, A2, \underline{A3} \text{ (FS auf S)}) \quad S(\underline{A3}, A4)$

2 ENTITY-MENGEN MIT N:1 - VERKNÜPFUNG



Ohne Attribute (Standard)

DEPT (DNO, DNAME)

PERS (PNO, PNAME, DNO (FS auf DEPT))

Mit Attribut Since für PERS-DEPT

DEPT (DNO, DNAME)

PERS (PNO, PNAME,)

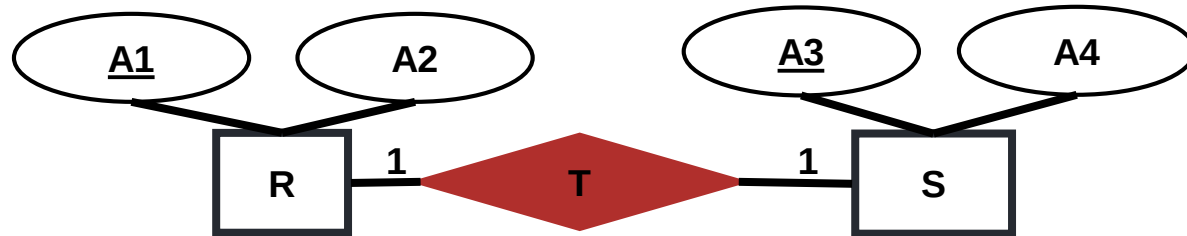
PERS-DEPT (PNO (FS auf PERS), DNO (FS auf DEPT), Since)

Obligatorisch - Multiplizität 1

DEPT (DNO, DNAME)

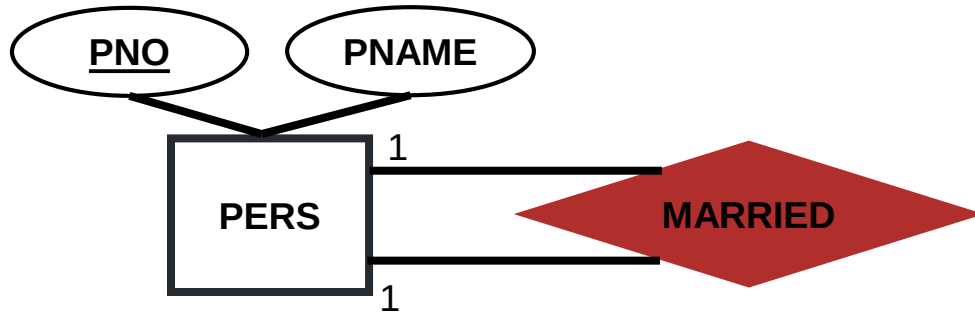
PERS (PNO, PNAME, DNO (FS auf DEPT, NOT NULL))

1 ENTITY-MENGE MIT 1:1 VERKNÜPFUNG



- **Regel:** 1:1-Beziehungen lassen sich ohne eigene Relation darstellen.
 - in Relation R, der pro Tupel maximal 1 Tupel der anderen Relation S zugeordnet ist, wird Primärschlüssel von S als Fremdschlüssel verwendet
 - Fremdschlüssel in R NOT NULL, wenn genau 1 Tupel aus S ein Tupel aus R zugeordnet ist
 - Fremdschlüssel in R muss eindeutig sein, da einem Tupel aus S nur maximal ein Tupel aus R zugeordnet sein kann
- $R(\underline{A1}, A2, A3(\text{FS auf } S, \text{ UNIQUE})) \quad S(\underline{A3}, A4)$

1 ENTITY-MENGE MIT 1:1 VERKNÜPFUNG



Allgemein

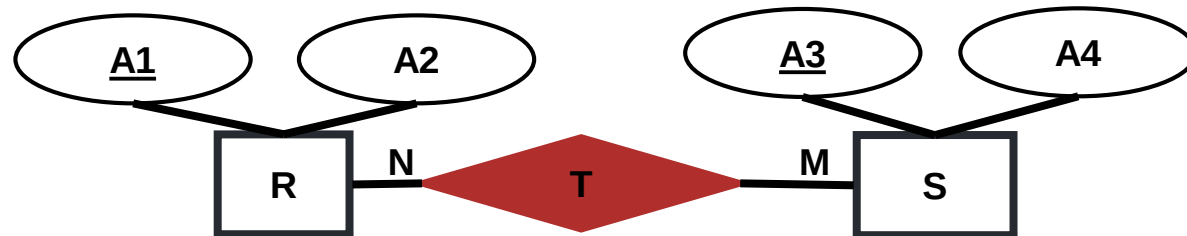
`PERS (PNO, PNAME, PARTNER (FS auf PERS, UNIQUE) )`

Obligatorisch - Multiplizität 1

`PERS (PNO, PNAME)`

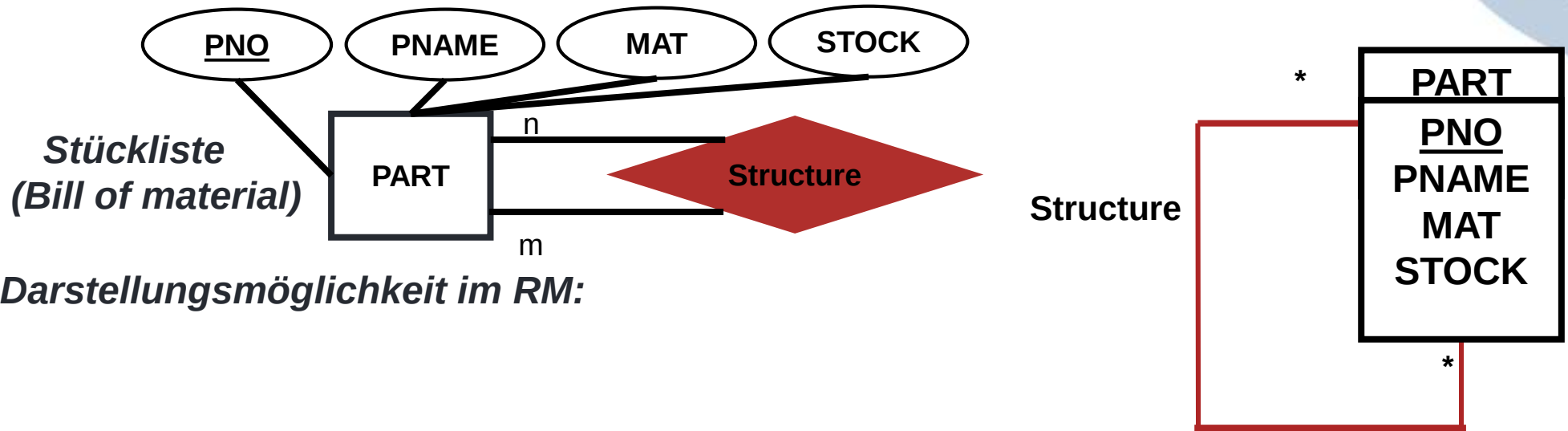
`DEPT (DNO, DNAME, MGR (FS auf PERS, UNIQUE, NOT NULL) )`

2 ENTITY-MENGEN MIT N:M - VERKNÜPFUNG



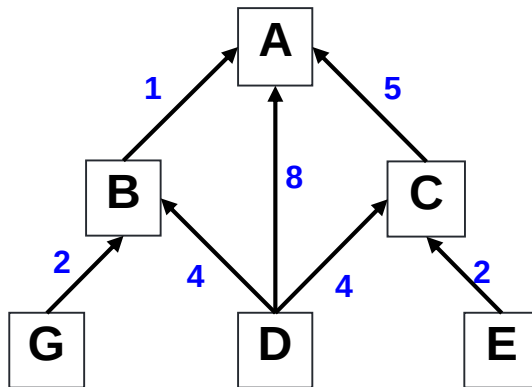
- **Regel:** n:m-Beziehungen benötigen eigenständige Relation
 - Primärschlüssel der dazugehörigen Entitätsmengen sind Fremdschlüssel in der eigenständigen Relation T und referenzieren die jeweiligen Relationen
 - Primärschlüssel der Relation T ist zusammengesetzt aus den Primärschlüssel der beteiligten Entitätsmengen
 - Eventuell Attribute der Relationship-Menge mit in den Primärschlüssel aufnehmen, falls Beziehung zwischen Entitäten mehrfach auftreten kann
 - In der Praxis generierte PS
- $R(\underline{A1}, A2) \quad S(\underline{A3}, A4)$
 $T(\underline{\underline{A1}} \text{ (FS auf R)}, \underline{\underline{A3}} \text{ (FS auf S)})$

1 ENTITY-MENGE MIT M:N-VERKNÜPFUNG



PART (PNO, PNAME, MAT, STOCK)

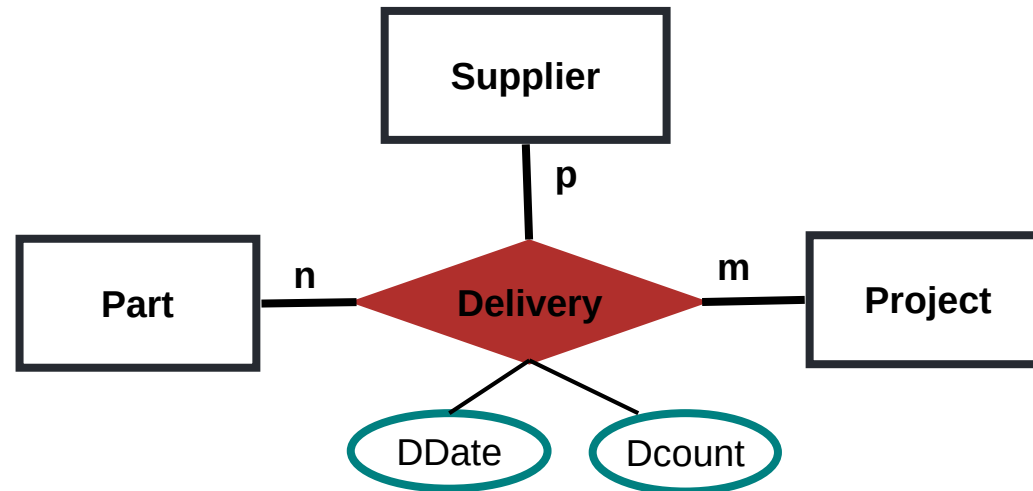
STRUCTURE (TOP(FS auf PART), COMP(FS auf PART), PCOUNT)



PNO	PNAME	MAT	STOCK
A	Getriebe	-	10
B	Gehäuse	Alu	0
C	Welle	Stahl	100
D	Schraube	Stahl	200
E	Kugellager	Stahl	50
F	Scheibe	Blei	0
G	Schraube	Chrom	100

TOP	COMP	PCOUNT
A	B	1
A	C	5
A	D	8
B	D	4
B	G	2
C	D	4
C	E	2

3 ENTITY-MENGEN MIT M:N-VERKNÜPFUNG



SUPPLIER(SNO, SNAME, SCITY, ...)

PROJECT(PRONO, PRONAME, PCITY, ...)

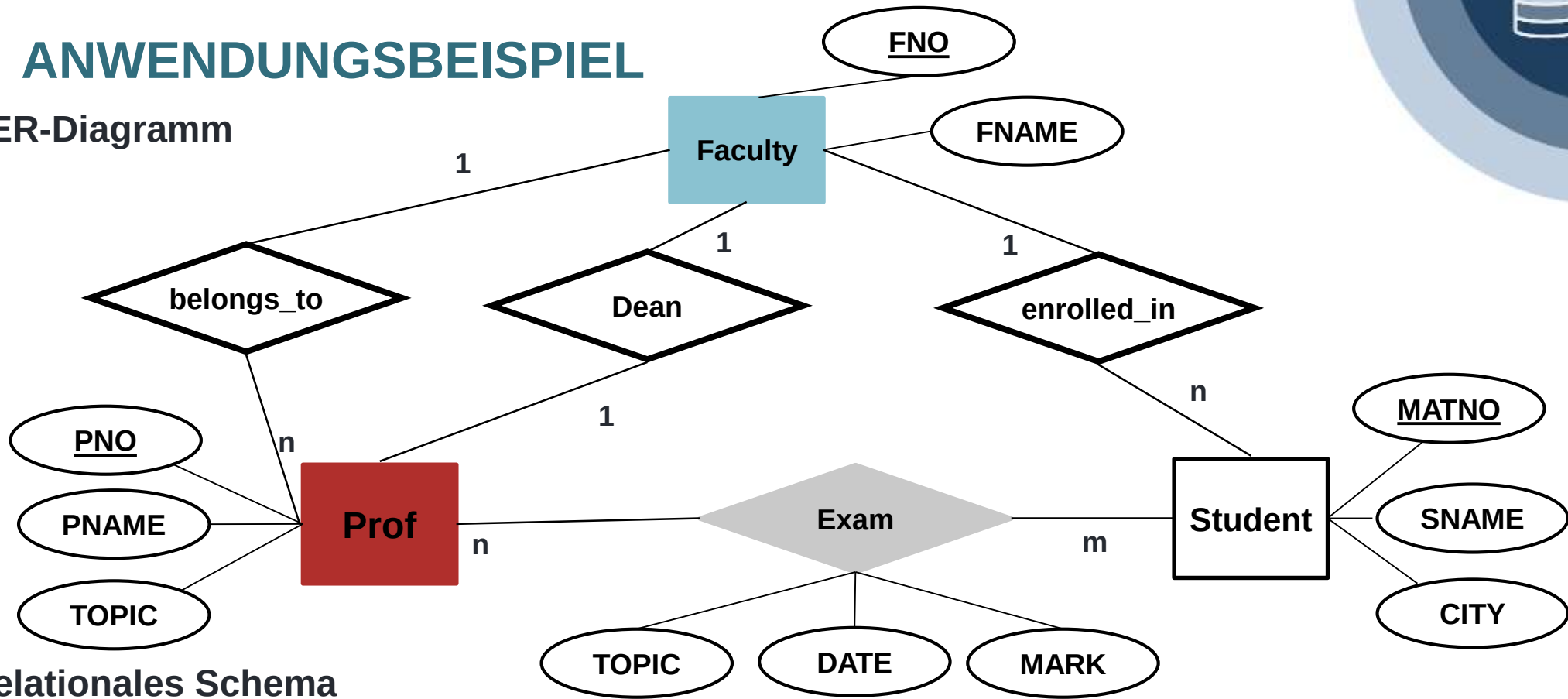
PART(PNO, PNAME, WEIGHT, ...)

DELIVERY(SNO(FS auf SUPPLIER),
PRONO(FS auf PROJEKT),
PNO(FS auf PART), DDate, Dcount)

- Einfügen eines künstlichen Primärschlüssels D-NO in Delivery möglich

ANWENDUNGSBEISPIEL

ER-Diagramm



relationales Schema

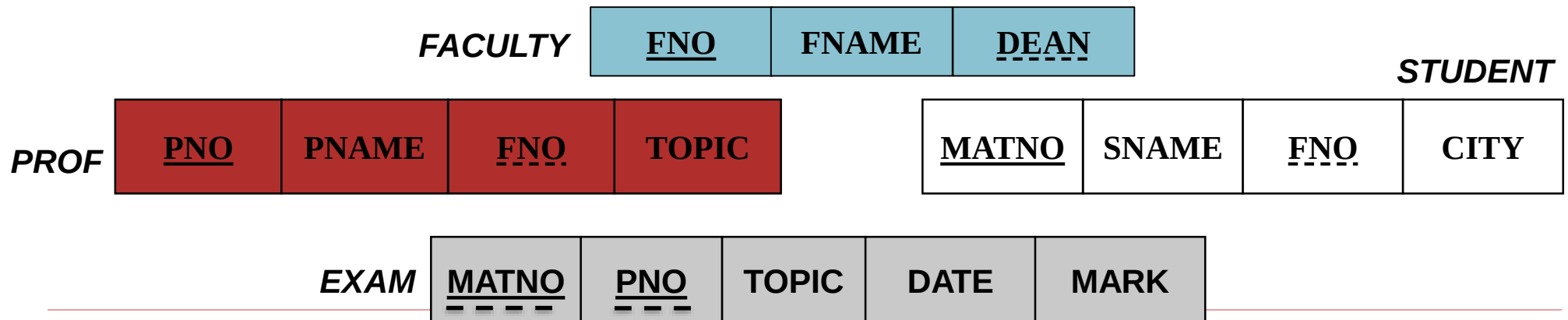
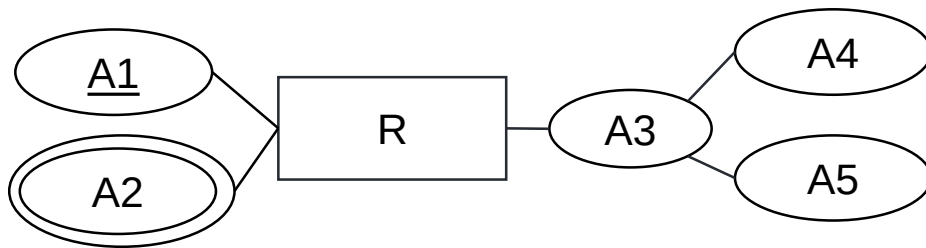


ABBILDUNG MEHRWERTIGER BZW. ZUSAMMENGESETZTER ATTRIBUTE



Darstellungsmöglichkeit im RM

$R(\underline{A1}, A4, A5)$

$R_A2(\underline{A1} \text{ (FS auf R)}, \underline{A2})$

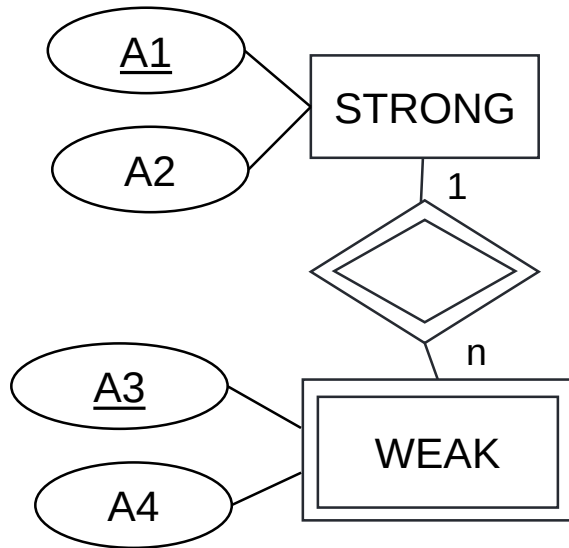
Mehrwertig:

- eigenständige 1:N Relation mit Primärschlüssel als Fremdschlüssel auf die Relation R
- PS zusammengesetzt aus Attributen des mehrwertigen Attributs und PS der Relation R

Zusammengesetzt

- Atomare Attribute des zusammengesetzten Attributs in Relation R übernehmen

ABBILDUNG SCHWACHER ENTITY-MENGEN



Darstellungsmöglichkeit im RM

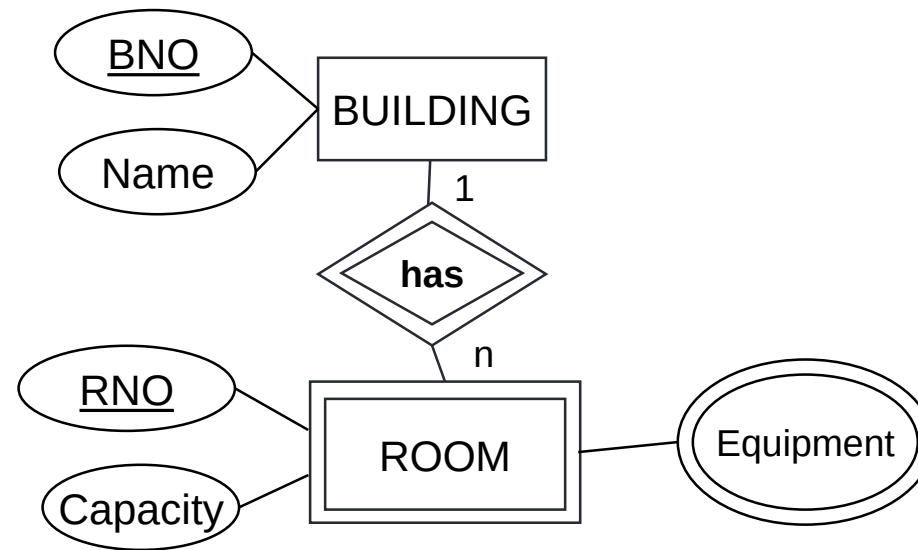
STRONG (A1, A2)

WEAK (A1 (FS auf STRONG ON {DELETE, UPDATE} CASCADE), A3, A4)

Regel

- wie n:1 Beziehung mit Lösch-und Updateregel CASCADE
- PS der WEAK Relation ist Kombination aus PS der STRONG Relation und partiellen PS der eigenen Relation

ABBILDUNG MEHRWERTIGER ATTRIBUTE BZW. SCHWACHER ENTITY-MENGEN - BEISPIEL



Darstellungsmöglichkeit im RM

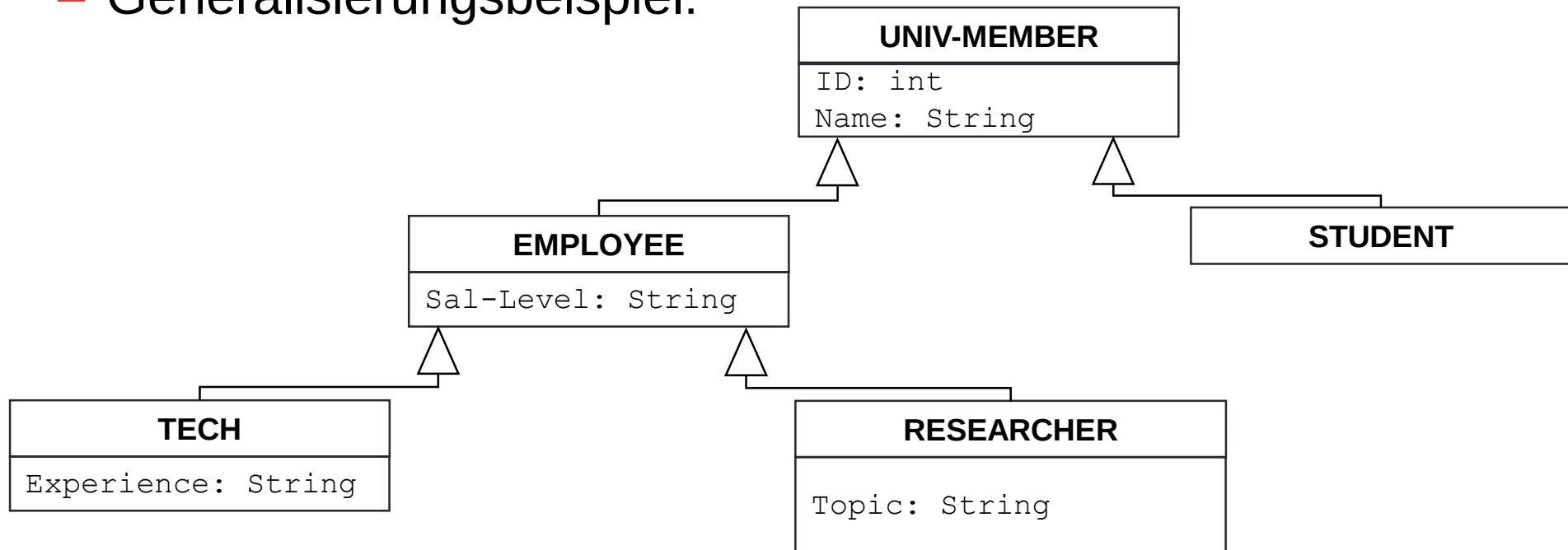
BUILDING (BNO, NAME)

ROOM(BNO(FS auf Building ON {DELETE, UPDATE} CASCADE), RNO,
Capacity)

EQUIPMENT_ROOM(BNO, RNO, EQUIPMENT) (BNO, RNO) FS auf ROOM

ABBILDUNGEN VON GENERALISIERUNG UND AGGREGATION IM RM

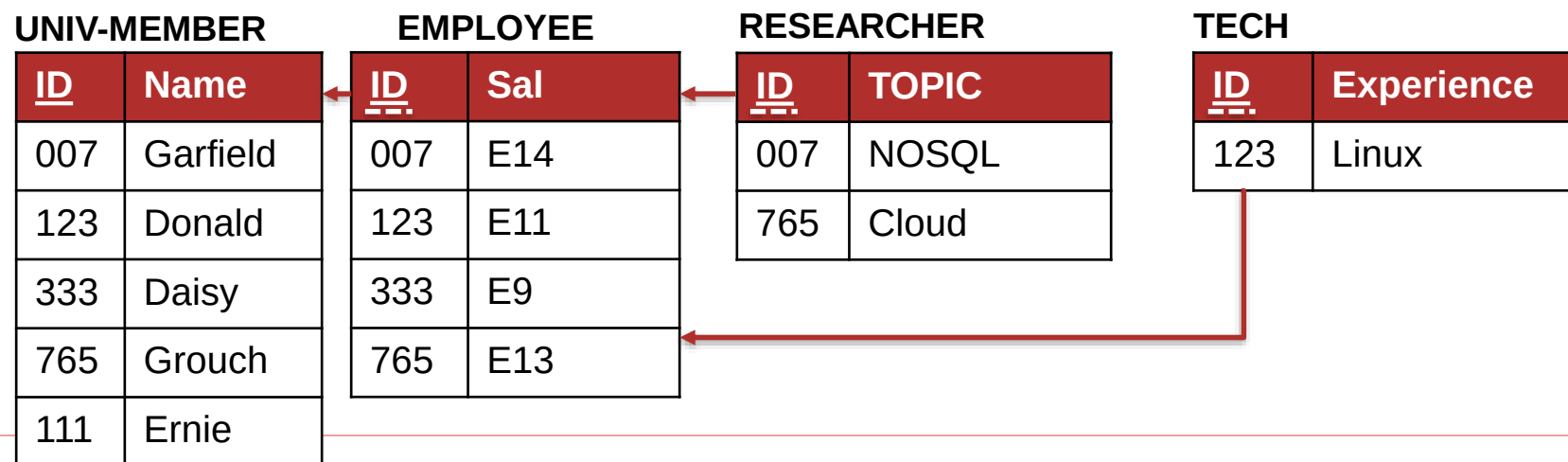
- RM sieht keine Unterstützung der Abstraktionskonzepte vor
 - keine Maßnahmen zur Vererbung (von Struktur, Integritätsbedingungen, Operationen)
 - „Simulation“ der Generalisierung und Aggregation eingeschränkt möglich
- Generalisierungsbeispiel:





GENERALISIERUNG – RELATIONALE SICHT

- pro Klasse 1 Tabelle (3 Varianten)
- Lösungsmöglichkeit 1: **vertikale Partitionierung**
 - jede Instanz wird entsprechend der Klassenattribute in der IS-A-Hierarchie zerlegt und in Teilen in den zugehörigen Klassen (Relationen) gespeichert.
 - nur das ID-Attribut wird dupliziert
 - Fremdschlüssel auf Super-Relation
- Eigenschaften
 - geringfügig erhöhte Speicherkosten, aber hohe Aufsuch- und Aktualisierungskosten
 - Integritätsbedingungen: $\text{TECH.ID} \subseteq \text{EMPLOYEE.ID}$, usw.
 - Instanzenzugriff erfordert Verbundoperationen, z.B. Bestimme alle TECH-Daten





GENERALISIERUNG – RELATIONALE SICHT (2)

– Lösungsmöglichkeit 2: **horizontale Partitionierung**

- jede Instanz ist genau einmal und vollständig in ihrer „Hausklasse“ gespeichert.
- keinerlei Redundanz

– Eigenschaften

- niedrige Speicherkosten und keine Änderungsanomalien
- Eindeutigkeit von ID zwischen Relationen aufwändiger zu überwachen
- Retrieval kann rekursives Suchen in Unterklassen erfordern.
- explizite Rekonstruktion durch Relationenoperationen (π , $\cup$ in Relationenalgebra) → Beispiel: Finde alle EMPLOYEE

UNIV-MEMBER	ID	Name
	111	Ernie

EMPLOYEE	ID	Name	Sal
	333	Daisy	E9

RESEARCHER	ID	Topic	Name	Sal
	007	NOSQL	Garfield	E14
	765	Cloud	Grouch	E13

TECH	ID	Experience	Name	Sal
	123	Linux	Donald	E11



GENERALISIERUNG – RELATIONALE SICHT (3)

- Lösungsmöglichkeit 3: **volle Redundanz**
 - eine Instanz wird wiederholt in jeder Klasse, zu der sie gehört, gespeichert.
 - sie besitzt dabei die Werte der Attribute, die sie geerbt hat, zusammen mit den Werten der Attribute der Klasse
- Eigenschaften
 - hoher Speicherplatzbedarf und Auftreten von Änderungsanomalien.
 - einfaches Retrieval, da nur Zielklasse (z. B. EMPLOYEE) aufzusuchen

UNIV-MEMBER

<u>ID</u>	Name
007	Garfield
123	Donald
333	Daisy
765	Grouch
111	Ernie

EMPLOYEE

<u>ID</u>	Name	Sal
007	Garfield	E14
123	Donald	E11
333	Daisy	E9
765	Grouch	E13

RESEARCHER

<u>ID</u>	Name	Sal	Topic
007	Garfield	E14	NOSQL
765	Grouch	E13	Cloud

TECH

<u>ID</u>	Name	Sal	Experience
123	Donald	E11	Linux

GENERALISIERUNG: VERFAHRENSVERGLEICH



	vertikale Partitionierung	horizontale Partitionierung	volle Redundanz
Änderungen			
Lesen			

AGGREGATION UND KOMPOSITION – RELATIONALE SICHT

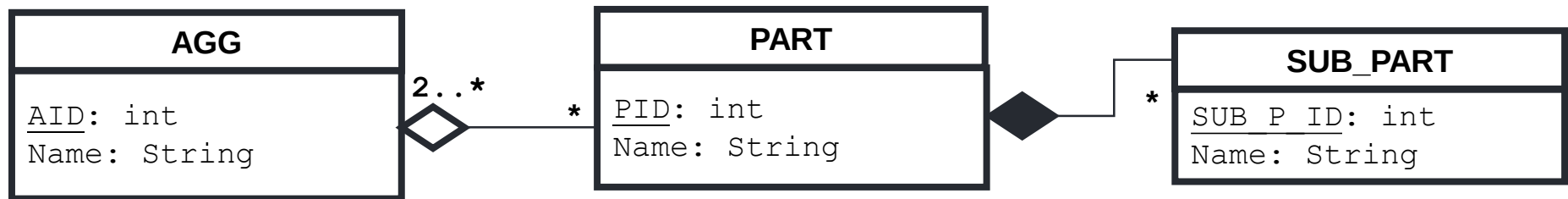


Aggregation

- Multiplizitäten ≥ 1 bei der Aggregat-Klasse erfordern n:m Relation ansonsten 1:N Modellierung

Komposition

- Wie 1:N Beziehung mit NOT NULL Bedingung für Fremdschlüssel



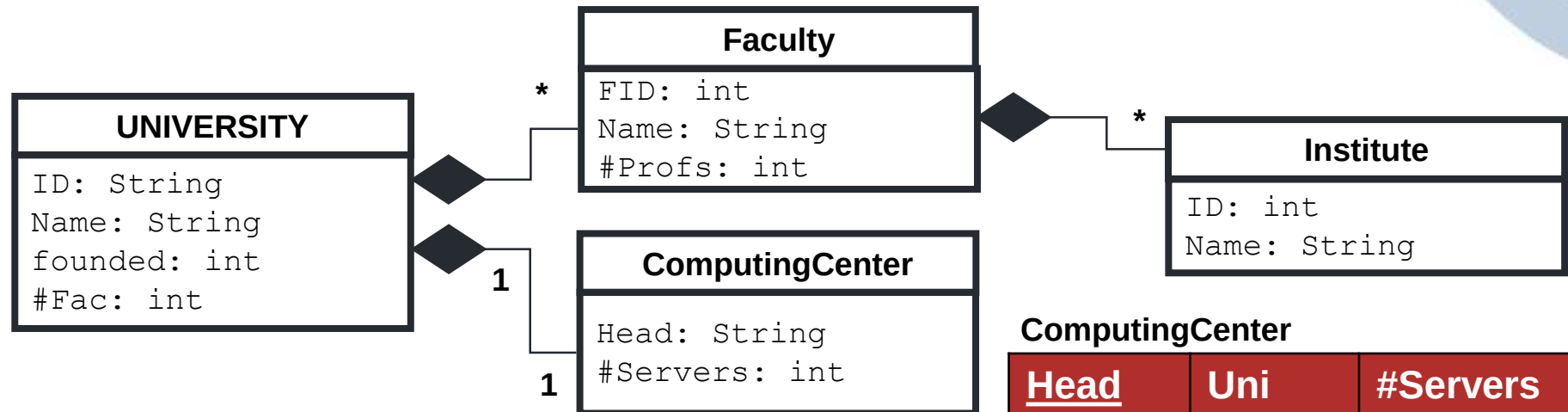
AGG (AID, Name)

PART (PID, Name)

AGG_PART (AID (FS auf AGG), PID (FS auf PART))

SUB_PART (SUB_P_PID, PID (FS auf PART, NOT NULL), Name)

AGGREGATION UND KOMPOSITION – RELATIONALE SICHT - BEISPIEL



University

<u>ID</u>	Name	founded	#Fac
UL	Leipzig Univ.	1409	14
TUD	TU Dresden	1828	14

Faculty

<u>FID</u>	Uni	Name	#Profs
123	UL	Theology	14
132	UL	Math/Comp. Science	28

ComputingCenter

<u>Head</u>	Uni	#Servers
Kühne	UL	120
Müller	TUD	130

Institute

<u>ID</u>	FID	Name
1234	123	New Testament Science
1235	123	Old Testament Science
1236	123	Practical Theology
1322	132	Computer Science



ZUSAMMENFASSUNG

- RM: einheitliche Datenrepräsentation durch normalisierte Relationen (Tabellen)
 - nur einfache (atomare) Attributwerte
 - eindeutiger Primärschlüssel pro Relation
 - Realisierung von Beziehungen über Fremdschlüssel
- Wartung der referentiellen Integrität
 - automatisch durch DBMS auf Basis von Nutzerspezifikation für Löschungen / PK-Updates referenzierter Sätze
- Abbildung ERM ins Relationenmodell
 - eigene Tabellen für n:m-Beziehungen sowie mehrwertige Attribute
- Nachbildung von is-a-Beziehungen und Aggregation
 - nur teilweise mit Fremdschlüsseln realisierbar (keine Vererbungssemantik)
 - mehrere Varianten für Generalisierung mit eigener Relation pro (Sub-)Klasse sowie Partitionierung bzw. Replikation von Attributen



4. RELATIONENALGEBRA

- Einleitung
- Selektion, Projektion, Umbenennung
- Mengenoperatoren
 - Vereinigung, Durchschnitt, Differenz
 - kartesisches Produkt
- Verbundoperationen (Join)
 - Theta-Join
 - natürlicher Verbund
 - Semi-Join
 - äußerer Verbund
- Division
- Beispielanfragen



SPRACHEN FÜR DAS RELATIONENMODELL

- Datenmodell = Datenobjekte + Operatoren
- im RM wird vereinheitlichte Sprache angestrebt für:
 - Anfragen (Queries) im 'Stand-Alone'-Modus
 - Datenmanipulation und Anfragen eingebettet in eine Wirtssprache
 - Datendefinition
 - Zugriffs- und Integritätskontrolle
 - Unterstützung verschiedener Benutzerklassen:
Anwendungsprogrammierer, DBA, gelegentliche Benutzer
- verschiedene Grundtypen von Sprachen
 - formale Ansätze: Relationenalgebra und Relationenkalkül
 - abbildungsorientierte Sprachen (z. B. SQL)
 - graphik-orientierte Sprachen (z. B. Query-by-Example)

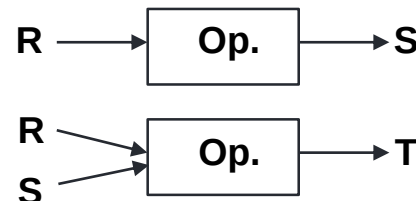


RELATIONENALGEBRA

- Algebra: ein System, das aus einer nichtleeren Menge und einer Familie von Operationen besteht
 - Relationen sind Mengen
 - Operationen auf Relationen arbeiten auf einer oder mehreren Relationen als Eingabe und erzeugen eine Relation als Ausgabe (Abgeschlossenheitseigenschaft) $\Rightarrow$ mengenorientierte Operationen
- Operationen

Klassische Mengenoperationen	Relationenoperationen
• Vereinigung • Differenz • Kartesisches Produkt • Durchschnitt	• Restriktion (Selektion) • Projektion • Umbenennung • Verbund (Join) (ableitbar) • Division (ableitbar)

- 1-stellige und 2-stellige Operationen





SELEKTION (RESTRIKTION)

- Auswahl von Zeilen einer Relation über Prädikate, abgekürzt σ_p

$$\sigma_p(R) = \{ t \mid t \in R \wedge P(t) \}$$

- P = log. Formel (ohne Quantoren !) zusammengestellt aus:
 - Operanden: Attributnamen oder Konstanten
 - Vergleichsoperatoren $q \in \{<, =, >, \leq, \neq, \geq\}$
 - logische Operatoren: $\vee, \wedge, \neg$
- Beispiele:
 - $\sigma_{\text{SALARY} < \text{BONUS}} (\text{PERS})$
 - $\sigma_{\text{OCCUPATION} = \text{'Programmer'} \wedge \text{AGE} < 50} (\text{PERS})$
- Eigenschaften
 - $\text{grad}(\sigma_p(R)) = \text{grad}(R)$
 - $\text{card}(\sigma_p(R)) \leq \text{card}(R)$



PROJEKTION

- Auswahl der Spalten (Attribute) $A_1, A_2, \dots, A_k$ aus einer Relation R (Grad $n \geq k$)

$$\pi_{A_1, A_2, \dots, A_k}(R) = \{ p \mid \exists t \in R : p = \langle t[A_1], \dots, t[A_k] \rangle \}$$

- Beispiel: $\pi_{\text{Name, Salary}}(\text{PERS})$
- Eigenschaften:
 - wichtig: Duplikate werden entfernt ! (Mengeneigenschaft)
z.B. für $\pi_{\text{DNO}}(\text{PERS})$
 - $\text{grad}(\pi_A(R)) \leq \text{grad}(R)$
 - $\text{card}(\pi_A(R)) \leq \text{card}(R)$



UMBENNUNG

- Umbenennung von Attributnamen und Relationsnamen einer Relation $R(B_1, \dots, B_n)$
 - Wichtig zum Auflösen von Mehrdeutigkeiten bei reflexiven Verbänden für eindeutige Adressierung des Attributs

$\rho_{S(A_1, \dots, A_n)}(R) \Rightarrow$ Umbenennung der Attribute und des Namens von R zu $S(A_1, \dots, A_n)$

$\rho_S(R) \Rightarrow$ Umbenennung der Relation R zu S

- Beispiel: $\rho_{MA(Nachname, Gehalt)}(\pi_{Name, Salary}(PERS))$



RELATIONENALGEBRA: BEISPIEL-DB

DEPT

<u>DNO</u>	DNAME	CITY
K51	Planung	Leipzig
K53	Einkauf	Frankfurt
K55	Vertrieb	Frankfurt

PERS

<u>PNO</u>	NAME	AGE	SALARY	<u>DNO(FS_auf_DEPT)</u>	<u>MGR(FS_auf_PERS)</u>
406	Abel	47	50700	K55	123
123	Schulz	32	43500	K51	-
829	Müller	36	40200	K53	406
574	Schmid	28	36000	K55	123

- Finde alle Angestellten aus Abteilung K55, die mehr als 40.000 verdienen
- $\sigma_{DNO='K55' \wedge SALARY > 40000} (PERS)$
- Finde alle Abteilungsorte
- $\pi_{CITY} (DEPT)$
- Finde den Abteilungsnamen von Abteilung K53
- $\pi_{DNAME} (\sigma_{DNO='K53'} (DEPT))$

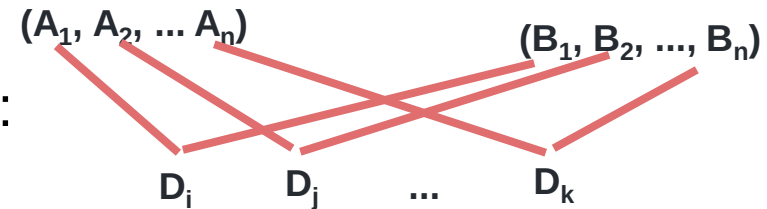
CITY
Leipzig
Frankfurt

KLASSISCHE MENGENOPERATIONEN

- Voraussetzung: **Vereinigungsverträglichkeit** der beteiligten Relationen:

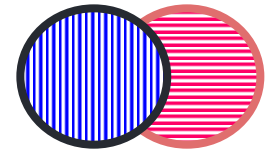
- gleicher Grad und gleiche Bereiche:

➤ $W(A_i) = W(B_i) \quad : \quad i = 1, n$



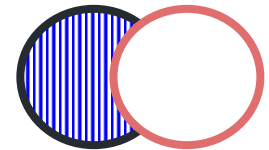
- Vereinigung: $R \cup S = \{t \mid t \in R \vee t \in S\}$

- $\text{card}(R \cup S) \leq \text{card}(R) + \text{card}(S)$



- Differenz: $R - S = \{t \mid t \in R \wedge t \notin S\}$

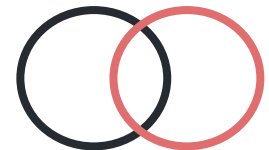
- $\text{card}(R - S) \leq \text{card}(R)$



- Durchschnitt:

$$R \cap S = R - (R - S) = \{t \mid t \in R \wedge t \in S\}$$

- $\text{card}(R \cap S) \leq \min(\text{card}(R), \text{card}(S))$



- Beispielanfrage: Welche Abteilungen (DNO) haben keine Mitarbeiter?



(ERWEITERTES) KARTESISCHES PRODUKT

- $R(A_1, \dots, A_r)$ (Grad r) und $S(B_1, \dots, B_s)$ (Grad s) beliebig

Relationensschema: $(R \times S)(A_1, \dots, A_r, B_1, \dots, B_s)$

Relation: $R \times S = \{k = x \circ y \mid x \in R \wedge y \in S\}$

- Beachte: $k = x \circ y = (x_1, \dots, x_r, y_1, \dots, y_s)$
nicht $((x_1, \dots, x_r), (y_1, \dots, y_s))$ wie übliches kart. Produkt
- $\text{grad}(R \times S) = \text{grad}(R) + \text{grad}(S)$
- $\text{card}(R \times S) = \text{card}(R) \cdot \text{card}(S)$

Beispiel

R		
A	B	C
a	g	1
d	a	2
b	b	3

S		
D	E	F
b	g	3
d	a	2

R × S					
A	B	C	D	E	F
a	g	1	b	g	3
a	g	1	d	a	2
d	a	2	b	g	3
d	a	2	d	a	2
b	b	3	b	g	3
b	b	3	d	a	2



VERBÜNDE

- Operatoren für die Verknüpfung von Tupeln $r \in R$ und $s \in S$ unter Berücksichtigung eines Verbundprädikats P
- Allgemeiner Verbund – Theta Join
 - Spezialfall Gleichverbund $\rightarrow$ Equi-Join
- Natürlicher Verbund – Natural Join
 - Equi Join über gleichnamige Attribute
- [linker, rechter] Semi-Join
 - Erhaltung der Attribute einer Relation
- Verlustfreie Verbünde
 - Erhalt der Tupel auch ohne Verbundpartner
 - [linker, rechter] Äußerer Verbund



ALLGEMEINER VERBUND (THETA-JOIN)

- grob: kartesisches Produkt zwischen zwei Relationen R und S.
 - eingeschränkt durch Θ -Bedingungen o.B.d.A. zwischen Attribut A_k von R und Attribut B_l von S $\rightarrow$ verallgemeinerbar für mehrere Attribute

Relationenschema: $(R \bowtie_{A_k \Theta B_l} S) (A_1, \dots, A_r, B_1, \dots, B_s)$

Relation: $(R \bowtie_{A_k \Theta B_l} S) = \{k = x \circ y \mid x \in R \wedge y \in S \wedge R: x[A_k] \Theta y[B_l]\}$

- Θ -Verbund zwischen R und S ableitbar:

$$R \bowtie_{A \Theta B} S = \sigma_{A \Theta B}(R \times S)$$

mit arithm. Vergleichsoperator $\Theta \in \{<, =, >, \leq, \neq, \geq\}$

- $\text{grad}_{A \Theta B}(R \bowtie S) = \text{grad}(R \times S) = \text{grad}(R) + \text{grad}(S)$
- $\text{card}_{A \Theta B}(R \bowtie S) \leq \text{card}(R \times S)$
- für häufigen Fall des **Gleichverbunds (Equi-Join)** gilt $\Theta = '=' :$



NATÜRLICHER VERBUND (NATURAL JOIN)

- grob: Gleichverbund über alle gleichnamigen Attribute und Projektion über die verschiedenen Attribute

- **natürlicher Verbund** zwischen R und S:

gegeben: $R(A_1, A_2, \dots, A_{r-j+1}, \dots, A_r), \quad S(B_1, B_2, \dots, B_j, \dots, B_s)$

o.B.d.A. (sonst. Umsortierung): $B_1 = A_{r-j+1}, B_2 = A_{r-j+2} \dots B_j = A_r$

$$R \bowtie S = \pi(A_1, A_2, \dots, A_r, B_{j+1}, \dots, B_s) \sigma_{A_{r-j+1}=B_1 \wedge \dots \wedge A_r=B_j} (R \times S)$$

$\bowtie$ Zeichen für **Natural Join** $\Rightarrow \textcircled{H} = '='$

- Join-Attribute sind durch Übereinstimmungsbedingung gegeben
- $\text{grad}(R \bowtie S) = \text{grad}(R) + \text{grad}(S) - j$
 - mit j Anzahl der gemeinsamen Attribute

R		
A	B	C
a ₁	b ₁	c ₁
a ₂	b ₂	c ₂



S		
C	D	E
c ₁	d ₁	e ₁
c ₃	d ₂	e ₂

=

Resultat				
A	B	C	D	E
a ₁	b ₁	c ₁	d ₁	e ₁

JOIN-BEISPIEL

DEPT

<u>DNO</u>	DNAME	CITY
K51	Planung	Leipzig
K53	Einkauf	Frankfurt
K55	Vertrieb	Frankfurt

PERS

<u>PNO</u>	NAME	AGE	SALARY	DNO(FS auf DEPT)	MGR(FS auf PERS)
406	Abel	47	50700	K55	123
123	Schulz	32	43500	K51	-
829	Müller	36	40200	K53	406
574	Schmid	28	36000	K55	123

- Finde alle Angestellten (PNO, AGE, DNAME), die in einer Abteilung in Frankfurt arbeiten und älter als 30 sind
- $\pi_{PNO, AGE, DNAME}(\sigma_{City='Frankfurt' \wedge AGE > 30}(DEPT \bowtie PERS))$

PNO	AGE	DNAME
406	47	Vertrieb
829	36	Einkauf



SEMI-JOIN

- Ergebnisbeschränkung des Gleichverbundes auf eine der beiden Eingaberelationen R und S

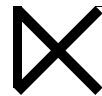
$$R \bowtie S = \pi_{R-Attribute}(R \bowtie S)$$

linker Semi-Join

$$R \bowtie S = \pi_{S-Attribute}(R \bowtie S)$$

rechter Semi-Join

R		
A	B	C
a ₁	b ₁	c ₁
a ₂	b ₂	c ₂

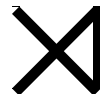


S		
C	D	E
c ₁	d ₁	e ₁
c ₃	d ₂	e ₂

=

Resultat		
A	B	C
a ₁	b ₁	c ₁

R		
A	B	C
a ₁	b ₁	c ₁
a ₂	b ₂	c ₂



S		
C	D	E
c ₁	d ₁	e ₁
c ₃	d ₂	e ₂

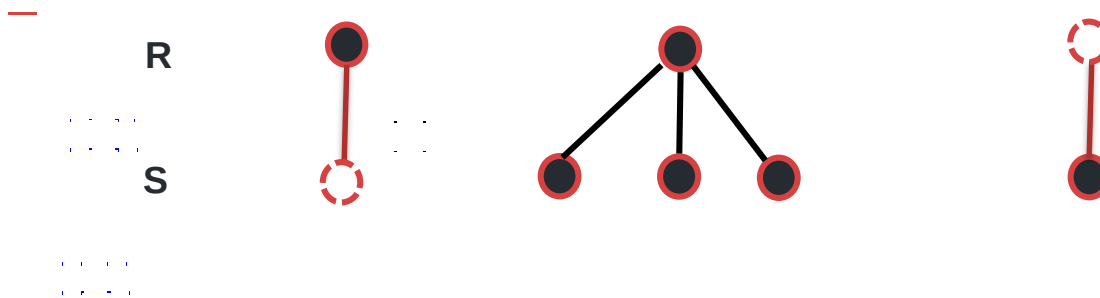
=

Resultat		
C	D	E
c ₁	d ₁	e ₁



ÄUßERER VERBUND (OUTER JOIN)

- Ziel: verlustfreier Verbund soll erzwungen werden
- Gleichverbund zwischen R und S ist verlustfrei, wenn alle Tupel von R und S am Verbund teilnehmen. Die inverse Operation Projektion erzeugt dann wieder R und S (lossless join).
- $R \bowtie S$ verlustfrei $\Leftrightarrow \pi_{R-\text{Attribute}} R \bowtie S = R \wedge \pi_{S-\text{Attribute}} R \bowtie S = S$
- bisher: R S liefert nur „vollständige Objekte“
 - es sollen aber auch Teilobjekte als Ergebnis geliefert werden (z. B. komplexe Objekte)
 - Trick: Einfügen künstlicher Verbundpartner, um verlustfreien Verbund zu erreichen





OUTER JOIN (2)

- Definition: seien A die Verbundattribute, $\{\equiv\}$ der undefinierte Wert

$$R' := R \cup ((\pi_A(S) - \pi_A(R)) \times \{ \underbrace{(\equiv, \dots, \equiv)}_{\text{grad}(R) - \text{grad}(\pi_A(R))} \})$$

$$S' := S \cup ((\pi_A(R) - \pi_A(S)) \times \{ \underbrace{(\equiv, \dots, \equiv)}_{\text{grad}(S) - \text{grad}(\pi_A(S))} \})$$

äußerer Gleichverbund

$$R \bowtie_{R.A=S.A} S := R' \bowtie_{R'.A=S'.A} S'$$

äußerer natürlicher Gleichverbund

$$R \bowtie S := R' \bowtie S'$$

- linker und rechter äußerer Gleichverbund

- nur die linke bzw. rechte Eingaberelation bleibt verlustfrei (Einfügen künstlicher Verbundpartner in rechter bzw. linker Eingaberelation)

linker äußerer Gleichverbund

$$R \bowtie S := R \bowtie S'$$

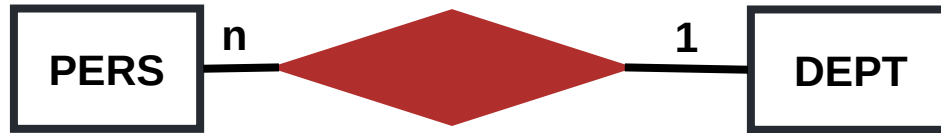
rechter äußerer Gleichverbund

$$R \bowtie S := R' \bowtie_{R'.A=S.B} S$$

verallgemeinerbar auf 2 (oder mehr) Joins, z.B. $R \bowtie S \bowtie T$

- selbst isolierte Tupel können zu einem vollständigen Pfad expandiert werden

OUTER JOIN - BEISPIEL



PERS

PNO	DNO ...
P1	D1
P2	D1
P3	D2
P4	-
P5	-

DEPT

DNO	DNAME ...
D1	A
D2	B
D3	C

PERS ⋈ DEPT

PNO	DNO	DNAME ...
P1	D1	A
P2	D1	A
P3	D2	B

PNO	DNO	DNAME ...
P1	D1	A
P2	D1	A
P3	D2	B

PERS ⋈ DEPT



PERS ⋈ DEPT

PNO	DNO	DNAME ...
P1	D1	A
P2	D1	A
P3	D2	B

PNO	DNO	DNAME ...
P1	D1	A
P2	D1	A
P3	D2	B

PERS ⋈ DEPT



DIVISION

- Beantwortung von Fragen, bei denen eine „ganze Relation“ zur Qualifikation herangezogen wird
- Simulation des Allquantors $\Rightarrow$ eine Attributwert-Kombination aus R steht mit allen Tupeln aus S in einer bestimmten Beziehung

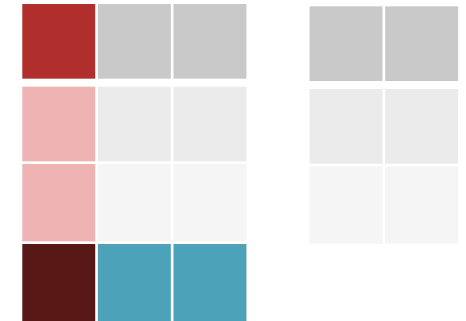
- **Definition**

Voraussetzung: S-Attribute $\subset$ R-Attribute

sei R vom Grad r und S vom Grad s, $r > s$

t sei (r-s)-Tupel, u sei s-Tupel;

dann gilt: $R \div S = \{ t \mid \forall u \in S : t u \in R \}$



$$\text{grad}(R \div S) = r - s$$

$$\text{card}(R \div S) \leq \text{card}(R)$$

DIVISION (2)

– Beispiel

iel

SUPPLY		
SNR	PRO	PART
L1	P1	T1
L1	P2	T1
L2	P1	T1
L2	P1	T2
L2	P2	T1

÷

PP	
PRO	PART
P1	T1
P1	T2
P2	T1

=

SNR
L2

– welche Lieferanten beliefern alle Projekte?

$$\pi_{\text{SNR,PRO}} (\text{SUPPLY}) \div \pi_{\text{PRO}} (\text{PP})$$

– welche Lieferanten liefern alle Teile?

$$\pi_{\text{SNR,PART}} (\text{SUPPLY}) \div \pi_{\text{PART}} (\text{PP})$$

– Zusammenhang zwischen Division und kartesischem Produkt:

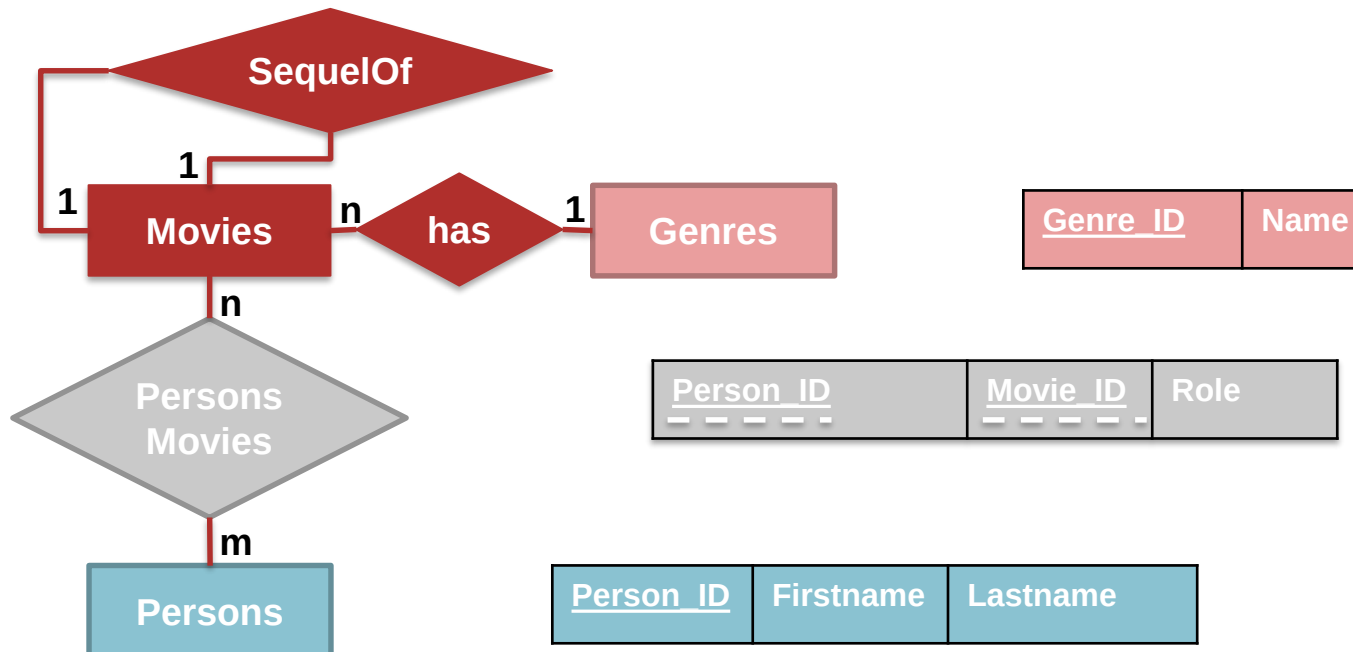
$$(R \times S) \div S = R$$

$$(R \div S) \times S = R ?$$

BEISPIEL-DB: DBS1 MOVIEDB



<u>Movie_ID</u>	Title	Release Date	<u>Genre_ID</u>	Budget	Opening Week	Profit	Runtime	Certificate	<u>Sequel_Of</u>	Distribution
-----------------	-------	--------------	-----------------	--------	--------------	--------	---------	-------------	------------------	--------------



https://dbis-uibk.github.io/relax/calc/gist/d37f667154aec34f5c4954723ae01db9/DBS1_MovieDB/0



BEISPIELANFRAGEN

- Welche Filme haben Überlänge (Runtime > 120)?

$\pi_{\text{Title}} (\sigma_{\text{Runtime} > 120} (\text{Movies}))$

- Welche Personen (Firstname, Lastname) waren an Filmen des Genres ‚Fantasy‘ beteiligt?

$\pi_{\text{Firstname, Lastname}} \sigma_{\text{Genres.Name} = \text{'Fantasy'}} (\text{Genres} \bowtie \text{Movies} \bowtie \text{PersonsMovies} \bowtie \text{Persons})$

- Finde alle Filme (Title), wo mindestens 2 Personen die Regie (Role = “direction“) geführt haben.

$\rho_{P2} (\sigma_{\text{Role} = \text{'direction'}} (\text{PersonsMovies}))$
 $\bowtie_{P2.Movie_ID = P1.Movie_ID \wedge P1.Person_ID \neq P2.Person_ID}$
 $\rho_{P1} (\sigma_{\text{Role} = \text{'direction'}} (\text{PersonsMovies}))$

- Finde die Genres (Name), zu denen kein Film existiert

$\pi_{\text{Genre_ID}} (\text{Genres}) -$

- Welche Personen (Person_ID) waren an allen Filmen des Genres ‚Drama‘ beteiligt?



5. ANFRAGESPRACHE SQL

- Grundlagen: Anfragekonzept, Befehlsübersicht
- SELECT: mengenorientierte Anfragen deskriptiver Art
 - Selektions- und Projektionsanfragen, Join-Anfragen
 - geschachtelte Anfragen (Sub-Queries)
 - Aggregatfunktionen
 - Gruppenanfragen (GROUP BY, HAVING)
 - Prädikate LIKE, BETWEEN, IN, IS NULL
 - quantifizierte Prädikate (ALL/ANY, EXISTS)
 - Mengen-Operationen: UNION, INTERSECT, EXCEPT
- Änderungsoperationen INSERT, DELETE, UPDATE, MERGE
- Vergleich mit der Relationenalgebra



ENTWICKLUNG VON SQL

- unterschiedliche Entwürfe für relationale Anfragesprachen
 - SEQUEL: Structured English Query Language (System R) -> SQL
 - QUEL (Ingres), . . .
- SQL: vereinheitlichte Sprache für alle DB-Aufgaben
 - einfache Anfragemöglichkeiten für gelegentliche Benutzer
 - mächtige Sprachkonstrukte für besser ausgebildete Benutzer
 - spezielle Sprachkonstrukte für DBA
- Standardisierung von SQL durch ANSI und ISO
 - erster ISO-Standard 1987
 - verschiedene Addenda (1989)
 - 1992: „SQL2“ bzw. SQL-92 (Entry, Intermediate, Full Level)
 - 1999 und später: SQL:1999 („SQL3“), SQL:2003, SQL:2008, SQL:2011, SQL:2016 u.a. mit objektorientierten Erweiterungen (objekt-relationale DBS), XML-Unterstützung, temporalen Anfragen, etc.



ABBILDUNGSORIENTIERTE ANFRAGEN IN SQL

- SQL: strukturierte Sprache, die auf englischen Schlüsselwörtern basiert
 - Zielgruppe umfasst auch Nicht-Programmierer
 - *relational vollständig*: Auswahlvermögen umfasst das der Relationenalgebra
- Grundbaustein

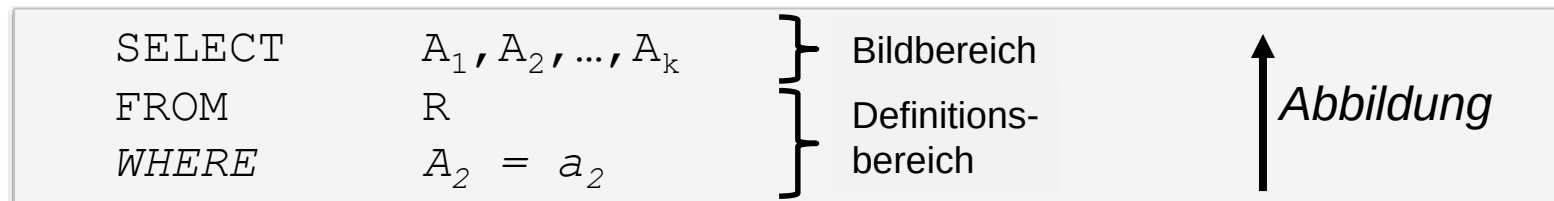


Abbildung: Eingaberelationen (**FROM**) werden unter Auswertung von Bedingungen (**WHERE**) in Attribute einer Ergebnistabelle (**SELECT**) abgebildet

- Allgemeines Format
 - <Spezifikation der Operation>
 - <Liste der referenzierten Tabellen>
 - [WHERE Boolescher Prädikatsausdruck]

ERWEITERUNGEN ZU EINER VOLLSTÄNDIGEN DB-SPRACHE



– Datenmanipulation

- Einfügen, Löschen und Ändern von individuellen Tupeln und von Mengen von Tupeln
- Zuweisung von ganzen Relationen

– Datendefinition

- Definition von Wertebereichen, Attributen und Relationen
- Definition von verschiedenen Sichten auf Relationen

– Datenkontrolle

- Spezifikation von Bedingungen zur Zugriffskontrolle
- Spezifikation von Zusicherungen (assertions) zur semantischen Integritätskontrolle

– Kopplung mit einer Wirtssprache

- deskriptive Auswahl von Mengen von Tupeln
- sukzessive Bereitstellung einzelner Tupeln

	Retrieval	Manipulation	Datendefinition	Datenkontrolle
Stand-Alone DB-Sprache	SQL RA	SQL	SQL	SQL
Eingebettete DB-Sprache	SQL	SQL	SQL	SQL

BEFEHLSÜBERSICHT (AUSWAHL)

Datenmanipulation (DML):

SELECT
INSERT
UPDATE
DELETE

MERGE

Aggregatfunktionen: COUNT, SUM, AVG, MAX, MIN

Datenkontrolle:

Constraint-Definitionen bei CREATE TABLE

CREATE TRIGGER
DROP TRIGGER
GRANT
REVOKE
COMMIT
ROLLBACK

Datendefinition (DDL):

CREATE SCHEMA
CREATE DOMAIN
CREATE TABLE
CREATE VIEW
ALTER TABLE
DROP SCHEMA
DROP DOMAIN
DROP TABLE
DROP VIEW

Eingebettetes SQL:

DECLARE CURSOR
FETCH
OPEN CURSOR
CLOSE CURSOR
SET CONSTRAINTS
SET TRANSACTION
CREATE TEMPORARY TABLE

SQL-TRAINING IN LOTS (<https://lots.uni-leipzig.de>)

- „freies Üben“ auf einer SQL-Datenbank (SELECT-Anweisungen)
 - Realisierung auf Basis von Postgres
- „aktives“ SQL-Tutorial



Leipzig Online-Test-System

UNIVERSITÄT LEIPZIG
Fakultät für Mathematik und Informatik
Institut für Informatik
Abteilung Datenbanken

- Logout
- News
- Training
- SQL-Training**
- XQuery
- Mein Profil
- Impressum

Tutorial

- 1 Einleitung
- 2 Datenbankmodellierung und Relationenmodell
- 3 SQL
- 4 Einfache SQL-Anfragen
 - 4.1 Vorbemerkungen
 - 4.2 Einfache Selektion und Projektion**
 - 4.3 Ausgabebearbeitung
- 5 Verbund-Anfragen
- 6 Unterabfragen
- 7 Aggregatfunktionen
- 8 Partitionierung in Gruppen und Auswahl
- 9 Suchbedingungen
- 10 Mengentheoretische Operationen
- 11 Datendefinition
- 12 Datenmanipulation auf

[Zurück](#) [Weiter](#) [Hoch](#) | [zurück zum SQL-Anfrageformular](#)

4.2 Einfache Selektion und Projektion

Die Projektion aus der [Relationenalgebra](#) stellt sich dar als Auflistung der Attribute nach dem Wort `SELECT`. Folgende SQL-Anfrage repräsentiert eine sehr einfache Projektion.

Beispiel:

BNF: [select-ausdruck](#) diese Anfrage ausführen

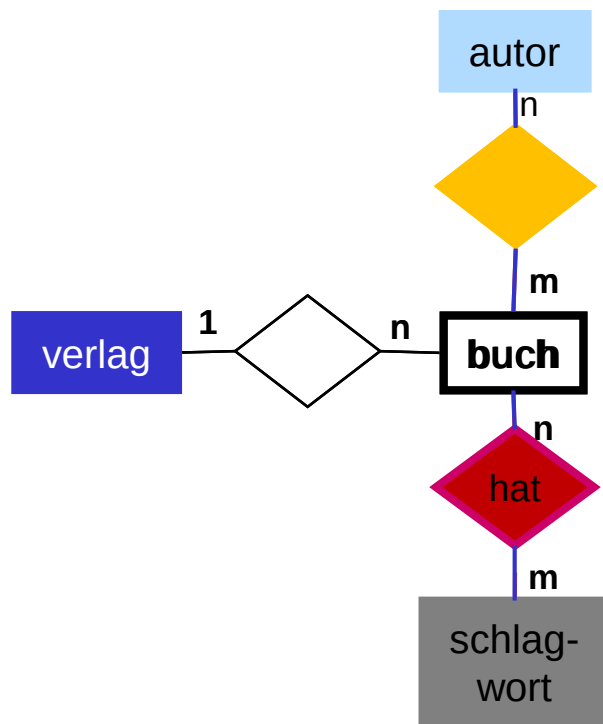
```
SELECT name
FROM verlag
```

Erklärung:

Es werden alle Attributwerte des Attributes "name" aus der Relation "verlag" ausgegeben. In der Reihenfolge der Zeilen ist keine Information codiert. Die Zeilen werden so ausgegeben, wie sie sich (zufällig) in der Relation befinden.

Eine Selektion der [Relationenalgebra](#) ist eigentlich eine Auswahl von Zeilen in der Relation. In der `WHERE`-Klausel werden logische Bedingungen angegeben. Für jede Zeile der Tabelle wird ausgewertet, ob die Bedingung erfüllt ist. Bei positivem Ergebnis wird die Zeile der Auswahlmenge hinzugefügt, andernfalls ignoriert.

TEST-DATENBANK



Verlag	<u>verlagsid</u>	name	ort
--------	------------------	------	-----

Autor	<u>autorid</u>	nachname	initialen	vornamen	zusatz
-------	----------------	----------	-----------	----------	--------

Buch_Aut	<u>buchid</u>	<u>autorid</u>	rolle	rang
----------	---------------	----------------	-------	------

Buch	<u>buchid</u>	titel	isbn	auflage	jahr	preis	waehrung	signatur	verlagsid
------	---------------	-------	------	---------	------	-------	----------	----------	-----------

Buch_SW	<u>buchid</u>	<u>swid</u>
---------	---------------	-------------

Schlagwort	<u>swid</u>	schlagwort
------------	-------------	------------

Buch_Aut.rolle kann sein: Herausgebers (H), Verfasser (V), Übersetzer (U), Mitarbeiter(M)

Buch_Aut.rang: Position des Autors in der Autorenliste (z.B. 1 für Erstautor)

Autor.zusatz: Namenszusatz wie „von“ oder „van“

Buch.signatur entspricht der Signatur in der Ifl-Bibliothek (Stand 1998)

- Mengengerüst (ca. 18.000 Sätze)
 - „Buch“ : 4873 Datensätze, „Verlag“ : 414 Datensätze
 - „Autor“ : 5045 Datensätze, „Buch_Aut“ : 5860 Datensätze
 - „Schlagwort“ : 843 Datensätze, „Buch_SW“ : 789 Datensätze

ANFRAGEMÖGLICHKEITEN IN SQL

```
select-expression ::=
    SELECT [ALL | DISTINCT] select-item+ | *
    FROM table-ref-commalist
    [WHERE cond-exp]
    [GROUP BY column-ref-commalist]
    [HAVING cond-exp]
    [ORDER BY order-item+]

select-item ::= derived-column | range-variable.*
derived-column ::= scalar-exp [AS column]
order-item ::= column [ ASC | DESC ]
```

- **SELECT** –Klausel spezifiziert auszugebende Attribute
 - mit **SELECT** * werden alle Attribute der spezifizierten Relation(en) ausgegeben
- **FROM**-Klausel spezifiziert zu verarbeitende Objekte (Relationen, Sichten)
- **WHERE**-Klausel kann eine Sammlung von Suchprädikaten enthalten, die mit **NOT**, **AND** und **OR** verknüpft sein können
 - dabei sind Vergleichsprädikate der Form $A_i \theta a_i \in W(A_i)$ bzw. $A_i \theta A_j$ möglich mit $\theta \in \{=, <>, <, \leq, >, \geq\}$

EINFACHE SELEKTIONEN UND PROJEKTIONEN

Q1: Welche (Berliner) Verlage gibt es?

```
SELECT  
FROM  
WHERE
```

Q2: Welche Bücher erschienen vor 1980 in einer Neuauflage?
(Ausgabe: Titel, Jahr)

```
SELECT  
FROM  
WHERE
```


SQL Anfrage

```
SELECT Titel, Jahr
FROM Buch
WHERE jahr < 1980 and auflage > 1
```

ausführen

löschen

akt

Bibl

DB

Zwi

zeige Datensätze 1 - 11 (11 insgesamt)

Zeige:

11

Datensätze, beginnend ab

1

titel	jahr
List Processing	1968
Moderne Logik : Abriß der formalen Logik	1972
Logic and logic design	1973
Wörterbuch der Datenverarbeitung : Englisch-Deutsch	1968
Introduction to switching theory and logical design	1974
Prädikatenkalkül der ersten Stufe	1975
The theory of error-correcting codes	1977
Computer data-base organisation	1977
Grundzüge der theoretischen Logik	1972
Matrizen und ihre technischen Anwendungen	1961
Introduction to switching theory and logical design	1974

SORTIERTE AUSGABE

ORDER BY

- Klausel zur Sortierung von Ergebnissätzen
 - aufsteigende (ascending) oder absteigende (descending) Sortierung
 - Voreinstellung: ASCending
 - statt Attributnamen sind in ORDER BY-Klausel auch relative Positionen der Attribute aus Select-Klausel möglich
 - Sortierung nach mehreren Attributen möglich

```
SELECT A1,..., Ak FROM R
ORDER BY Ai ASC, ..., Am DESC
```

Q3: wie **Q2**, jedoch sortiert nach Jahr (absteigend), Titel (aufsteigend)

```
SELECT titel, jahr
WHERE jahr < 1980 and auflage > 1
FROM Buch
ORDER BY
```



DUPLIKATELIMINIERUNG

- SELECT **DISTINCT** erzwingt Duplikateliminierung
 - Default-mäßig werden Duplikate in den Attributwerten der Ausgabe nicht eliminiert (**ALL**)

Q4: Welche Verlagsorte gibt es?

```
SELECT  
FROM Verlag
```



UMBENENNUNGEN

- Benennung von Ergebnis-Spalten
 - Umbenennung von Attributen (AS optional, DBMS abhängig)
 - Vergabe von Attributnamen für Texte und Ausdrücke

```
SELECT A [AS] B, (num_value/2) [AS] B FROM R
```

- Umbenennung von Tabellen (FROM-Klausel)
 - Einführung sogenannter Alias-Namen bzw. Korrelationsnamen
 - Entscheidend um gleichnamige Attribute aus unterschiedlichen Tabellen zu adressieren → Namenskollision

```
SELECT * FROM R [AS] NEW_R
```



SKALARE FUNKTIONEN: CASE

- Ausgabewert abhängig von verschiedenen Fällen, die durch boolesche Ausdrücke C1,...Cn dargestellt werden
- fehlender ELSE-Zweig: NULL-Wert für sonstige Fälle

```
SELECT CASE
    WHEN C1 THEN a1
    ...
    WHEN C2 THEN a2
    ELSE default END AS case_attr
FROM R
```

Beispiel:

```
SELECT titel, jahr, CASE
    WHEN jahr > 2008 THEN 'Aktuell'
    WHEN jahr > 1991 THEN 'Mittel'
    ELSE 'Veraltet' END AS aktualitaet
FROM Buch
```



WEITERE SKALARE FUNKTIONEN (AUSWAHL)

String-Funktionen

- `||` (String-Konkatenation), `CHAR_LENGTH`, `BIT_LENGTH`
- `SUBSTRING` *Bsp.:* `SUBSTRING (Name FROM 1 FOR 20)`
- `POSITION`, `LOWER`, `UPPER`
- `TRIM` löscht White Spaces am Anfange und am Ende eines Strings
 `TRIM(character FROM Attribute)` löscht definiertes Zeichen
 (SQL Server)

Zeit/Datumsfunktionen

- `Now()`, `CURRENT_TIME`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`
- `EXTRACT(field FROM time/date value)`

Typkonversionen

- `CAST(Expression AS datatype)`
- *Bsp.:* `CAST('2022-04-24' AS DATE)`, `CAST(20.7 AS INT)`



EINSATZ VON MENGEN-OPERATOREN

- Vereinigung (UNION), Durchschnitts- (INTERSECT) und Differenzbildung (EXCEPT) von Relationen

```
table-exp {UNION | EXCEPT | INTERSECT }[ALL] table-exp
```

- vor Ausführung werden Duplikate entfernt (außer für ALL)
- für die Operanden werden Vereinigungsverträglichkeit und gleicher/ähnlicher Datentyp gefordert
- **Q5:** Welche Schlagworte wurden nie verwendet ?

```
(SELECT swid FROM Schlagwort) EXCEPT  
(SELECT swid FROM Buch_SW)
```

JOIN-AUSDRÜCKE (1)



```
SELECT *  
FROM R, S  
WHERE R.A  $\theta$  S.A
```



- Angabe der beteiligten Relationen in FROM-Klausel
- WHERE-Klausel: Join-Bedingung sowie weitere Selektionsbedingungen
- Join-Bedingung erfordert bei gleichnamigen Attributen Hinzunahme der Relationen- oder Alias-Namen (Korrelationsnamen)
- Ohne Join-Bedingung in der WHERE-Klausel kartesisches Produkt



JOIN-AUSDRÜCKE (2)

- Join-Spezifikation direkt in FROM-Klausel (explizit)

```
join-table-exp ::= table-ref [NATURAL] [join-type] JOIN table-ref  
                [ON cond-exp | USING (column-commalist) ]  
                | table ref CROSS JOIN table-ref | (join-table-exp)  
table-ref ::= table | (table-exp) | join-table-exp  
join type ::= INNER | { LEFT | RIGHT | FULL } [OUTER]
```

- Join-Bedingung: ON cond-exp Ergebnisrelation führt Join Attribute nach Vorkommen der Attribute auf
 - Using einmalige Aufführung der Attribute
 - Natural Join zu vergleichende Attribute durch Gleichnamigkeit festgelegt und einmalige Aufführung
- Keine Angabe des join-type und nicht NATURAL → INNER
- table-ref: Relation, SELECT-Query oder bereits gejointe Relationen z.B.
$$\underbrace{(A \bowtie B)}_{\text{join-table-exp}} \bowtie C$$



JOIN-AUSDRÜCKE (3)

- Outer Joins: LEFT JOIN, RIGHT JOIN, FULL JOIN

```
SELECT schlagwort  
FROM Schlagwort LEFT OUTER JOIN Buch_SW USING (swid)  
WHERE buchid IS NULL
```

- kartesisches Produkt:

```
SELECT * FROM A CROSS JOIN B  $\Leftrightarrow$  SELECT * FROM A, B
```

JOIN-ANFRAGEN (1)

Q6: Welche Buchtitel wurden von Berliner Verlagen herausgebracht?

```
SELECT  *  
FROM    Buch B, Verlag V  
WHERE   B.verlagsid = V.verlagsid AND V.ort = 'Berlin'
```

```
SELECT  * FROM Buch NATURAL JOIN Verlag  
        WHERE ort = 'Berlin'
```

```
SELECT  * FROM Buch B JOIN Verlag V ON B.verlagsid = V.verlagsid  
WHERE   ort = 'Berlin'
```

Q7: Welche Bücher sind von Autor „Rahm“ vorhanden?

```
SELECT  titel  
FROM    Buch NATURAL JOIN Buch_Aut NATURAL JOIN Autor  
WHERE   nachname= 'Rahm'
```



JOIN-ANFRAGEN (2)

- hierarchische Beziehung auf einer Relation (PERS)
 - Beispielschema:
 - PERS (PNO int, Name, Salary, ..., MNO int, PRIMARY KEY (PNO), FOREIGN KEY (MNO) REFERENCES PERS)
- **Q8**: Finde die Angestellten, die mehr als ihre (direkten) Manager verdienen (Ausgabe: Name, Gehalt, Name des Managers)

```
SELECT  
FROM PERS  
WHERE
```

- Verwendung von Korrelationsnamen obligatorisch!

<u>PNO</u>	Name	Salary	MNO
34	Mey	32000	37
35	Schultz	42500	37
37	Abel	41000	-

<u>PNO</u>	Name	Salary	MNO
34	Mey	32000	37
35	Schultz	42500	37
37	Abel	41000	-

IN-PRÄDIKATE

```
row-constr [NOT] IN (table-exp) |  
scalar-exp [NOT] IN (scalar-exp-commalist)
```

- ein Prädikat in einer *WHERE*-Klausel kann ein Attribut oder ein Tupel von Attributen auf Zugehörigkeit zu einer Menge testen:
 - *explizite Mengendefinition*: $A_i \text{ IN } (a_1, a_j, a_k)$
 - *implizite Mengendefinition mittels Subquery*: $A_i \text{ IN } (\text{SELECT } \dots)$
- Semantik

$$x \text{ IN } (a, b, \dots, z) \Leftrightarrow x=a \text{ OR } x=b \dots \text{ OR } x=z$$
$$x \text{ NOT IN } (\text{table-exp}) \Leftrightarrow \text{NOT } (x \text{ IN } (\text{table-exp}))$$

Q9: Finde die Autoren mit Nachname Maier, Meier oder Müller

```
SELECT *  
FROM Autor  
WHERE nachname IN ('Maier', 'Meier', 'Müller')
```



GESCHACHTELTE ANFRAGEN (SUB-QUERIES)

- Auswahlbedingungen können sich auf das Ergebnis einer „inneren“ Anfrage (Sub-Query, Unteranfrage) beziehen

```
SELECT *  
FROM R  
WHERE A IN (  
    SELECT B  
    FROM S  
)
```

- innere und äußere Relationen können identisch sein
- eine geschachtelte Abbildung kann beliebig tief sein
- Join-Berechnung mit Sub-Queries
 - teilweise prozedurale Anfrageformulierung
 - weniger elegant als symmetrische Notation
 - schränkt Optimierungsmöglichkeiten des DBS ein



SUB-QUERIES (2)

– einfache Sub-Queries

- 1-malige Auswertung der Sub-Query
- Ergebnismenge der Sub-Query (Zwischenrelation) dient als Eingabe der äußeren Anfrage

```
SELECT *  
FROM R  
WHERE A IN  
      (SELECT B  
       FROM S)
```

– korrelierte Sub-Queries (verzahnt geschachtelte Anfragen)

- Sub-Query bezieht sich auf eine äußere Relation
- Sub-Query-Ausführung erfolgt für jedes Tupel der äußeren Relation
- Verwendung von Korrelationsnamen i.a. erforderlich
- „äußere“ Attribute können in der inneren Query verwendet werden

```
SELECT *  
FROM R r  
WHERE EXISTS  
      (SELECT *  
       FROM S s  
       WHERE r.A = s.B  
       )
```

besser: Join-Berechnung ohne Sub-Queries

- Sub-Queries sind nützlich zur Berechnung komplexer Vergleichswerte in WHERE-Klausel, z.B. durch Anwendung von Aggregatfunktionen in der Sub-Query

SUB-QUERIES -BEISPIEL (2)

- Q10: Welche Buchtitel wurden von Berliner Verlagen veröffentlicht?

einfache Sub-Query

```
SELECT B.titel
FROM buch B
WHERE B.verlagsid IN
      (SELECT V.verlagsid
       FROM verlag V
       WHERE V.ort = 'Berlin')
```

korrelierte Sub-Query

```
SELECT B.titel
FROM buch B
WHERE 'Berlin' IN
      (SELECT V.ort
       FROM verlag V
       WHERE V.verlagsid = B.verlagsid)
```




WEITERGEHENDE VERWENDUNG VON SUB-QUERIES

- 3 Arten von Sub-Queries
 - Table Sub-Queries (mengenwertige Ergebnisse)
 - Row Sub-Queries (Tupel-Ergebnis)
 - skalare Sub-Queries (atomarer Wert; Kardinalität 1, Grad 1)
- Table-Sub-Queries können überall stehen, wo ein Relationenname möglich ist, insbesondere in der FROM-Klausel.
 - Vergabe eines Alias für die Table-Expression

```
SELECT titel
FROM (SELECT * FROM Verlag WHERE Ort='Leipzig')
AS LVerlag NATURAL JOIN Buch
```

- skalare Sub-Queries können auch in SELECT-Klausel stehen

```
SELECT titel, (SELECT name FROM Verlag V
               WHERE V.verlagsid=B.verlagsid) AS Verlag
FROM Buch B
WHERE jahr =2001
```



BENUTZUNG VON AGGREGAT (BUILT-IN)- FUNKTIONEN

```
aggregate-function-ref ::= COUNT (*) | {AVG | MAX | MIN | SUM | COUNT}  
                        ([ALL | DISTINCT] scalar-exp)
```

- Standard-Funktionen: AVG, SUM, COUNT, MIN, MAX
 - Elimination von Duplikaten : DISTINCT
 - keine Elimination : ALL (Defaultwert)
 - Typverträglichkeit erforderlich
- Aggregatfunktionen können nicht direkt in WHERE-Klausel verwendet werden

- Einsatz von Sub-Queries

```
SELECT *  
FROM R  
WHERE k  $\theta$   
(SELECT aggregate-function-ref FROM S ...)
```

- Auswertung
 - Count(select-item) zählt keine NULL-Werte
 - Built-in-Funktion (AVG) wird angewendet auf einstellige Ergebnisliste
 - Verwendung von arithmetischen Ausdrücken ist möglich

AGGREGATFUNKTIONEN – BEISPIELE (1)

Q11: Wie viele Verlage gibt es?

```
SELECT  
FROM Verlag
```

Q11b: Wie viele Verlage und wieviel davon in Berlin gibt es?

```
SELECT  
  
FROM Verlag
```

Q12: An wie vielen Orten gibt es Verlage?

```
SELECT  
FROM Verlag
```

Q13: Für wie viele Bücher ist der Verlag nicht bekannt?

```
SELECT  
FROM Verlag
```



AGGREGATFUNKTIONEN – BEISPIELE (2)

– Aggregatfunktion in WHERE-Klausel

Q14: Welches Buch (Titel, Jahr) ist am ältesten?

```
SELECT titel, jahr
FROM Buch
WHERE jahr =(SELECT MIN(jahr)
               FROM Buch)
```

Q15: An welchen Orten gibt es mehr als drei Verlage?

```
SELECT    DISTINCT V.ort
FROM      Verlag V
WHERE     3 < (SELECT COUNT(*)
               FROM Verlag V2
               WHERE V2.ort=V.ort)
```



PARTITIONIERUNG EINER RELATION IN GRUPPEN

```
SELECT A1,...,Ak [,aggrgeate_function_ref+] FROM ...  
[WHERE ...]  
[GROUP BY column-ref-commalist]
```

– Gruppenbildung auf Relationen: GROUP-BY-Klausel

- Tupel mit übereinstimmenden Werten für Gruppierungsattribut(e) bilden je eine Gruppe
- ausgegeben werden können nur:
Gruppierungsattribute, Konstante, Ergebnis von Aggregatfunktionen (→ 1 Satz pro Gruppe)
- $\{A1, \dots Ak\} \subseteq \text{column} - \text{ref} - \text{commalist}$
- die Aggregatfunktion wird jeweils auf die Tupel einer Gruppe angewendet

A1	A2	A3
a	1	2
a	2	3
b	3	1
b	5	1
b	3	2
c	2	2

GROUP BY – BEISPIEL (1)

Q16: Liste alle Verlage (verlagsid) mit der Anzahl ihrer Bücher auf

```
SELECT  
  FROM Buch NATURAL JOIN Verlag  
  GROUP BY
```

Ausgabe von Verlagsname?

```
SELECT name  
  FROM Buch NATURAL JOIN Verlag  
  GROUP BY
```



GROUP BY – BEISPIEL

Q17: Liste alle Abteilungen und das Durchschnitts- sowie Spitzengehalt ihrer Angestellten auf.

```
SELECT DNO, AVG(Salary) AS Avg_Salary, Max(Salary) AS Max_Salary
FROM PERS
GROUP BY DNO
```

PERS

<u>PNO</u>	Name	Age	Salary	DNO
235	Schmid	31	32500	K51
123	Schulz	32	43500	K51
237	Bauer	21	21000	K53
234	Meier	23	42000	K53
829	Müller	36	42000	K53
321	Klein	19	27000	K55
406	Abel	47	52000	K55
574	Schmid	28	36000	K55

DNO	Avg_Salary	Max_Salary
K51	38000	43500
K53	35000	42000
K55	38333	52000

AUSWAHL VON GRUPPEN (HAVING-KLAUSEL)

```
SELECT ... FROM ... [WHERE ...]  
[ GROUP BY column-ref-commalist ]  
[ HAVING cond-exp ]
```

- HAVING: Bedingungen nur bezüglich Sätzen einer Gruppe
 - meist Verwendung von Aggregatfunktionen
- Fragen werden in den folgenden Reihenfolge bearbeitet:
 1. Tupel werden ausgewählt durch die WHERE-Klausel.
 2. Gruppen werden gebildet durch die GROUP-BY-Klausel.
 3. Gruppen werden ausgewählt, wenn sie die HAVING-Klausel erfüllen



HAVING-KLAUSEL (2)

- **Q18:** Für welche Abteilungen in Leipzig ist das Durchschnittsalter kleiner als 30?

```
SELECT DNO
  FROM PERS NATURAL JOIN DEPT
 WHERE ort = 'Leipzig'
  GROUP BY DNO
  HAVING AVG(AGE) < 30
```

Q15': An welchen Orten gibt es mehr als drei Verlage?

„Profi-Version“ ohne korrelierte Sub-Query

```
SELECT ort
  FROM Verlag
  GROUP BY ort
  HAVING COUNT(*) > 3
```



SUCHBEDINGUNGEN

- Sammlung von Prädikaten
 - Verknüpfung mit AND, OR, NOT
 - Auswertungsreihenfolge ggf. durch Klammern
- nicht-quantifizierte Prädikate
 - Vergleichsprädikate
 - LIKE-, BETWEEN-, IN-Prädikate
 - Test auf Nullwert
 - UNIQUE-Prädikat: Test auf Eindeutigkeit
 - MATCH-Prädikat: Tupelvergleiche
 - OVERLAPS-Prädikat: Test auf zeitliches Überlappen von DATETIME-Werten
- quantifizierte Prädikate
 - ALL
 - ANY
 - EXISTS

VERGLEICHSPRÄDIKATE

```
comparison-cond ::= row-constructor  $\theta$  row-constructor  
row-constructor ::= scalar-exp | (scalar-exp-commalist) | (table-exp)
```

- skalarer Ausdruck (scalar-exp):
 - Attribut, Konstante bzw. Ausdrücke, die einfachen Wert liefern
- Tabellen-Ausdruck (table-exp) für Row Sub-Query
 - darf hier höchstens 1 Tupel als Ergebnis liefern (Kardinalität 1)
- Vergleiche zwischen Tupel-Konstruktoren (row constructor) mit mehreren Attributen

- $(a_1, a_2, \dots, a_n) = (b_1, b_2, \dots, b_n) \Leftrightarrow a_1 = b_1 \text{ AND } a_2 = b_2 \dots \text{ AND } a_n = b_n$
- $(a_1, a_2, \dots, a_n) < (b_1, b_2, \dots, b_n) \Leftrightarrow (a_1 < b_1) \text{ OR } ((a_1 = b_1) \text{ AND } (a_2 < b_2)) \text{ OR } (\dots)$

```
SELECT ...  
WHERE ('Leipzig', 2000) = (Select Ort, Gründungsjahr FROM Verein  
... )
```

LIKE-PRÄDIKATE

```
char-string-exp [ NOT ] LIKE char-string-exp  
                [ ESCAPE char-string-exp ]
```

- Suche nach Strings, von denen nur Teile bekannt sind (pattern matching)
- LIKE-Prädikat vergleicht einen Datenwert mit einem „Muster“ („Maske“)
- Aufbau einer Maske mit Hilfe zweier spezieller Symbole
 - % bedeutet „0 oder mehr beliebige Zeichen“
 - _ bedeutet „genau ein beliebiges Zeichen“
 - das LIKE-Prädikat ist TRUE, wenn der entsprechende Datenwert der aufgebauten Maske mit zulässigen Substitutionen von Zeichen für '%' und '_' entspricht
 - Suche nach '%' und '_' selbst durch Voranstellen eines Escape-Zeichens möglich.

LIKE-PRÄDIKATE - BEISPIEL



LIKE-Klausel	Wird z.B. erfüllt von
NAME LIKE '%SCHMI%'	'H. SCHMIDT' 'SCHMIED'
ANR LIKE '_7%'	'17' 'K75'
NAME NOT LIKE '%-%'	
STRING LIKE '%__%' ESCAPE '\'	'DBS_1' 'X_Y_Z'

BETWEEN-PRÄDIKATE

```
row-constr [ NOT] BETWEEN row-constr AND row-constr
```

- Überprüfung des Enthaltenseins eines Elements y in einem Intervall $[x, z]$
- Ordnung des Intervalls datentypspezifisch

Semantik

$y \text{ BETWEEN } x \text{ AND } z \quad \Leftrightarrow \quad x \leq y \text{ AND } y \leq z$

$y \text{ NOT BETWEEN } x \text{ AND } z \quad \Leftrightarrow \quad \text{NOT } (y \text{ BETWEEN } x \text{ AND } z)$

- Beispiel

```
SELECT DNO, count(*)  
FROM PERS  
WHERE DNO NOT BETWEEN 'K50' AND 'K54'  
      GROUP BY DNO  
      HAVING AVG (Age) BETWEEN 20 AND 35
```



NULL-WERTE

- pro Attribut kann Zulässigkeit von Nullwerten festgelegt werden
 - unterschiedliche Bedeutungen: Datenwert ist momentan nicht bekannt
 - Attributwert existiert nicht für ein Tupel, z.B. akademischer Titel
- Behandlung von Nullwerten
 - das Ergebnis einer arithmetischen Operation (+, -, *, /) mit einem NULL-Wert ist der NULL-Wert
 - Tupel mit NULL-Werten im Verbundattribut nehmen am Verbund nicht teil
 - Auswertung eines NULL-Wertes in einem Vergleichsprädikat mit irgendeinem Wert ist UNKNOWN (?)
- bei Auswertung von Booleschen Ausdrücken wird 3-wertige Logik eingesetzt
 - das Ergebnis ? bei der Auswertung einer WHERE-Klausel wird wie FALSE behandelt.

NOT	
T	F
F	T
?	?

AND	T	F	?
T	T	F	?
F	F	F	F
?	?	F	?

OR	T	F	?
T	T	T	T
F	T	F	?
?	T	?	?



NULL-WERTE: PROBLEMFÄLLE

- 3-wertige Logik führt zu unerwarteten Ergebnissen
Bsp.: PERS (Age $\leq$ 50) vereinigt mit PERS (Age $>$ 50)
ergibt nicht notwendigerweise Gesamtrelation PERS
- Nullwerte werden bei SUM, AVG, MIN, MAX nicht berücksichtigt, während COUNT(*) alle Tupel (inkl. Null-Tupel, Duplikate) zählt.

SELECT AVG (Salary) FROM PERS → Ergebnis X

SELECT SUM (Salary) FROM PERS → Ergebnis Y

SELECT COUNT (*) FROM PERS → Ergebnis Z

es gilt nicht notwendigerweise, dass $X = Y / Z$

(-> ggf. Verfälschung statistischer Berechnungen)

- spezielles SQL-Prädikat zum Test auf NULL-Werte:

```
row-constr IS [NOT] NULL
```

```
SELECT PNO, PNAME  
FROM PERS  
WHERE SALARY IS NULL
```


QUANTIFIZIERTE PRÄDIKATE

– All-or-Any-Prädikat

```
row-constr  $\theta$  { ALL | ANY | SOME } (table-exp)
```

- θ **ALL**: Prädikat wird zu `true` ausgewertet, wenn der θ -Vergleich für alle Ergebniswerte von `table-exp` `true` ist.
- θ **ANY**/ θ **SOME**: analog, wenn der θ -Vergleich für mindestens einen Ergebniswert `true` ist.

```
SELECT *  
  FROM R  
 WHERE A | (A1, ..., AK)  $\theta$  ALL  
    (SELECT B | (B1, ..., BK)  
     FROM S)
```

EXISTENZTESTS

```
[NOT] EXISTS (table-exp)
```

- Prädikat wird `false`, wenn `table-exp` auf die leere Menge führt, sonst `true`
- im `EXISTS`-Kontext kann `table-exp` mit `(SELECT * ...)` spezifiziert werden (Normalfall)
- Semantik

$$x \in \mathbf{ANY}(\text{SELECT } y \text{ FROM } T \text{ WHERE } p) \Leftrightarrow \text{EXISTS}(\text{SELECT } * \text{ FROM } T \text{ WHERE } (p) \text{ AND } (x \in T.y))$$
$$x \in \mathbf{ALL}(\text{SELECT } y \text{ FROM } T \text{ WHERE } p) \Leftrightarrow \text{NOT EXISTS}(\text{SELECT } * \text{ FROM } T \text{ WHERE } (p) \text{ AND NOT } (x \in T.y))$$



QUANTIFIZIERTE PRÄDIKATE - BEISPIELE

- **Q19:** Finde die Manager, die mehr verdienen als alle ihre Angestellten

```
SELECT M.PNO
FROM   PERS  M
WHERE  M.Salary > ALL(SELECT P.Salary
                      FROM PERS P
                      WHERE P.MNO=M.PNO)
```

- **Q19' (EXISTS)**

```
SELECT M.PNO
FROM PERS M
WHERE NOT EXISTS (SELECT *
                  FROM PERS P
                  WHERE P.MNO=M.PNO AND
                        NOT (M.Salary > P.Salary)
                  )
```

QUANTIFIZIERTE PRÄDIKATE – BEISPIELE (2)

Q20: Finde die Schlagworte, die für mindestens ein (... **kein**) Buch vergeben wurden

```
SELECT S.*  
FROM schlagwort S  
WHERE NOT EXISTS (SELECT *  
                  FROM Buch_SW B WHERE B.swid=S.swid)
```



DIVISION

- Keine explizite Realisierung in SQL
- abgeleiteter Operator basierend auf kartesischen Produkt und Differenz
- Sei $R(E1, E2, C1, C2, C3)$ und $S(C1, C2, C3)$ Relationen, wobei $C1, C2, C3$ gemeinsame Attribute und $E1, E2$ exklusive für R
 - $R \div S = \pi_{E1, E2}(R) - \pi_{E1, E2}[(\pi_{E1, E2}(R) \times S) - R]$

E1	E2	C1	C2	C3
1	1	x	y	z
1	1	a	b	c
2	3	x	y	z

C1	C2	C3
x	y	z
a	b	c

```
SELECT R1.E1, R1.E2
FROM R R1
WHERE (E1, E2) NOT IN (
    SELECT E1, E2
    FROM ((SELECT R.E1, R.E2 FROM R) R2 CROSS JOIN S) cp WHERE
    (cp.E1, cp.E2, cp.C1, cp.C2, cp.C3) NOT IN (SELECT * FROM R)
)
```

DIVISION



– Korrelierte Subquery

```
SELECT DISTINCT R1.E1, R1.E2
FROM R R1
WHERE NOT EXISTS
```

```
(SELECT *
FROM S WHERE (S.C1, S.C2, S.C3) NOT IN
(SELECT R2.C1, R.C2, R.C3
FROM R R2
WHERE R1.E1=R2.E1 AND R1.E2=R2.E2)
)
```

für ein Ergebnistupel $e=(E1,E2)$ darf so ein gemeinsames Tupel aus S nicht existieren

Tupel aus S, die nicht in R sind bzgl. der gemeinsamen Tupel (C1,C2, C3) für ein bestimmtes exklusiv Tupel $e=(E1,E2)$

E1	E2	C1	C2	C3
1	1	x	y	z
1	1	a	b	c
2	3	x	y	z

C1	C2	C3
x	y	z
a	b	c

– GROUP BY und HAVING COUNT

```
SELECT DISTINCT R.E1, R.E2
FROM R, S
WHERE R.C1 = S.C1 AND R.C2 = S.C2 AND R.C3 = S.C3
GROUP BY R.E1, R.E2
HAVING COUNT(*) = (SELECT COUNT(DISTINCT S.C1, S.C2, S.C3) FROM S)
```

Tupel in R die Verbundpartner in S haben



DIVISION: BEISPIEL

- **Q21:** Welche Bücher umfassen alle Schlagworte, die das Buch 3545 enthält?
- $\pi_{buchid,swid}(Buch_sw) \div \pi_{swid}(\sigma_{buchid=3545}(Buch_sw))$
- Abwandlung mithilfe von NOT IN muss nicht Vereinigungsverträglichkeit beachtet werden
- $\pi_{buchid}(Buch) - \pi_{buchid}[(\pi_{buchid}(Buch) \times \pi_{swid}(\sigma_{buchid=3545}(buch_sw))) - \pi_{buchid,swid}(buch_sw)]$

```
SELECT DISTINCT b.titel
FROM Buch b WHERE b.buchid NOT IN (
    SELECT b_cross.buchid
    FROM (SELECT buchid FROM BUCH) b_cross CROSS JOIN
    (SELECT swid FROM buch_sw WHERE buchid = 3545) buch_sel_cross
    WHERE (b_cross.buchid, buch_sel_cross.swid) NOT IN
        (SELECT buchid, swid FROM buch_sw)
)
```



DIVISION: BEISPIEL

– Q21': korreliert

```
SELECT DISTINCT b.titel
FROM Buch b WHERE NOT EXISTS (
    SELECT *
    FROM buch_sw
    WHERE buchid = 3545 AND swid NOT IN
    (SELECT swid FROM buch_sw WHERE b.buchid = buch_sw.buchid)
)
```

– Q21'': GROUP BY und COUNT

```
SELECT DISTINCT b.titel
FROM Buch b NATURAL JOIN buch_sw NATURAL JOIN
(SELECT swid FROM buch_sw WHERE buchid = 3545) b_sel
GROUP BY buch_id HAVING(COUNT(swid))=
    (SELECT COUNT(DISTINCT SWID)
     FROM buch_sw WHERE buchid = 3545)
```


EINFÜGEN VON TUPELN (INSERT)

```
INSERT INTO table[(column-commalist)]  
    {VALUES row-constr-commalist|  
    table-exp|  
    DEFAULT VALUES}
```

- satzweises Einfügen (direkte Angabe der Attributwerte)
- Bsp.: Füge Autor Garfield mit Autorid 4711 ein

```
INSERT INTO Autor(autorid, vorname)  
    VALUES (4711, 'Garfield')
```

- alle nicht angesprochenen Attribute erhalten Nullwerte
- falls alle Werte in der richtigen Reihenfolge versorgt werden, kann Attributliste entfallen (NICHT zu empfehlen)
- Integritätsbedingungen müssen erfüllt werden



INSERT (2)

- mengenorientiertes Einfügen: einzufügende Tupeln werden mit Hilfe einer SELECT-Anweisung ausgewählt
- *Bsp.: Füge die Verlage aus Leipzig in Relation TEMP ein*

```
INSERT INTO TEMP
      SELECT *
      FROM Verlag
      WHERE ort='Leipzig'
```

- (leere) Relation TEMP mit kompatiblen Attributen sei vorhanden
 - Reihenfolge der Wertebereiche der selektierten Attributwerte muss kompatibel mit Wertebereichen der Attribute der Relation sein
- spezifizierte Tupelmenge wird ausgewählt und in Zielrelation kopiert
- die eingefügten Tupel sind unabhängig von denen, von denen sie abgeleitet/kopiert wurden
- Einfügen von Werten für bestimmter Attribute mittels table-expression erfordert Spezifikation der Attribute in die einzufügende Relation

ÄNDERN VON TUPELN (UPDATE)

```
searched-update ::=      UPDATE table
                        SET update-assignment-commalist
                        [WHERE cond-exp]
update-assignment ::= column = {scalar-exp|DEFAULT|NULL}
```

Gib den Mitarbeitern der Abteilung „K56“ eine Gehaltserhöhung von 2%

```
UPDATE PERS  SET Salary=Salary * 1.02
WHERE DNO='K56'
```

Stelle die DM-Preise für die nach 1997 erschienenen Bücher auf Euro um (Verhältnis 1:1,95)

```
UPDATE Buch
SET preis=preis/1.95, SET waehrung='Euro'
WHERE jahr > 1997 AND waehrung='DM' AND
      preis IS NOT NULL
```

LÖSCHEN VON TUPELN (DELETE)

searched-delete ::= **DELETE** FROM table [WHERE cond-exp]

- Aufbau der WHERE-Klausel entspricht dem der SELECT-Anweisung.
 - Lösche den Autor mit der Autorid 4711

```
DELETE FROM Autor  
WHERE autorid=4711
```

- Lösche alle Schlagworte, die nicht verwendet werden.

```
DELETE FROM Schlagwort  
WHERE swid NOT IN (SELECT swid FROM Buch_SW)
```



MERGE (SEIT SQL:2003)

- Kombination von Insert/Update/Delete beim Mischen von Tabellen

```
MERGE INTO table1 USING table2 | table-exp ON cond-expr
WHEN MATCHED THEN
    UPDATE SET update-assignment-commalist [WHERE cond-expr]
    [ DELETE WHERE cond-expr ]
WHEN NOT MATCHED THEN
    INSERT (column-commalist) VALUES row-constr-commalist
    [WHERE cond-expr]
```

- Unterscheidung zwischen bereits vorkommenden Werten und neuen
- Ändern oder Löschen der existierenden Werte table1 mit Werten aus table2 (Einschränkung der zu ändernden/löschenden Tupel durch WHERE-Bedingung)
- Bedingtes (WHERE cond-expr) Einfügen der neuen Tupel aus table2



MERGE - BEISPIEL

Course	Task	Stud	Credits
C1	1	S1	4
C1	2	S1	5
C1	1	S2	3
C1	2	S2	5

Course	Task	Stud	Credits
C1	1	S1	5
C1	2	S1	5
C1	1	S2	3
C1	2	S2	6
C1	3	S2	3

```
MERGE INTO CREDITS AS C USING NEW_CREDITS as N
  ON C.Course = N.Course AND C.Task = N.Task AND
  C.Stud = N.Stud
WHEN MATCHED THEN UPDATE SET Credits = N.Credits
WHEN NOT MATCHED THEN INSERT VALUES(N.Course, N.Task,
N.stud, N.Credits)
```



RELATIONENALGEBRA VS. SQL (RETRIEVAL)

Relationenalgebra	SQL
$\pi_{A1, A2, \dots A_k} (R)$	SELECT DISTINCT A1, A2, ... Ak FROM R
$\sigma_P (R)$	SELECT * FROM R WHERE P
$R \times S$	SELECT * FROM R,S bzw. FROM R CROSS JOIN S
$R \bowtie_{R.A \theta S.B} S$	SELECT * FROM R,S WHERE R.A θ S.B bzw. SELECT * FROM R JOIN S ON (R.A θ S.B)
$R \bowtie S$	SELECT * FROM R NATURAL JOIN S
$R \Join S$	SELECT * FROM R LEFT JOIN S
$R \Join S$	SELECT * FROM R FULL JOIN S
$R \ltimes S$	SELECT R.* FROM R NATURAL JOIN S
$R \cup S$	SELECT * FROM R UNION SELECT * FROM S
$R \cap S$	SELECT * FROM R INTERSECT SELECT * FROM S
$R - S$	SELECT * FROM R EXCEPT SELECT * FROM S
$R \div S$	SELECT ... FROM R WHERE NOT EXISTS (SELECT *...)



ZUSAMMENFASSUNG

- SQL wesentlich mächtiger als Relationenalgebra
- Hauptanweisung: SELECT
 - Projektion, Selektion, Joins
 - Aggregatfunktionen
 - Gruppenbildung (Partitionierungen)
 - quantifizierte Anfragen
 - Unteranfragen (einfache und korrelierte Sub-Queries)
 - allgemeine Mengenoperationen UNION, INTERSECT, EXCEPT
- Datenänderungen: INSERT, UPDATE, DELETE
- hohe Sprachredundanz
- SQL-Implementierungen weichen teilweise erheblich von Standard ab (Beschränkungen / Erweiterungen)